

Votegral: Towards Usable, End-to-End Verifiable, Coercion-Resistant Online Voting

Presented on 5th December 2025

School of Computer and Communication Sciences
Decentralized and Distributed Systems Lab
Doctoral program in Computer and Communication Sciences

for the award of the degree of Docteur ès Sciences (PhD)

by

Louis-Henri Manuel Jakob MERINO

Accepted on the jury's recommendation

Prof. T. E. Bourgeat, jury president
Prof. B. A. Ford, thesis director
Dr O. Spycher, examiner
Prof. D. Levin, examiner
Prof. C. Pit-Claudel, examiner

Live as if you were to die tomorrow.
Learn as if you were to live forever.
— Mahatma Gandhi

To my wife, parents and family...

Acknowledgements

First and foremost, I am immensely grateful to my PhD advisor, Prof. Bryan Ford. The support and kindness he shows to me and to everyone in the lab are beyond words. He gave me the opportunity to work on projects that I am incredibly passionate about, especially those revolving around societal technologies. Over time, I came to realize how rare this freedom is. His unwavering dedication to impactful research has been an example to us all and has deeply shaped how I think about my own work.

I am also incredibly grateful to my Cyber Defence Campus advisor, Prof. Bernhard Tellenbach, for his guidance, support, and many insightful discussions throughout this project. He was always willing to help, and his perspective was invaluable in keeping the work grounded and practically relevant.

Next, I would like to thank my undergraduate advisors, Prof. Michel Cukier and Prof. Dave Levin, who supported me in becoming a PhD candidate. I learned a tremendous amount through the UMD ACES program with Prof. Michel Cukier and Prof. Bertrand Sobesto, and through conducting censorship research with Prof. Dave Levin and Dr. Kevin Bock. Their guidance and encouragement set me on the path that ultimately led to this dissertation.

I also want to thank the former and current engineers in our lab, in particular Linus Gasser and Pierluca Borsò-Tan, as well as the administrative assistants Margaret Church, Sandra Hünsch, and Fara Raharisolo. Their support throughout my PhD made an enormous difference, both in day-to-day work and in navigating more difficult periods, and they ensured that the lab ran smoothly behind the scenes.

I am also grateful to my fellow PhD students in the lab, in particular Simone Colombo, Shailesh Mishra, Pasindu Tennage, Haoqian Zhang, and Enis Ceyhun Alp, whose discussions, feedback, and camaraderie have made this journey both more rewarding and intellectually stimulating. I am grateful to my defense jury for their time, feedback, and thoughtful discussion. Their questions and comments helped sharpen the final version of this work.

I would like to thank the study participants for their invaluable insights and for generously giving their time. A large part of this dissertation would not have been possible without their willingness to engage with our system and share their experiences.

I am also grateful for the financial support that made this work possible, including the Swiss Cyber Defence Fellowship from armasuisse, and the Swiss Government Excellence / Fulbright Scholarship, which supported my move to EPFL and first introduced me to DEDIS. I also thank EPFL for providing an excellent research environment and Switzerland for creating such an inviting place to live and work.

Acknowledgements

Next, I want to thank my parents and brother for their love, encouragement, and relentless support throughout this long journey. They have helped shape who I am today, and I am deeply grateful for the many sacrifices they have made along the way.

And finally, my deepest gratitude goes to my wife, for her unwavering support for more than twelve years. Her confidence in me, even when I doubted myself, has meant more than I can express; I truly would not be where I am today if it were not for her. She brings immense love, patience, and happiness into our lives and has stood by me through every high and low of this process. I am especially grateful for her patience and understanding as we navigated long-distance so that each of us could pursue our respective career paths. This dissertation is as much hers as it is mine.

Lausanne, 24 October, 2025

Louis-Henri Merino

Abstract

Online voting promises greater convenience and accessibility, but moving from supervised polling places to unsupervised settings magnifies the risk of coercion and vote buying. A compelling strategy is to give voters fake credentials: credentials that look and behave like real voting credentials but whose ballots are silently excluded from the tally. Despite its conceptual appeal, practical realizations and usability evidence for fake credentials have remained limited.

This dissertation presents Votebral, the first end-to-end verifiable, coercion-resistant online voting system with empirical evidence towards practical usability. Votebral has two components: TRIP and VLT. TRIP is a trust-limited, in-person registration scheme that issues voters a real credential and any number of fake credentials on paper, without trusted hardware. TRIP embeds an interactive zero-knowledge proof into the physical printing process so that real credentials carry sound proof transcripts while fake credentials carry identically formatted but unsound proof transcripts — distinguishable only by the voter during issuance and not transferable thereafter.

VLT is a tallying scheme that constrains ballots to registrar-issued credentials to enable linear-time filtering of fake ballots. VLT also introduces *standing votes*: a voter facing extreme coercion can, at registration, delegate their voting rights to a publicly registered political party and leave the booth with only fake credentials. Tallying then credits the party's ballot by the number of such delegations and publishes publicly auditable proofs, resulting in both transparency and coercion evidence — evidence that an aggregate number of voters felt unsafe to leave the registrar with a real credential.

Our prototype tallies 1 million ballots in about 14 hours on a 128-core, 256 GB RAM machine; this puts Votebral on par with modern end-to-end verifiable systems such as Swiss Post, while significantly outperforming prior JCJ-style systems such as Civitas. TRIP's end-to-end, voter-observable registration session completes in under 20 seconds on resource-constrained hardware. In our main user study with 150 demographically diverse participants recruited in Boston, Massachusetts, 83% successfully registered and cast a ballot in our mock election. Among the 120 participants exposed to fake credentials, 96% correctly understood the purpose of fake credentials. These promising results suggest a path for practical viability of coercion-resistant, end-to-end verifiable online voting using fake credentials.

Résumé

Le vote en ligne promet une plus grande commodité et accessibilité, mais passer de bureaux de vote supervisés à des contextes non supervisés amplifie le risque de coercition et d'achat de votes. Une stratégie convaincante consiste à fournir aux électeurs de faux crédentiels : des crédentiels qui ressemblent et se comportent comme de véritables crédentiels de vote, mais dont les bulletins sont silencieusement exclus du dépouillement. Malgré son attrait conceptuel, les concrétisations pratiques et les éléments de preuve d'utilisabilité des faux crédentiels demeurent limités.

Cette thèse présente Votebral, le premier système de vote en ligne vérifiable de bout en bout et résistant à la coercition, avec des preuves empiriques en faveur d'une utilisabilité pratique. Votebral comporte deux composants. TRIP est un schéma d'enregistrement en personne à confiance limitée qui délivre à l'électeur un credentiel réel et autant de faux crédentiels sur papier, sans matériel d'enregistrement de confiance. TRIP intègre une preuve interactive à connaissance nulle dans le processus d'impression physique, de sorte que les crédentiels réels portent des transcrits de preuve valides tandis que les faux crédentiels portent des transcrits au format identique mais non valides — distinguables uniquement par l'électeur au moment de l'émission et non transférables par la suite.

VLT est un schéma de dépouillement qui restreint les bulletins aux crédentiels délivrés par le registraire afin de permettre l'élimination en temps linéaire des bulletins factices. VLT introduit également les votes « permanents » : un électeur confronté à une coercition extrême peut, lors de l'enregistrement, déléguer son vote à un parti politique inscrit publiquement et quitter l'isoloir avec uniquement de faux crédentiels. Le dépouillement crédite alors le bulletin unique du parti du nombre de telles délégations et publie des preuves vérifiables publiquement, offrant à la fois transparence et « traces » de coercition — l'indication qu'un nombre agrégé d'électeurs ne se sentaient pas en sécurité pour emporter un credentiel réel hors de l'isoloir.

Notre prototype décompte 1 million de bulletins en environ 14 heures sur une machine 128 cœurs et 256 Go de RAM ; cela place Votebral au niveau des systèmes modernes vérifiables de bout en bout tels que Swiss Post, tout en surpassant nettement les systèmes de type JCJ antérieurs comme Civitas. La session d'enregistrement de bout en bout observable par l'électeur avec TRIP s'achève en moins de 20 secondes sur du matériel contraint. Dans une étude principale auprès de 150 participants divers recrutés à Boston (Massachusetts), 83 % se sont enregistrés et ont voté avec succès dans notre élection simulée. Parmi les 120 participants exposés aux faux crédentiels, 96 % ont correctement compris leur finalité. Ces résultats

Résumé

prometteurs suggèrent la viabilité pratique d'un vote en ligne vérifiable de bout en bout et résistant à la coercition fondé sur des faux crédentiels.

Contents

Acknowledgements	i
Abstract (English/Français)	iii
1 Introduction	1
1.1 Contributions	4
I Background and Related Work	5
2 Voting Systems	7
2.1 The Baseline Standard	7
2.1.1 In-Person Voting	7
2.1.2 Postal Voting	8
2.2 Electronic Voting Systems	8
2.2.1 Electronic Voting Architecture	8
2.2.2 E2E Verifiability: What “Transparency” Means	9
2.3 Coercion-Resistant Electronic Voting Systems	10
2.3.1 Coercion-Evasion Strategies	10
2.4 Open Challenges	11
2.4.1 The Materialization and Usability of Fake Credentials	11
2.4.2 The Verifiability of Credentials	11
2.4.3 Prohibitive Cost of Tallying	12
3 Standardized Usability Metrics	15
3.1 System Usability Scale	15
3.2 User Experience Questionnaire	16
4 Cryptographic Primitives	19
4.1 Distributed Key Generation Scheme	19
4.2 ElGamal Encryption Scheme	19
4.3 Signature Scheme	20
4.4 Zero-Knowledge Proofs	20
4.4.1 Σ -protocols	21
4.5 Hash functions	22
	vii

Contents

4.6	Message Authentication Code.	22
4.7	Verifiable Mixnet (Shuffle)	22
4.8	Verifiable Distributed Deterministic Tags	23
4.9	Verifiable Threshold ElGamal Decryption	24
II	Votegral System	27
5	System Overview	29
5.1	System Properties	29
5.2	System and Threat Models	30
5.2.1	System Model	30
5.2.2	Threat Model	32
5.3	Protocol Overview	34
6	TRIP: Trust-Limited Coercion-Resistant In-Person Voter-Registration	39
6.1	Design Overview	39
6.1.1	Interactively Proving Real Credentials Real	39
6.1.2	Physically Materializing Usable Credentials	40
6.1.3	Voter-Facing Design	42
6.1.4	Additional Design Considerations	43
6.2	The TRIP Registration Protocol	45
6.2.1	Setup.	46
6.2.2	Voter Check-In and Kiosk Authorization	47
6.2.3	Interactive Credential Creation	47
6.2.4	Check-Out.	50
6.2.5	Activation.	51
6.3	Security Analysis and Formal Proofs	52
6.3.1	Individual Verifiability	53
6.3.2	Coercion-Resistance Proof	69
6.3.3	Privacy	76
6.4	Implementation	77
6.5	Evaluation	78
6.5.1	Experimental Setup and Benchmarks	78
6.5.2	Voter-Observable Registration Latencies	79
6.5.3	Registration Performance Across Systems	81
6.6	Discussion	81
6.6.1	Mitigating Credential Exposure	81
6.6.2	Side Channel Attacks and Defenses	82
7	VLT: A Verifiable Coercion-Resistant Tallying Scheme with Standing Votes	85
7.1	Design Overview	85
7.1.1	Linear-Time Filtering	85
7.1.2	Standing Votes	86

7.1.3	Design Goals	88
7.2	Scheme	88
7.2.1	Registration	89
7.2.2	Voting	89
7.2.3	Tally	90
7.3	Security Analysis	92
7.3.1	Adversaries and Assumptions	92
7.3.2	Voter Eligibility	94
7.3.3	Privacy	94
7.3.4	Universal Verifiability	96
7.3.5	Individual Verifiability	97
7.3.6	Coercion-Resistance	98
7.4	Scheme Instantiation & Implementation	99
7.4.1	Instantiation	100
7.4.2	Code	100
7.5	Evaluation	101
7.6	Discussion	102
7.6.1	Re-Voting	102
7.6.2	Scaling to Millions of Voters	102
7.6.3	Voting History	102
7.6.4	Voting Methods	103
III	Votegral Usability	105
8	Study Methodology	107
8.1	Study Groups	108
8.2	Ethical Considerations	109
8.3	Study Materials	110
8.4	Study Setting	116
8.5	Preliminary Studies	119
9	Study Results	121
9.1	Participants	121
9.1.1	Demographics	121
9.1.2	Perceptions & Experiences with Coercion	121
9.1.3	Trust Rating in Voting Channels	124
9.2	Usability Results	127
9.2.1	Qualitative Results	127
9.2.2	Use Errors	129
9.2.3	Perceived Usability and User Experience	132
9.2.4	Malicious Kiosk	134
9.3	Usability of Fake Credentials	135

Contents

9.3.1	Understanding Fake Credentials	135
9.3.2	Usability of Real and Fake Credentials	136
9.3.3	Credential Markings	137
9.3.4	Fake Credentials in Reality?	138
9.4	Key Takeaways and Limitations	138
 IV Conclusion and Future Work		143
 Bibliography		156
Curriculum Vitae		157

1 Introduction

Elections lie at the heart of any democracy, granting citizens the power to shape their society through peaceful expression [88, 6, 121]. The most widely accepted method to run an election today is in-person voting with hand-marked paper ballots. Inside a private booth, voters are free to mark their true choices while independent observers monitor both the casting and tallying of the ballots.

In practice, however, in-person voting imposes real burdens: long queues [38, 125], closures of polling places [112], and intimidation near voting sites [125]. It can be especially difficult for citizens abroad, including expatriates and military personnel [111]. During crises such as pandemics or wars, voting in-person may even be unsafe [123]. Remote electronic (online) voting promises greater convenience by allowing people to vote from their own devices, from anywhere [81, 63]. Convenience, in turn, can translate into broader participation and outcomes that better reflect the electorate’s will.

Yet, online voting is not simply “online banking with ballots”. To sustain public trust, an election system must be publicly auditable while preserving ballot secrecy: anyone — especially the losing parties — should be able to check that the outcome reflects the cast ballots, and no one — including the election authority — should be able to learn how any individual voter voted. Banks, on the other hand, can operate opaquely and access private individual data; election systems cannot. Moreover, online voting amplifies the risk of coercion and vote buying [125, 23, 46, 44, 41, 100, 98]. An abusive spouse might supervise their partner while voting; organized actors may attempt vote buying at scale, as reported in Moldova [98]; gangs may confiscate identity cards from select people to prevent voting, as seen in El Salvador [23]. Unlike a bank, an election authority cannot “call a voter” to verify intent without violating secrecy.

Addressing coercion is difficult. Early approaches focused on receipt-freeness — preventing voters from obtaining a transferable receipt proving how they voted [105, 17, 53]. Despite their promise, receipt-free systems succumb to practical attacks [56]: forced abstention (preventing a voter from voting), randomization (forcing an invalid vote), and simulation (obtaining

Chapter 1. Introduction

voting materials and acting as the voter). These attacks exploit the link between a voter and their encrypted ballot — information needed to prevent double voting but that enables compliance tests. These limitations motivated coercion-resistant schemes, which let voters plausibly deny compliance, and importantly, prevent even a willing voter from proving that they complied [56, 26, 73, 6].

A prominent coercion-resistant strategy gives voters the ability to have fake credentials [56, 26, 40, 37]. Fake credentials are indistinguishable from real ones to third parties, but ballots cast with them are silently excluded from the tally. A coerced voter can thus appear compliant — handing over credentials and voting under supervision — while retaining the ability to cast their intended vote with a separate real credential. Turning this strategy into a practical system raises three questions: How should such credentials be issued securely in the real world? Will voters understand and be able to use fake credentials correctly? How can a voter reliably distinguish their real credential from their fake ones while a coercer cannot?

Efforts to make fake credentials practical often make strong and arguably unrealistic assumptions that all voters have special trusted hardware, such as expensive smart cards, at registration [26, 40, 37, 82]. In practice, however, a “duress PIN” does not prevent a coercer from simply confiscating the device to prevent voters from even casting a vote (forced abstention). Other approaches either compromise transparency by assuming a fully trusted registrar [56] or require voters to coordinate across multiple registration tellers [26], a usability challenge that has yet to be studied.

This dissertation presents Votebral, the first end-to-end verifiable and coercion-resistant on-line voting system with evidence towards potential usability. Votebral consists of two schemes. TRIP is a trust-limited, in-person registration scheme that issues verifiable credentials on paper. VLT is a tallying scheme that counters ballot flooding and introduces “standing votes” for voters under extreme coercion — voters who cannot safely access an unsupervised device — to continue participating in the democratic process.

TRIP addresses three key challenges: 1) verifiably issuing real voting credentials without requiring voters to have a trusted personal device at issuance; 2) issuing fake credentials indistinguishable to third parties yet distinguishable to the voter; and 3) materializing credentials so they are usable and inexpensive. VLT addresses two further challenges: 1) efficiently filtering fake ballots without quadratic cost, and 2) supporting extreme coercion scenarios where voters cannot conceal a real credential or access an uncompromised device.

TRIP leverages an in-person process where voters interact with a kiosk in a privacy booth, a coercion-free environment to create their real and fake credentials. Unlike in-person voting, registration can be scheduled at the voter’s convenience and voting credentials can be reused across elections. To address the first challenge of issuing verifiable real credentials, the kiosk engages the voter in an interactive zero-knowledge proof (IZKP) materialized in the physical printing process. By observing a distinct sequence of printing steps, the voter is convinced of their real credential’s authenticity without needing to understand the underlying

cryptography. To address the second challenge of creating fake credentials, the kiosk follows a visibly different process to forge a *false* IZKP. The final printed credentials appear identical, but since the printing steps are visibly different, the voter knows which one is real and which ones they can safely surrender to others. To address the third challenge of materializing usable and inexpensive credentials, credentials are created using only paper. Paper credentials are inexpensive and device-agnostic, making it easy for voters to create a fake credentials, including for reasons other than coercion risks. Voters can activate credentials on any device they choose, including a trusted friend’s device if their own is under the control of a coercer. Printed envelopes supplied to voters in the booth form a part of each credential, simplifying the voter’s task of choosing a random challenge for each IZKP, and serving to conceal sensitive secret keys during the transport of one’s credentials.

To address fake-ballot flooding, VLT leverages TRIP’s natural constraint: because voters can spend only a finite amount of time in a booth and each credential may cast at most one ballot, the overall number of ballots is bounded by the number of issued credentials (real and fake). A major advantage of this approach is that it enables the safe use of deterministic tags [113, 122, 43] that enable a practical linear-time filtering step, as shown in Koenig et al.’s scheme [61]. This is in comparison to other coercion-resistant voting systems, such as Juels–Catalano–Jakobsson (JCJ) and Civitas, which have a quadratic-time filtering step [56, 26]. Unlike Koenig et al.’s scheme [61], our design avoids involving talliers during the voting phase: the ledger alone applies ballot admissibility rules, aligning with real deployments where at least one tallier is airgapped to harden vote privacy (e.g., Swiss Post [94, 117]).

To address extreme coercion — voters unable to access an unsupervised device, or are at risk of an immediate post-registration search — VLT introduces standing votes. At registration, in the booth, such voters can delegate their “voting rights” to a certified trusted entity (e.g., political party), allowing the voter to leave the booth with only fake credentials. By policy, no standing-vote receipt is issued, so no transferable evidence is created. This design sacrifices individual verifiability for those voters: they cannot perform the necessary checks to verify correctness. However, if a voter cannot safely access an unsupervised device to activate a real credential, those checks would be unattainable in any case, with their real credential being at risk from being confiscated. During tallying, after verifiably unlinking ballots and credentials from voters, deterministic tags count delegations per party and tie them to the party’s ballot. Importantly, VLT publicly reports, for each entity, the number of delegations and the party’s selected choice, enabling delegating voters to audit party behavior and provide coercion-evidence [49]. Such public evidence of distressed voters willing to forego having a real credential can help the electorate and authorities recognize and address coercion.

We implemented a prototype of Votebral in 4,449 lines of Go [78], and 424 lines of C. We evaluated it against three state-of-the-art e-voting systems: (1) Civitas, an end-to-end verifiable and coercion-resistant system based on fake credentials [26], (2) the Swiss Post’s verifiable but non-coercion-resistant system [117], and (3) VoteAgain, a coercion-resistant but not end-to-end verifiable system based on deniable re-voting [73]. Votebral’s end-to-end latency is

Chapter 1. Introduction

comparable to Swiss Post and VoteAgain and significantly improves upon Civitas. In particular, tallying one million ballots takes approximately 14 hours on a 128-core, 256GB RAM machine, whereas we estimate Civitas would take on the order of 1,768 years under the same hardware. For TRIP, we implement the use of peripherals to determine TRIP’s voter-observable latency across several setups: (1) a Point of Sale Kiosk, (2) a Raspberry Pi 4, (3) a MacBook Pro and (4) a Mini PC. We find that TRIP’s voter-observable latency is slowest on the Kiosk at 19.7 seconds and fastest on the MacBook Pro at 15.8 seconds, both suitable inside a booth where voters typically spend a few minutes.

We also conducted systematic user studies with 227 participants. In our main study of 150 demographically diverse participants in Boston, Massachusetts, Votebral achieved an 83% success rate and a System Usability Scale score of 70.4 – above the industry average of 68. Participants found Votebral as usable as a simplified variant with only real credentials. Moreover, 47% of participants who received security education detected and reported a misbehaving kiosk, versus 10% without security education.

Roadmap. The dissertation is structured as follows. Part 1 offers an overview of related work, the cryptographic primitives employed by Votebral, and the metrics used to assess the usability of Votebral. Part 2 presents the Votebral system, detailing the registration and tallying components together with our implementation and evaluation, along with security proofs. Part 3 presents the user study on Votebral, presenting the methodology and study’s results.

1.1 Contributions

- Votebral, the first concrete, end-to-end verifiable and coercion-resistant online voting system built on a paper-based, in-person registration process.
- The first use of paper transcripts of interactive zero-knowledge proofs in an online voting system to deliver verifiable voting materials without trusted personal hardware, while allowing voters to distinguish real from fake credentials *only* via the in-booth interaction order (non-transferable outside the booth).
- The first end-to-end verifiable, JCJ-style system that introduces standing votes as a measure for voters facing extreme coercion, and with auditable delegation counts that enables coercion-evidence.
- Security proofs demonstrating that TRIP achieves individual verifiability (including a strong iterative form bounding at-scale manipulation) and coercion-resistance, along with proof sketches that VLT achieves end-to-end verifiability and coercion-resistance.
- An implementation and evaluation of Votebral with measurements across multiple hardware platforms for TRIP and multiple state-of-the-art voting systems for VLT.
- The first study to systematically assess whether voters find fake-credential-based coercion-resistant online voting understandable, usable, and trustworthy, and whether a voter-verifiable credentialing workflow enables detection of a misbehaving kiosk.

Background and Related Work

Part I

2 Voting Systems

Elections serve two mandates that are often in tension: they must be transparent enough to convince the electorate of the outcome — in particular the losing party — while simultaneously protecting voters against retaliation, coercion and vote buying. This chapter surveys how modern voting systems balance these mandates in research and in practice.

We structure this chapter around three essential properties of any election system:

- Transparency through verifiability: enabling observers (and voters) to observe/verify the tallying process of the cast ballots that produced the announced outcome [42, 88, 17, 14, 56].
- Ballot secrecy: deterring retaliation, vote buying and coercion, in contrast to elections engineered to support predetermined outcomes [124, 126, 44, 98].
- Usability: ensuring that the full electorate — including voters with disabilities — can successfully participate [97, 74, 4, 62].

2.1 The Baseline Standard

2.1.1 In-Person Voting

Hand-marked paper ballots cast in person remain the only approach with near-universal expert acceptance for high-stakes elections. Paper is human-readable, immutable, and supports risk-limiting audits [71, 19]. A typical workflow has a voter check-in, receive a ballot, mark it in a privacy booth, and deposit it into a ballot box observed by officials and party observers. While this process is by no means perfect — for example, “ballot selfies” or covert recordings can undermine secrecy [16] — the mere presence of officials and a chain-of-custody of ballots provides both psychological deterrence and verifiability.

Ballot marking devices

Ballot marking devices (BMDs) improve voter accessibility and usability by providing voters with text-to-speech functionality, multi-language support, and error-free markings while preserving a human-verifiable artifact: the ballot. In principle, voters are expected to then inspect BMD-printed ballots before casting. This aligns with Rivest’s notion of software independence, where undetected software changes cannot cause undetectable changes in the election outcome [99]. In practice, however, voter verification is imperfect: absent prompts or structured review, most voters fail to notice altered selections [20]. Furthermore, BMDs do not provide non-repudiable evidence of a malicious selection and to ensure ballot secrecy, BMDs do not change the need to attend a controlled, in-person polling place.

2.1.2 Postal Voting

The inconvenience and sometimes impracticality of in-person voting create pressure to adopt remote processes, such as postal voting, which allows voters to vote from home. Because the election authority cannot control arbitrary locations, however, remote voting compromises ballot secrecy and hence resistance to coercion or vote-buying. The risks are not merely theoretical: absentee ballot fraud in North Carolina altered election outcomes [48, 118]. Despite these risks, the need for convenience has led some countries, such as Switzerland, to adopt postal voting as the normal case [72]. The United States, one of the largest democratic countries, has also seen a growing percentage of mail-in ballots compared to the pre-pandemic era [3]. Pandemics and other crisis-level events also showcase the need for a secure remote voting alternative [123].

2.2 Electronic Voting Systems

Remote electronic (online) voting — casting ballots on personal devices over the Internet — promises convenience and accessibility but introduces two fundamental challenges. First, similar to postal voting, voters can be compelled to follow arbitrary instructions from a coercer (e.g., to abstain or vote for a certain option). Second, voters’ devices may be compromised, acting as a man-in-the-middle between the voter’s actions and the voting server. Some examples that these risks make possible or further enable are:

- Household or spousal supervision and telecoercion [77].
- Organized coercion, including confiscation of voting materials [98, 23].
- Market-driven vote buying on a smart contract deployed on a decentralized network where little or no recourse is possible [10].

2.2.1 Electronic Voting Architecture

Electronic systems typically share three distinct phases: setup, voting, and tallying.

Setup. During setup, the election authority establishes the roster of eligible voters and the rules governing each ballot, then prepares system parameters and a public ledger $\mathbb{L}$. Depending on jurisdiction, voting materials may leverage existing identity materials (e.g., smart cards, as in Estonia’s online voting system [54]) or be election-specific (e.g., codes, as in Swiss Post’s online voting system [94]).

Voting. When casting a vote, the voter’s device encrypts their vote on-device and posts the resulting ballot to $\mathbb{L}$, typically accompanied by proofs that attest to the ballot being well-formed.

Tallying. After the voting period ends, the system tallies the submitted ballots. Works mainly use one of two distinct cryptographic operations. Mixnet-based tallying verifiably shuffles ciphertexts to break any link to the casting voter, then decrypts and proves the correctness of the shuffle and decryption [80, 50, 56]. Homomorphic tallying aggregates encrypted votes and proves a correct decryption of the aggregate without individually decrypting ballots [7, 17, 66]. Once tallying is complete, the outcome is computed based on the decrypted content.

2.2.2 E2E Verifiability: What “Transparency” Means

Achieving end-to-end verifiability means that anyone can check, from public evidence, that the election outcome reflects the electorate’s cast ballots, and that each honest voter can check that their own ballot was correctly cast. Concretely, end-to-end verifiability decomposes into individual and universal verifiability. Individual verifiability is tailored to convincing voters that the vote they cast was properly recorded and tallied, while universal verifiability is tailored to convincing the electorate that the outcome reflects the recorded ballots. Importantly, the ledger $\mathbb{L}$ serves as the common audit trail on which these checks rely: it is append-only and publicly accessible, so any observer can reconstruct the verification from the same transcript.

In-Person E2E systems

There exist several end-to-end verifiable systems for supervised, in-person voting; this property is not limited to remote voting systems. Prêt à Voter [58], Scantegrity II [24], and STAR-Vote [13] all issue anonymized take-home receipts that let voters verify inclusion of their ballot on a public ledger without revealing their vote. Trust therefore shifts from opaque machines (direct recording devices and ballot marking devices) to cryptography and an independent, voter-chosen verification device. Nevertheless, these systems rely on voters verifying their vote post-submission since they cannot access their own device inside the booth. This presents a usability challenge as most voters, like in the BMD case, will likely not complete this check.

2.3 Coercion-Resistant Electronic Voting Systems

The goal of a coercion-resistant online voting is to deliver transparency and ballot secrecy under the much riskier conditions of remote voting.

Coercion-resistant voting systems therefore not only aim to achieve end-to-end verifiability but also provide a means for voters to cast their intended vote even when faced with a coercer who may issue arbitrary instructions [56]. This notion goes beyond receipt-freeness which merely prevents a voter from being capable of proving how they voted [17, 53, 58, 24]. Receipt-freeness fails to defeat plausible coercion strategies such as forced abstention (prevent any vote from being cast) and supervised voting (casting a vote while under the coercer's supervision). Receipt-freeness works best in situations where voting is mandatory and the voter votes in-person.

The biggest drawback of a coercion-resistant voting system is that voters must now learn a coercion evasion strategy that they can use when faced with a coercer. Coercion-resistant systems typically realize this evasion-strategy in one of two ways.

- Fake credentials (the Juels-Catalano-Jakobsson lineage): voters use indistinguishable fake credentials to comply with the coercer and outside the coercion moment, use their concealed real credential to cast their intended vote [56, 26, 40, 82].
- Deniable re-voting: after the coercion moment, voters cast a fresh vote to override their previously coerced vote [93, 6, 73, 95].

2.3.1 Coercion-Evasion Strategies

We now discuss these evasion strategies in more detail.

Fake Credentials

Fake credentials allow voters to safely provide or vote with them to any coercer or vote buying. By being indistinguishable, the voter can always claim that the credential they are providing the coercer is their real one. The coercer has no way to test for the realness of the credential, and therefore no way of knowing whether the claimed “real” credential is actually real.

Deniable Re-Voting

Deniable re-voting was introduced to ease usability as voters do not need to incur the costs of having additional credentials. Estonia is currently the only nation to deploy an e-voting system with coercion resistance, and uses the re-voting approach [35] at the cost of end-to-end verifiability [114]. Despite its appeal, deniable re-voting can be circumvented by coercing a last-minute vote, or by seizing a voter's voting materials after forcing the voter to cast the coercer's vote. The latter of which has been shown to occur in other democratic countries [23].

We therefore focus our work on the fake credential approach.

2.4 Open Challenges

A number of problems remain to make coercion-resistant voting systems usable and practical for the real world. First, how should real and fake credentials be materialized in the real world and be verifiable only to the voter that their real credential is, in fact, real? Second, how will voters react to a never-before-seen item: the coercion-evasive strategy of fake credentials and would they be able to understand and use it? And finally, how to construct an efficient tallying process that scales to millions of ballots? This dissertation aims to address these problems to work towards a usable coercion-resistant voting system.

2.4.1 The Materialization and Usability of Fake Credentials

Making fake credentials practical requires physically embodying or *materializing* credentials and their issuance, in a way that ordinary people can understand and use [64, 83, 82]. One key challenge is that the voter must know which credential is real and must personally be able to *verify* it as real for transparency, but then must be unable to *prove* this fact to anyone else. A coercer must also not know how many fake credentials the voter has or can create; otherwise a coercer can simply demand *all* of the voter’s credentials.

Usability-focused approaches to materializing fake credentials have generally followed Estonia’s lead by assuming trusted hardware, such as smart cards that store both real and fake credentials under different PINs [26, 83, 82, 40, 63]. Even under the as-yet-unproven assumption that sufficiently powerful and secure smart cards can be developed, they will be expensive, making even one per voter difficult to justify economically – let alone *several* per voter, as would be required to achieve real coercion resistance. Furthermore, these works have yet to conduct a systematic study with real users focusing on the usability of smart cards with fake credentials. Theoretical practical concerns include mistyping a digit in one’s PIN and accidentally casting a fake ballot instead of a real credential [37].

In this dissertation, we not only introduce a coercion-resistant registration scheme which issues voters real and fake credentials (chapter 6) but also conduct a user study on this scheme consisting of 150 diverse participants recruited from a suburban park in Boston, Massachusetts (Part III).

2.4.2 The Verifiability of Credentials

Registration-time transparency is another key challenge. Why can’t a compromised election authority just issue fake smart cards, for example, which silently issue *only* fake credentials, while the authority retains the secret keys required to cast “real” votes on behalf of all voters?

Civitas [26] proposes that voters interact with multiple registration tellers to reduce trust in any one teller. Asking real-world voters to interact with several tellers, however, incurs significant complexity and convenience costs, whose practicality has never been tested with a usability study. Further, the election authority faces the unenviable prospect of justifying this convenience cost to voters that risks sounding like: “You must interact with several registration tellers because you can’t fully trust any of them, even though we hired and trained them and they answer to us.” Election authorities usually want to and are even mandated to promote trust in elections and electoral processes, not to undermine that very trust.

In Krivoruchko’s work on registration [63], voters must generate their real credential, encrypt it, and then provide the encrypted version to the registrar. This process ensures that the registrar never obtains the real credential, eliminating the need for the registrar to prove the credential’s integrity. Nevertheless, this approach has two shortcomings with respect to coercion resistance: First, voters must possess a device prior to registration, which the coercer could have compromised or confiscated beforehand. Second, the scheme lacks a mechanism that proves to the registrar that the voter’s device knows the real credential from the encrypted version it submits to the registrar.

The coercion-resistant registration scheme we present in this dissertation ensures verifiability of the voter’s real credential using a single registrar device and paper. Coerced voters therefore avoid the predicament of having to choose a trusted teller like in Civitas but rather trust the registrar device (and realistically the registrar) not to collude with their coercer, or else comply with the coercer’s demands—a simpler binary choice.

2.4.3 Prohibitive Cost of Tallying

Tallying plays a critical role not only in ensuring that the properties of transparency and ballot secrecy are met, but also in revealing the outcome in a timely manner. In practice, elections routinely involve hundreds of thousands to millions of ballots, and any scheme whose tallying cost grows superlinearly in the number of ballots quickly becomes impractical in such settings. Large tallies are also desirable from a coercion-resistance perspective: the larger the tally, the larger the anonymity set in which each individual vote is hidden.

JCJ presented the first end-to-end verifiable, coercion-resistant scheme [56] which works similarly to other mixnet-based end-to-end verifiable schemes. The notable difference is that the ledger must accept any kind of ballot — real or fake — to prevent tests of realness by the coercer. Therefore, during tallying, the process involves filtering out fake ballots in a privacy-preserving manner. In particular, JCJ uses plaintext-equivalence tests (PETs [55]) between voter-unlinked ballots and encrypted representations of voters’ real credentials to determine the real ballots. Because the scheme requires cross matching each ballot with each real credential, this process essentially exhibits quadratic complexity ($S \cdot n_V$) where S is the number of post-duplicate-removal ballots and n_V is the number of registered voters. Furthermore, the duplicate-ballot removal process which requires removing ballots containing

the same (real or fake) credential also requires cross matching each ballot with every other ballot, incurring $O(B^2)$ PETs, where B is the number of ballots cast. In practice, this leads to a denial of service attack with amplification where an adversary consistently posts fake ballots, which forces an additional $A^2 + 2(A)(B)$ work on A adversarially cast ballots. We show in §6.5 that Civitas [26], an implementation of JCJ's scheme, would take an estimated 1,768 years to tally 1 million ballots on current hardware.

Smith and Weber [113, 122] shortly thereafter proposed the use of deterministic tags on encrypted data that they could then use to test equality in linear time. A set join between fingerprints computed on ballots and real credentials would therefore only take $O(S + n_V)$. These deterministic tags, however, introduce a subtle vulnerability: if an adversary could inject algebraically related credentials (e.g., one ballot with σ and another with σ^α), they could check for this relationship post-shuffle to learn whether the voter complied with their demands [61, 115, 91].

In this dissertation, we design a tallying scheme (chapter 7) similar to the proposed scheme by Koenig et al. [61] which involves constraining the cast ballots to credentials issued by the registrar as it naturally fits the design of our registration. With this approach, the coercer is no longer able to submit specially crafted ballots.

3 Standardized Usability Metrics

In this chapter, we introduce two standardized questionnaires employed in our usability study (Part III): the System Usability Scale (SUS) and the User Experience Questionnaire (UEQ). These metrics are widely used to obtain reliable, quick assessments of usability and user experience.

3.1 System Usability Scale

One of the most widely adopted metrics for perceived system usability is the System Usability Scale (SUS), originally developed by John Brooke in 1986 [21]. Brooke described the SUS as a “quick and dirty” usability scale intended for low-cost, rapid evaluations, but despite the informal nature it has proven to be highly robust and reliable in practice [69]. The questionnaire consists of the following 10 statements that participants rate on a 5-point Likert scale from 1 (“strongly disagree”) to 5 (“strongly agree”).

1. I think that I would like to use this system frequently.
2. I found the system unnecessarily complex.
3. I thought the system was easy to use.
4. I think that I would need the support of a technical person to be able to use this system.
5. I found the various functions in this system were well integrated.
6. I thought there was too much inconsistency in this system.
7. I would imagine that most people would learn to use this system very quickly.
8. I found the system very cumbersome to use.
9. I felt very confident using the system.
10. I needed to learn a lot of things before I could get going with this system.

These statements intentionally alternate between positive and negative phrasing so that participants read each statement carefully and think about their level of agreement. This alternation helps mitigate response biases (e.g., the tendency to uniformly agree or disagree without careful consideration).

Chapter 3. Standardized Usability Metrics

After participants submit the SUS questionnaire, we convert their answers into an overall usability score ranging from 0 to 100. To calculate this score, for each positively worded item (odd-numbered questions), we take the user's rating and subtract 1; for each negatively worded item (even-numbered questions), we subtract the user's rating from 5. This yields a score between 0 and 4 for each item. We then sum these scores and multiply the result by 2.5 to scale the final score between 0 and 100.

By including SUS in our study, we can use this single metric to gauge overall perceived usability. However, this number should be interpreted in comparison to other studies as it does not represent some kind of "percentage." Instead, Bangor et al. found the SUS score of 68 to be the industry average based on the results from over 200 studies across diverse products; a score of 68 therefore represents the 50th percentile. Bangor et al. [11] even proposed an "adjective rating scale" to help interpret SUS scores: for example, a SUS score above 73 earns a "Good" label, while a score above 85 earns an "Excellent" label.

3.2 User Experience Questionnaire

While SUS focuses on usability, the User Experience Questionnaire (UEQ) expands to include user experience [67]. User experience encompasses emotions, perceptions, and attitudes about the system [67], capturing how participants feel about the system, not just how easy or functional it is. This is important because a system could be usable yet not provide a pleasing or engaging experience, and vice versa. After all, if voters feel that the voting system fails to meet their needs, they might contest the results due to their dissatisfaction even if the system meets ideal transparency.

The UEQ is a standardized questionnaire for user experience developed by Laugwitz et al. in 2008 [67]. The questionnaire consists of 26 bipolar items — pairs of contrasting adjectives. For each of the 26 bipolar items, participants are presented with opposite terms and asked to rate their impression on a 7-point scale between those extremes. For example, an item might be "annoying" on the left and "enjoyable" on the right, and a user would mark how they felt about the system on that spectrum. A rating of 1 would mean "very annoying", 7 would mean "very enjoyable," with 4 being neutral. These ratings are then mapped from -3 to +3 so that -3 corresponds to the most negative rating and +3 corresponds to the most positive.

Each of the 26 items (listed in survey materials, table 8.2) contributes to one of the following six scales:

- **Attractiveness:** The appeal of the system to the user.
- **Perspiciuity:** The difficulty level for users to use the system. High perspicuity means users can understand the system quickly without confusion.
- **Efficiency:** The user's perception of efficiency to achieve tasks.
- **Dependability:** The users' trust and comfort that the system behaves as expected.
- **Stimulation:** The level of engagement with the user, such as whether the system is exciting

and motivating to use.

- **Novelty:** The user's perception of the system's creativity and originality.

Among the six scales, Perspicuity, Efficiency, and Dependability are categorized as pragmatic quality (i.e., goal-oriented), while Stimulation and Novelty are hedonic quality (i.e., satisfaction). Attractiveness is an overall rating that can be influenced by both pragmatic and hedonic impressions – essentially it measures the general appeal of the system to the user. Such insights are valuable because they pinpoint specific areas of the user experience that might need improvement, beyond general usability. For instance, a system might score very high on Efficiency and Dependability but low on Stimulation and Novelty, suggesting it is effective but perhaps boring.

UEQ Key Performance Indicator Extension (KPI) Hinderks et al. [52] introduced six additional questions as an extension to the UEQ questionnaire known as UEQ Key Performance Indicators (KPI). Each question asks the participant to rate how important they consider one of the six UX dimensions (Attractiveness, Perspicuity, etc.) *after* the main questionnaire. This allows us to determine how critical each scale is to participants and can therefore help future design iterations. For instance, if participants view efficiency as extremely important, a moderate efficiency score might be a bigger concern than a low novelty score if participants deem novelty unimportant for the system.

4 Cryptographic Primitives

This chapter presents the cryptographic primitives used in Votegral.

We work in a cyclic group G of prime order q with generator g (e.g., an elliptic-curve group such as P-256). We denote the security parameter by λ . For x sampled uniformly at random from $\mathbb{Z}_q$, we write $x \leftarrow \$ \mathbb{Z}_q$.

4.1 Distributed Key Generation Scheme

To avoid a single point of trust, we employ a distributed key generation protocol that allows n parties to jointly form a collective public key and n secret-key shares [90, 45]. We adopt the simple n -of- n variant: On input (G, q, g, n) , DKG outputs per-party key shares $(P_i^{\text{sk}}, P_i^{\text{pk}})$ and a collective public key P^{pk} :

$$\left\{ (P_i^{\text{sk}}, P_i^{\text{pk}}) \right\}_{i=1}^n, P^{\text{pk}} \leftarrow \text{DKG}(G, q, g, n).$$

such that $P_i^{\text{pk}} = g^{P_i^{\text{sk}}}$ and $P^{\text{pk}} = \prod_{i=1}^n P_i^{\text{pk}}$.

Correctness and liveness hinge on all parties contributing to decryption. Auditability comes from posting all P_i^{pk} and P^{pk} publicly, along with each party proving in zero-knowledge $P_i^{\text{pk}} = g^{P_i^{\text{sk}}}$ (§4.4).

4.2 ElGamal Encryption Scheme

The ElGamal scheme [36] consists of the following algorithms:

- $\text{EG.KGen}(G, q, g)$: a randomized key generation algorithm which takes as input the group definition and outputs a public key pk along with a private key sk such that $\text{pk} = g^{\text{sk}}$ where $\text{sk} \leftarrow \$ \mathbb{Z}_q$.

Chapter 4. Cryptographic Primitives

- $\text{EG.Enc}(\text{pk}, m)$: a randomized encryption algorithm which inputs a public key pk and a message $m \in G$ and outputs a ciphertext $C = (C_1, C_2) = (g^r, \text{pk}^r m)$ for $r \leftarrow \mathbb{Z}_q$.
- $\text{EG.Dec}(\text{sk}, C)$: a deterministic decryption algorithm which takes as input two private keys sk and a ciphertext C and outputs a message $m = C_2(C_1^{\text{sk}})^{-1}$.

ElGamal relies on the Decisional Diffie-Hellman hardness assumption for IND-CPA security.

A nice property of ElGamal is that it allows for re-randomized ciphertexts, which “re-encrypts” the same plaintext m with fresh randomness without knowing m :

$$\text{EG.rerand}(\text{pk}, C; s \leftarrow \mathbb{Z}_q) = (C_1 \cdot g^s, C_2 \cdot \text{pk}^s)$$

4.3 Signature Scheme

We assume a standard signature scheme Sig that is existentially unforgeable under chosen-message attacks (EUF-CMA) and consists of the following algorithms:

- $\text{Sig.KGen}(1^\lambda)$: a randomized key generation algorithm which takes as input the security parameter 1^λ and outputs a signing key pair (sk, pk) .
- $\text{Sig.Sign}(\text{sk}, m)$: a randomized signature algorithm which takes as input a private key sk and a bit-string $m \in \{0, 1\}^*$ and outputs a signature σ .
- $\text{Sig.Vf}(\text{pk}, m, \sigma)$: a deterministic verification algorithm which takes as input a public key pk , a bit-string $m \in \{0, 1\}^*$, and a signature σ and outputs $\top$ if σ is a valid signature of m under pk and $\perp$ otherwise.

In our implementation of Votegral, we use the Schnorr signature scheme but Votegral is compatible with any EUF-CMA scheme.

4.4 Zero-Knowledge Proofs

We model proofs against relations $R \subseteq X \times W$ over public instances $x \in X$ and witnesses $w \in W$. The associated language is $L_R = \{x \in X : \exists w \text{ with } (x, w) \in R\}$. A zero-knowledge proof system for R consists of the following algorithms:

- $\text{ZK.Prove}_R(x, w) \rightarrow \pi$
- $\text{ZK.Verify}_R(x, \pi) \rightarrow \top / \perp$

It satisfies the following properties:

- **Completeness:** Honest proofs output $\top$.
- **Zero-Knowledge:** There exists a simulator capable of producing indistinguishable proofs without w .
- **Soundness:** There exists an extractor that outputs w from any prover convincingly with non-negligible probability.

4.4.1 Σ -protocols

Σ -protocols have three moves between a prover and verifier: the prover sends a commit a , the verifier responds with a random challenge c , and the prover answers with r . Verification involves checking linear relations in the exponent. Two meta-properties are important:

- **Special Soundness:** Two accepting transcripts with the same a but different challenges result in an extractor for w .
- **Honest-Verifier Zero-Knowledge (HVZK):** There is a simulator producing accepting transcripts given random challenges.

We use the following Σ -protocols:

- $\Sigma_{\log}$ (Schnorr [109]): Relation $R_{\log} = \{(y; x) : y = g^x\}$. Protocol follows:
 1. Commit: $a = g^t$ for $t \leftarrow \mathbb{Z}_q$.
 2. Challenge: $c \in \mathbb{Z}_q$.
 3. Response: $r = t + cx \pmod{q}$.
 Verification proceeds as $g^r = a \cdot y^c$.
- Σ_{eq} (Chaum-Pedersen [25]): Relation: $R_{eq} = \{(g_1, g_2, y, z; x) : y = g_1^x \wedge z = g_2^x\}$. This involves the same structure as $\Sigma_{\log}$ but with two verification equations using the same challenge c and response r on two different commits: $g_1^r = a_1 \cdot y^c$ and $g_2^r = a_2 \cdot z^c$.

In most but not all of our constructions, proofs are Σ -protocols made non-interactive via the Fiat-Shamir heuristic with a collision-resistant hash function H (random-oracle model).

Conjunctive (AND) and disjunctive (OR) composition. Let $R_1, \dots, R_k$ be relations with Σ -protocols. We use classical composition rules [28].

- **AND:** To prove $\bigwedge_j (x_j \in L_{R_j})$, run all Σ -protocols in parallel with a single challenge $c \leftarrow \mathbb{Z}_q$. Verification process checks each sub-proof with the same c .
- **OR of 2 relations:** To prove $x \in L_{R_0} \cup L_{R_1}$ while holding a witness only for one branch, say R_b :
 1. Simulate a transcript $(a_{1-b}, c_{1-b}, r_{1-b})$ for the false branch using HVZK simulator with random $c_{1-b} \leftarrow \mathbb{Z}_q$.
 2. Generate a real commitment a_b for the true branch with a random fresh t .
 3. Receive the challenge c , and define $c_b = (c - c_{1-b}) \pmod{q}$.
 4. Compute $r_b = t + c_b \cdot w_b \pmod{q}$.
 The proof becomes $(a_0, a_1, c_0, c_1, r_0, r_1)$ with $c = c_0 + c_1 \pmod{q}$. Verification involves replaying both branch verifications with their respective c_j and checks $c_0 + c_1 = c$.
- **OR of 1-of- k :** Extend the above by simulating $k - 1$ branches and answering only one branch honestly; the verifier checks that the challenges sum to the challenge $c \pmod{q}$.

4.5 Hash functions

We use collision-resistant hash functions H with outputs of length 2λ for binding structured data and for Fiat-Shamir challenges when turning Σ -protocol non-interactive.

4.6 Message Authentication Code.

A MAC provides integrity using symmetric encryption (with a shared key), and consists of the following algorithm:

- $\text{MAC.Sign}(k, m)$: a deterministic signing algorithm which takes a (secret) key k and a message m and outputs an authorization tag τ .
- $\text{MAC.Vf}(k, m)$: a deterministic verification algorithm which takes a (secret) key k , a message m and an authorization tag τ and outputs $\top$ if τ is a valid signature under m and k ; otherwise $\perp$.

4.7 Verifiable Mixnet (Shuffle)

A verifiable mixnet takes a public list of ciphertexts and outputs a permuted, independently re-randomized list along with with a publicly verifiable proof that no plaintexts were changed. This provides unlinkability between inputs and outputs (privacy) while providing for public verifiability.

Let $\mathbf{C} = (C_1, \dots, C_M)$ be ElGamal ciphertexts under a public key P_{pk} with each ciphertext C_i being two group elements $(C_{1,i}, C_{2,i})$. A correct shuffle produces $\mathbf{C}'$ and a proof π such that $\exists$ permutation π and $s_1, \dots, s_M \in \mathbb{Z}_q : \forall j, C'_j = \text{EG.rerand}(P^{\text{pk}}, C_{\pi(j)}; s_j)$. A mixnet therefore consists of the following two algorithms:

- $\text{Mix.shuffle}(P^{\text{pk}}, \mathbf{C}) \rightarrow (\mathbf{C}', \Pi)$.
- $\text{Mix.verify}(P^{\text{pk}}, \mathbf{C}, \mathbf{C}', \Pi) \rightarrow \top / \perp$.

The security properties are:

- Completeness: Honest shuffles always verify.
- Soundness: If verify accepts, then with negligible probability the output list is a re-randomized permutation of input list.
- Zero-Knowledge: The proof Π reveals nothing about the underlying permutation beyond that the plaintexts remain identical.
- Composability: Multiple mix servers can be used, and if at least one mix server is honest by keeping its permutation secret, input-output linkability remains hidden.

Instantiations of verifiable shuffles in Votegral include Neff [80] and Bayer-Groth [12].

4.8 Verifiable Distributed Deterministic Tags

To perform equality tests on encrypted data — such as filtering — without revealing the underlying plaintexts, we use a distributed deterministic tagging protocol. Adapted from UnLynx [43], this protocol allows n parties to transform an ElGamal ciphertext into a publicly verifiable tag.

The primitive consists of the following three algorithms:

- $\text{Tag.setup}(1^\lambda) \rightarrow (\{S_i = g^{s_i}\}_{i=1}^n, \pi_i^s)$: each party i commits to a short-term secret s_i by publishing S_i and a Schnorr NIZK π_i^s proving knowledge of s_i .
- $\text{Tag.tag}(C) \rightarrow (T, \Pi_{\text{tag}})$: The parties execute a two-round protocol that transforms ciphertext C into a tag T , publishing a complete transcript of NIZKs Π_{tag} .
- $\text{Tag.verify}(C, T, \Pi_{\text{tag}}, P_{\text{sk}_i}, S_i) \rightarrow \top / \perp$. which accepts if and only if all sub-proofs in the transcript Π_{tag} are valid.

We use the construction specified in Unlynx [43]. It is completed in two rounds:

Round 1 (Offset). Starting with the input ciphertext $C = (C_1, C_2)$, each party i computes

$$C'_2 \leftarrow C_2 \cdot S_i \leftarrow C_2 \cdot g^{s_i}$$

After this round, the ciphertext is $(C_1, C'_2) = (g^r, m \cdot (P^{\text{pk}})^r \cdot g^{\sum_i s_i})$. This step prevents the encryption of an identity element resulting in the identity element after Tag.

Round 2 (Deterministic Randomness). Let $(\tilde{C}_{1,0}, \tilde{C}_{2,0}) \leftarrow (C_1, C'_2)$. For $i = 1, \dots, n$:

$$\tilde{C}_{1,i} = (\tilde{C}_{1,i-1})^{s_i}, \quad (4.1)$$

$$D_i = \tilde{C}_{2,i-1} \cdot (\tilde{C}_{1,i-1})^{-P_i^{\text{sk}}}, \quad (4.2)$$

$$\tilde{C}_{2,i} = D_i^{s_i}. \quad (4.3)$$

The final tag is $T \leftarrow \tilde{C}_{2,n}$. By construction, the randomness r is removed, and the tag is

$$T = m^s \cdot g^{s \cdot \sum_{i=1}^n s_i}, \text{ where } s = \prod_{i=1}^n s_i. \quad (4.4)$$

NIZK Proofs For each round, party i generates a set of proofs using a challenge $c_1 \in \mathbb{Z}_q$ (Fiat-Shamir style).

- A Chaum-Pedersen proof for eq. (4.1):

$$\text{CP}\left\{s_i : \log_g(S_i) = \log_{\tilde{C}_{1,i-1}}(\tilde{C}_{1,i})\right\}$$

Chapter 4. Cryptographic Primitives

With commitment $(a_{1,i}, a_{2,i})$ and response $r_{1,i}$, verification checks:

$$\begin{aligned} g^{r_{1,i}} &\stackrel{?}{=} a_{1,i} \cdot S_i^{c_i}, \\ \tilde{C}_{1,i-1}^{r_{1,i}} &\stackrel{?}{=} a_{2,i} \cdot \tilde{C}_{1,i}^{c_i} \end{aligned}$$

- Let $R_i = D_i \cdot \tilde{C}_{2,i-1}^{-1} = (\tilde{C}_{1,i-1}^{-1})^{P_i^{\text{sk}}}$. A Chaum-Pedersen proof for eq. (4.2):

$$\text{CP}\left\{P_i^{\text{sk}} : \log_g(P_i^{\text{pk}}) = \log_{\tilde{C}_{1,i-1}^{-1}}(R_i)\right\}$$

With commitment $(b_{1,i}, b_{2,i})$ and response $r_{2,i}$, verification checks:

$$\begin{aligned} g^{r_{2,i}} &\stackrel{?}{=} b_{1,i} \cdot (P_i^{\text{pk}})^{c_i} \\ (\tilde{C}_{1,i-1}^{-1})^{r_{2,i}} &\stackrel{?}{=} b_{2,i} \cdot R_i^{c_i} \end{aligned}$$

- A Chaum-Pedersen proof for eq. (4.3):

$$\text{CP}\left\{s_i : \log_g(S_i) = \log_{D_i}(\tilde{C}_{2,i})\right\}$$

With commitment $(d_{1,i}, d_{2,i})$ and response $r_{3,i}$, verification checks:

$$\begin{aligned} g^{r_{3,i}} &\stackrel{?}{=} d_{1,i} \cdot S_i^{c_i}, \\ D_i^{r_{3,i}} &\stackrel{?}{=} d_{2,i} \cdot \tilde{C}_{2,i}^{c_i} \end{aligned}$$

The security properties are:

- **Completeness:** An honestly computed transcript always verifies, and the output tag is correct and deterministic.
- **Soundness:** A malformed step is rejected except with negligible probability, due to the soundness of the underlying Σ -protocols. The proofs ensure that each party correctly uses its committed sort-term secret s_i and its long-term secret key corresponding to the public P_i^{pk} .
- **Privacy (Equality-Only Leakage):** If at least one authority is honest and keeps its s_i secret, the tag reveals nothing about m beyond what can be inferred from equality tests, except with negligible probability (DL Hardness).
- **Public Verifiability.** Anyone can perform `Tag.verify` on a posted transcript, confirming the integrity of the tag computation.

4.9 Verifiable Threshold ElGamal Decryption

A set of n parties holds secret key shares and want to jointly decrypt some ciphertext without ever reconstructing a full secret key on any device. While decrypting, each party produces a publicly verifiable decryption share with a non-interactive zero-knowledge proof (NIZK) of

correctness. Anyone can check shares and combine them to recover the plaintext.

The set of algorithms for this primitive is:

- $\text{DDec.share}(P_{\text{sk}_i}, C) \rightarrow (S_i, \pi_i)$: a deterministic algorithm which takes as input the respective party's secret share P_{sk_i} and ElGamal ciphertext $C = (C_1, C_2)$ and outputs its decryption share $S_i = C_1^{P_{\text{sk}_i}} = (g^r)(P_{\text{sk}_i}) = g^{r \cdot P_{\text{sk}_i}}$ and a Chaum-Pedersen NIZK proof of correctness that $\log_g(P_{\text{pk}_i}) = \log_{C_1}(S_i)$.
- $\text{DDec.verify}(P_{\text{pk}_i}, C, S_i, \pi_i) \rightarrow \top / \perp$: a public deterministic algorithm which takes as input the party's public key P_{pk_i} , the ciphertext C , the share S_i and the proof of correctness π_i and outputs $\top$ if the proof holds, $\perp$ otherwise.
- $\text{DDec.combine}(C, \{S_i\}_{i=1}^n) \rightarrow m$: a public deterministic algorithm which, after verifying all shares, computes $S = \prod_{i=1}^n S_i = g^{(r \cdot \sum_i P_{\text{sk}_i})} = P_{\text{pk}}^r$ and recover $m = C_2 \cdot S^{-1}$.

This primitive has the following properties:

- **Completeness:** Honestly generated shares always verify, combined shares equals the original plaintext m .
- **Soundness:** A malformed share is rejected except with negligible probability, by soundness of the Chaum-Pedersen proof (Fiat-Shamir in the random-oracle model).
- **Privacy:** Under DDH, IND-CPA security of ElGamal, fewer than n shares reveal no information about m . No subset of parties $n - 1$ can recover m .
- **Public verifiability & auditability:** Each decryption share comes with a NIZK proof and therefore anyone can attribute a failed rejection of DDec.verify to a specific party.

Votegral System Part II

5 System Overview

This chapter provides a high-level overview of Votebral, our coercion-resistant, end-to-end verifiable online voting system. We begin by defining the core security, verifiability, and usability properties the system must satisfy. We then outline the three protocol phases: Registration, Voting, and Tallying. The registration component, TRIP (presented in chapter 6), enables voters to create real and fake credentials in a verifiable manner using only paper. The voting and tallying component, VLT (presented in chapter 7), performs linear time filtering without involving talliers during the voting phase and introduces standing votes, which are chosen at registration for cases of extreme coercion. In part III, we present the user study evaluating Votebral’s usability.

5.1 System Properties

Property 1 (Voter Eligibility). Only eligible voters are permitted to participate in an election, and each eligible voter contributes at most one real vote.

Property 2 (Privacy [117, 56]). Ballot privacy holds against all parties, including the election authority: no one learns how an individual voter voted beyond what is implied by the announced outcome.

Property 3 (Universal Verifiability [56, 117]). From a public transcript, any observer can verify that the published outcome reflects the count of the accepted ballots consistent with the roster of eligible voters.

Property 4 (Individual Verifiability [18, 117]). Any honest voter who follows the protocol can verify, using their local materials and the public transcript, that the system recorded their participation correctly.

Property 5 (Coercion-Resistance [56]). Even after interacting arbitrarily with a target voter and observing all public transcripts and anything the voter discloses, a coercer cannot determine whether the voter followed the coercer’s demands or instead deviated to cast their intended vote.

Property 6 (Usability [85, 33, 64]). The system is understandable and usable by its target audience — effective (voters can complete registration and voting), efficient (within reasonable time and effort), and satisfactory (voters trust the process). A coercion-resistant system, in particular, requires that voters understand the anti-coercion mechanism and can use it correctly.

5.2 System and Threat Models

We now present the system model, introducing the system’s actors, their capabilities and responsibilities, and their interactions with one another. Subsequently, we present the threat model, introducing the adversaries, their goals and capabilities.

5.2.1 System Model

Votegral consists of three main entities: an *Election Authority* ($\mathbb{A}$), a *Registrar* ($\mathbb{R}$), and a set of voters ($\mathbb{V}$). The election authority ($\mathbb{A}$) acts as the root of trust: it defines the election rules, maintains a public ledger ($\mathbb{L}$) and produces a publicly verifiable final tally. The registrar ($\mathbb{R}$), operating under $\mathbb{A}$ ’s purview, manages the physical process of voter enrollment $\mathbb{V}$.

Election Authority

The *election authority*, $\mathbb{A} = \{A_1, \dots, A_{n_A}\}$, is a set of n_A mutually distrusting members that collectively administer the election. In practice, $\mathbb{A}$ may include government agencies and, as observers, political parties. Its primary goal is to orchestrate the process and provide public evidence of integrity. To this end, $\mathbb{A}$ provisions two distributed back-end services:

1. **The Ledger ($\mathbb{L}$):** A cryptographically-enforced immutable, append-only public broadcast channel for all election data. We model the ledger $\mathbb{L}$ as a collection of named, ordered logs (akin to database tables): $\mathbb{L} = (\mathbb{L}_R, \mathbb{L}_E, \mathbb{L}_V, \mathbb{L}_P, \mathbb{L}_C)$ representing (respectively) registration records, public commitments, ballots cast, the standing-party registry, and the registrar-issued credential registry. $\mathbb{L}_E$ contains public commitments made by the registrar; voters use these when activating their credentials to help verify the soundness of their real credential.
2. **The Tally Service:** A service responsible for computing the final election outcome from $\mathbb{L}_V$ in a publicly verifiable manner, as detailed in chapter 7.

Security of these back-end services relies on a threshold trust assumption. Each $A_i \in \mathbb{A}$ operates independent infrastructure (e.g., a server or data center). We assume at most $t < \frac{n_A}{3}$ Byzantine corruptions; under standard partially synchronous BFT, the services therefore remain correct and live. Alternatively, in settings where trust in $\mathbb{A}$ is limited, $\mathbb{A}$ can outsource auditability to a public decentralized network (e.g., Bitcoin or Ethereum), forwarding data on-chain; voters then verify that their data is recorded on that network. For readability, we

refer to $\mathbb{L}$ as a single entity, leaving this distributed trust model implicit.

Additional responsibilities of $\mathbb{A}$ include:

- Defining the electorate, typically by compiling an *electoral roll* ($\mathbb{V}$) of eligible voters.
- Providing open-source client-side software for voters and registration officials, which allows independent experts to audit that it conforms to the public protocol specification.
- Using software attestation to verify that the code running on backend servers and registrar equipment matches the public open-source codebase. This is defense-in-depth and not relied upon for verifiability.

Registrar

The *registrar* $\mathbb{R}$ is the entity responsible for managing physical registration sites and issuing voting credentials to eligible voters. Its role is to bridge the physical world of identity verification with the digital world of cryptographic credentials, while ensuring voters receive a coercion-free environment to receive such credentials. To this end, it comprises three sub-actors:

1. **Registration Officials** ($\mathbb{O} = \{O_1, \dots, O_{n_o}\}$): Human officials, represented by their official supporting device (OSD), whose task is to authenticate voter's real-world identity and authorize them for credential issuance.
2. **Registration Kiosks** ($\mathbb{K} = \{K_1, \dots, K_{n_k}\}$): Automated devices in privacy booths that interact with voters to issue their credentials.
3. **Envelope Printers** ($\mathbb{P} = \{P_1, \dots, P_{n_p}\}$): Printers either at each registration site or at a central location that produce envelopes with per-envelope secrets/commitments integral to the verification process of real paper credentials.

Voters

The set of eligible voters is denoted by $\mathbb{V} = \{V_1, \dots, V_{n_v}\}$. A voter's journey in VoteGral involves the following physical and digital interactions:

1. **Enrollment:** The voter V_i visits a registration site, is authenticated by an official O_j and uses a kiosk K_k in a privacy booth. This process results in the issuance of a real paper credential and zero or more fake paper credentials.
2. **Activation & Voting:** The voter uses a *Voter Supporting Device* (VSD) — typically a smartphone or computer running the open-source election app. Using the VSD camera, the voter scans their paper credential to activate it, which securely transfers and verifies the credential's cryptographic materials. The voter then uses VSD to cast their ballot using their intended option, which is subsequently recorded on the voting ledger $\mathbb{L}_V$. Voters no longer need their paper credential to cast ballots in future elections, requiring only their VSD.
3. **Verification:** The election app on VSD also empowers the voter by acting as an independent

verifier. To ensure individual verifiability, it checks that the voter's registration session and ballot appear correctly on $\mathbb{L}_R$ and $\mathbb{L}_V$. Furthermore, as part of universal verifiability, voters can verify the tally posted by $\mathbb{A}$ on VSD.

5.2.2 Threat Model

A secure voting system must reconcile two seemingly contradictory requirements. On one hand, to achieve end-to-end verifiability, voters must not need to trust third parties — within the bounds that verifiability is defined — and instead each act as an independent auditor. On the other hand, to provide coercion resistance, voters must rely on some entity to provide them a safe environment where they may act freely when faced with coercion.

Our model therefore rests on the deliberate separation of the trust assumptions required for coercion-resistance from those required for end-to-end verifiability to maximize the capabilities of each adversary. For verifiability, we model the registrar as malicious and voters as honest auditors. For coercion-resistance, we model voters as malicious and the registrar as honest — providing a physically secure and private environment, the registration booth. Within this environment, a coerced voter can freely generate the necessary materials (e.g., fake credentials) needed for plausible deniability when faced with their coercer. Therefore, while a voter under coercion must trust the registrar to provide them with a safe space, the voter does not need to trust the registrar for the integrity of their real credential.

This model also mirrors in-person voting. The integrity of the election outcome is achieved through transparent, auditable processes that do not require trust in poll workers. The safety of the voter, however, is achieved by trusting these poll workers to provide them with a safe space.

We now define three distinct, computationally bounded adversaries to evaluate VoteGral's properties. Unless stated otherwise, cryptographic primitives and secure channels (e.g., TLS) are assumed sound. Table 5.1 summarizes which components/actors each adversary can compromise.

Integrity

The integrity adversary $\mathcal{I}$'s goal is to manipulate the election outcome *without detection*, as any detected intervention would undermine the public's confidence in the outcome. For example, $\mathcal{I}$ can compromise the registrar and attempt to present a fake credential as the voter's real one. $\mathcal{I}$ can compromise all actors except:

- **The Voter's Devices (VSD):** This allows us to focus specifically on threats originating from the registration process itself. We leave verifying the ballot's contents out of scope but VoteGral is compatible with popular cast-as-intended mechanisms, such as Benaloh challenge [15]. Further discussion is present in §7.6.3.

5.2 System and Threat Models

Adv. \ Device	Ledger	Authority	OSD	Kiosk	Envelope Printers	VSD w/ Cred.	
						Real	Fake
Integrity	Yes	All	Yes [†]	Yes [†]	Yes [†]	No	No
Privacy	Yes	$n_A - 1$	Yes	Yes	Yes	No	Yes
Coercion	Yes	$n_A - 1$	No	No	No	No [‡]	Yes

Table 5.1: **Threat Model:** This table depicts an adversary’s ability to compromise a device and the entity or credential it represents.

[†] The risk of the adversary compromising voter registration is small per registration, and becomes negligible when considering the combined probability across registrations.

[‡] If voters cannot conceal their device from a coercer, they can entrust their real credential to a trusted third party to cast votes on their behalf.

- **The Electoral Roll:** The integrity of the electoral roll is out of scope. While critical for any real-world election, its compilation involves administrative and legal processes that are orthogonal to the cryptographic protocol design of a voting system. Nevertheless, we allow for information related to a voter’s identity to be present on the registration ledger $\mathbb{L}_R$ for public auditability as will later be discussed.

$\mathcal{I}$ may control the infrastructure writing to $\mathbb{L}$, but remains bound by $\mathbb{L}$ ’s properties (public, append-only). It can attempt *censorship* by refusing or delaying inclusion of valid data, or *flooding* by submitting large volumes of well-formed inputs. Censorship is a standard, detectable liveness failure: honest clients check inclusion on $\mathbb{L}$, and independent parties can corroborate omissions. We therefore treat this remediation as an operational concern and out of scope. Our focus is on flooding with well-formed inputs that seek to make cryptographic tasks impractical within reasonable time.

Privacy

The privacy adversary $\mathcal{P}$ ’s goal is to *reveal* a voter’s real vote (e.g., for targeted persuasion as seen with Cambridge Analytica [96]). $\mathcal{P}$ can compromise all actors except:

- **At least one election authority member:** a common trust assumption in electronic voting literature [117, 56] and in privacy-enhancing systems in general [107, 43]. It enables the use of threshold decryption, where the refusal of a single honest member prevents illicit decryption of any individual ballot. For simplicity, our threshold is $t = |\mathbb{A}|$ but t may be less than $|\mathbb{A}|$.
- **The Voter’s Device:** The voter’s device containing their real credential. This is a common assumption in e-voting systems, as voters are inherently unable to perform cryptographic computations mentally.

Coercion

The coercion adversary $\mathcal{C}$ interacts with voters and aims to ensure that these targeted voters comply with their demands.

$\mathcal{C}$'s capabilities:

- *Voter Compromise*: $\mathcal{C}$ can exert direct physical and psychological pressure on a voter to, for example, compel disclosure of secrets, demand credentials, and supervise actions on a VSD.
- *Backend Compromise*: As with $\mathcal{P}$, $\mathcal{C}$ may compromise all but one member of $\mathbb{A}$.

Limitations:

- *Registrar*: $\mathcal{C}$ cannot compromise the registrar, including its devices and registration sites. It therefore may not observe the voter's actions while inside the booth. This is the foundational assumption that enables real/fake credential distinction to work. While side channels are a potential attack vector, they are out of scope for this thesis but discussed in §6.6.2.
- *Trusted Device*: While $\mathcal{C}$ may compromise VSDs holding fake credentials, it cannot compromise VSDs containing the voters' real credentials. We assume voters can find private moments to access their trusted device (e.g., to cast their real vote unobserved).
- *Anonymous Voting Channel*: As is typical in coercion-resistant voting schemes [56, 26], the channel used to cast a ballot is assumed to be anonymous. This prevents $\mathcal{C}$ from forced abstaining the voter by simply monitoring the communication channel between the voter and the ledger.

5.3 Protocol Overview

This section outlines the end-to-end operation of Votebral, with the protocol flow for a single voter, Alice, illustrated in Figure 5.1. Alice wishes to protect herself from coercion by creating one real and one fake credential. The protocol unfolds in three main phases now detailed at a high level: **Registration, Voting, Tallying**.

Registration

Alice begins by registering in person at a registration site. This phase bridges her verified physical identity with her anonymous (paper) credentials. This interactive phase, detailed later in chapter 6, generates the following materials:

- **A Real (Paper) Credential**: This paper credential contains a unique signing key pair (R, r) and a *public credential tag* $T = \text{EG.Enc}(A^{\text{pk}}, R)$ which is a probabilistic encryption of Alice's real public key R under the election authority's public key A^{pk} . This tag T serves as the cryptographic link between Alice's real credential on the registration ledger $\mathbb{L}_R$ and the anonymous ballot she will later cast. To convince Alice that she can use R to cast her

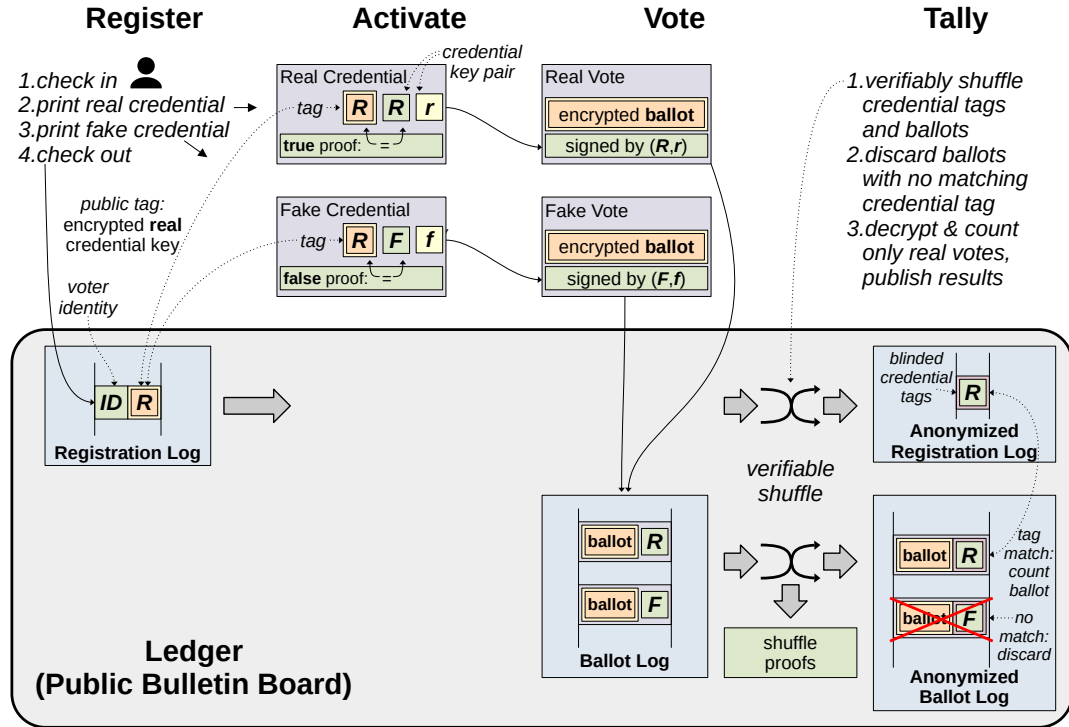


Figure 5.1: Votegral Protocol Design Summary

intended vote — that T encrypts R — the paper credential contains a transcript of her interaction with a kiosk that forms a sound proof.

- **Zero or more Fake (Paper) Credentials:** These distinct paper credentials appear identical to the real credential but contain a different key pair (F, f) along with the appropriately forged proof that falsely proves that these credentials casts real votes — that T encrypts F . The visible difference in the printing process to create these credentials allows Alice, and only Alice, to distinguish these credentials from her real credential.

Upon completing registration, the registrar posts a record of Alice's registration on the *registration ledger* $\mathbb{L}_R$. This record may contain metadata that uniquely identifies Alice, such as Alice's name and address along with her public credential tag T . Any previous registration records for Alice are superseded and therefore invalidated, ensuring that each voter has at most one active registration — public credential tag — at any time. Alice's current credentials therefore remain valid to cast votes in any subsequent elections until she either undergoes registration again or they expire — a policy set by the election authority.

$\mathbb{L}_R$ Transparency. The transparent nature of $\mathbb{L}_R$ helps mitigate issues like “ghost voters.” By making registration records public, auditors or concerned citizens can verify the list against census data or identify irregularities, such as a deceased individual remaining on the electoral roll. The actual metadata included is orthogonal to the protocol and is a policy decision

Chapter 5. System Overview

balancing voter privacy and verifiability.

Voting

Before casting for the first time, Alice activates her real paper credential on a trusted VSD. Activation serves two purposes: it securely transfers the secret signing key r from the paper to her device, and it allows her VSD to perform an automated verification of the credential-tied proofs. As part of verification, VSD uses the corresponding public commitments that the registrar's printers posted on $\mathbb{L}_E$ prior to registration.

The activation process for a fake paper credential is identical, including the verification steps, except that the signing key f is transferred from the paper to the device. Therefore, votes cast using this signing key are later removed from the final tally.

During the election period, Alice can cast ballots using any of her credentials. Regardless of the credential used, real or fake, each ballot is authenticated — signed by the respective credential — and posted on the public *voting ledger* $\mathbb{L}_V$. To an outside observer, ballots appear indistinguishable — hiding both the vote and the voter casting the vote. The latter being necessary to prevent forced abstention where the coercer demands the voter to refrain from voting and check the ledger to determine whether the voter complied.

Standing Votes

If Alice were to face extreme coercion (e.g., she cannot safely access an unsupervised device or risks an immediate post-registration search), she could instead cast a standing vote:

1. **Party Selection in the booth.** At registration, the kiosk presents Alice the available political parties to choose from — the *party registry* $\mathbb{L}_P$, each bound to a public key P . Alice selects a party, and the kiosk then forms the public credential tag $T = \text{EG.Enc}(A^{\text{pk}}, P)$.
2. **Materialization.** To preserve coercion resistance, the kiosk prints any number of fake credentials Alice desires. No standing-vote receipt is issued and the voter leaves the booth carrying only fake paper credentials. The check-out, activation and voting processes remains identical, with these credentials all casting fake ballots; the corresponding party's vote will be this voter's real vote.

The drawback of standing votes is that these voters forego individual verifiability, but if they cannot safely activate a real credential in any case, they already do not participate in individual verifiability.

Party ballot casting. Each certified party casts a ballot under its public key P using the standard voting protocol described earlier.

Tallying

After the deadline, the tally service run by $\mathbb{A}$ runs a publicly verifiable tally process that anonymously links active registrations to real ballots while discarding fake ones.

At a high level:

1. **Inputs.** $\mathbb{A}$ collects the public credential tags $\{T_i = \text{EG.Enc}(A^{\text{pk}}, R)\}$ and ballots B from $\mathbb{L}_R$ and $\mathbb{L}_V$, respectively.
2. **Mix.** Verifiably shuffle the public credentials T and, separately, the ballots B to break linkability to voters.
3. **Deterministic tags.** Run a distributed *deterministic* tag protocol on the mixed roster credentials and ballot credentials, producing equality-only tags.
4. **Join and standing-vote multiplicities.** Publicly match ballot credential tags to roster credential tags, and keep exactly the ballots whose credential tags appear in the roster. If a roster tag appears multiple times (party key from $\mathbb{L}_P$ through standing votes), credit that ballot with the corresponding multiplicity.
5. **Decrypt and prove.** Verifiably decrypt kept ballot vote ciphertexts and publish the final tally and a proof. For tags with multiplicity > 1 , decrypt the corresponding ballot credential to link the political party to their vote, allowing voters to audit the political parties' public statements with their vote.

Revocation

When a voter's ability to participate in future elections must be revoked (e.g., upon death or loss of eligibility), the registrar updates the registration ledger $\mathbb{L}_R$ by publicly "removing" the voter's current public credential tag T . Concretely, the registrar appends a new registration record for that voter identifier in which T is set to a distinguished empty value $\perp$.¹ During tallying, only non-empty tags in $\mathbb{L}_R$ are used, so a revoked voter contributes no entry to the roster.

¹We use $\perp$ to denote the absence of any active credential for that voter.

6 TRIP: Trust-Limited Coercion-Resistant In-Person Voter-Registration

In this chapter, we present TRIP, the registration scheme of VoteGral. We start by providing an overview of the TRIP design, continue with the formal scheme along with a security analysis and proofs and finally finish with an evaluation of TRIP and some miscellaneous discussions.

6.1 Design Overview

6.1.1 Interactively Proving Real Credentials Real

To enable Alice to verify individually that the ballot she cast with her real credential will be counted, she needs to “know” that the public credential tag printed on all her credentials and recorded in the registration log is a correct encryption of her real credential’s public key:

$$T = \text{EG.Enc}(A^{\text{pk}}, R)$$

This is ultimately the critical single bit of information Alice needs to complete the end-to-end transparency chain and verify that her real vote will count. For coercion resistance, however, Alice must be unable to prove that critical information bit to anyone else.

As a strawman, the kiosk might give Alice an ordinary non-interactive zero-knowledge proof, of the kind commonly used in digital signatures, that her credentials’ public tag is an encryption of her real credential. Alice could then be certain that her real credential is real – but then she might also take that proof (receipt) with her upon leaving the booth and hand it to a coercer or vote buyer, who would be equally convinced. Such a non-interactive proof would therefore support transparency but undermine coercion resistance.

A key observation is that we can use *interactive* zero-knowledge proofs or IZKPs to convince Alice that her real credential is real without making that information transferable. VoteGral uses IZKPs taking the common Σ -*protocol* form, which consist of three steps: *commit*, *challenge*, and *response*. For such an IZKP to be sound and effectively prove anything, the protocol must

Chapter 6. TRIP: Trust-Limited Coercion-Resistant In-Person Voter-Registration

be executed in precisely this order: *first* the prover chooses a cryptographic commitment; *then* the verifier chooses a challenge value previously-unknown to the prover; *finally* the prover computes the sole valid response corresponding to the combination of commit and challenge. If the Σ -protocol is executed in the wrong order – in particular, if the prover knows or can guess the verifier’s challenge *before* choosing the commit – then the IZKP is unsound and the prover can trivially “prove” anything.

Leveraging this observation, Votegral includes in all credentials, real and fake, a *transcript* of the three phases of an IZKP, which *purports* to prove that the public credential tag contains an encryption of the credential’s public key – i.e., that the credential is real. Anyone can verify the structural validity of these IZKP proof transcripts, and voters’ devices do so automatically when activating credentials. Such a transcript alone omits one crucial bit of information, however: was the commit, or the challenge, chosen first? Votegral achieves coercion-resistant verifiability by revealing this one crucial bit of information to the voter only interactively, via the sequence of steps the voter takes in the privacy booth to print real and fake credentials. Once the voter observes this distinction in printing steps, the credentials themselves embody only IZKP transcripts, which lack the crucial bit of “voter’s-eyes-only” information, and are useless to prove to anyone else which credential is real. Real credentials contain sound IZKPs, fake credentials contain unsound ones, and the two are cryptographically indistinguishable once printed.

We are not the first to use IZKPs for coercion resistance along these lines; similar ideas have been explored for in-person voting [79]. To our knowledge, however, Votegral is the first design to use IZKPs in *registration* for e-voting to achieve verifiability and coercion resistance using fake credentials.

6.1.2 Physically Materializing Usable Credentials

Although IZKPs in principle resolve the conflict between individual verifiability and coercion resistance, for usability we cannot expect ordinary voters to understand e-voting systems or cryptography, or any of the technical details underlying credentials, or precisely *why* technically Alice should be convinced that her real credential is real. In particular, we need a physically *materialized* credentialing process that ordinary voters can understand and follow given only minimal training, and ideally that is cheap and efficient enough to deploy at scale. These are the considerations that motivate Votegral’s specific design using paper credentials.

One particular issue is that to ensure that the kiosk *must* honestly produce sound IZKPs for real credentials, the voter – taking the verifier’s role in the IZKP – must choose and give the kiosk a cryptographic challenge, only *after* the kiosk has printed the IZKP commit. Choosing and entering a high-entropy random challenge manually would be tedious and burdensome, and ordinary users are bad at choosing random numbers [84]. These considerations motivate Votegral’s choice to use envelopes, each pre-printed with a unique random QR code, enabling voters to pick a challenge by selecting any envelope from a supply provided in the booth. A

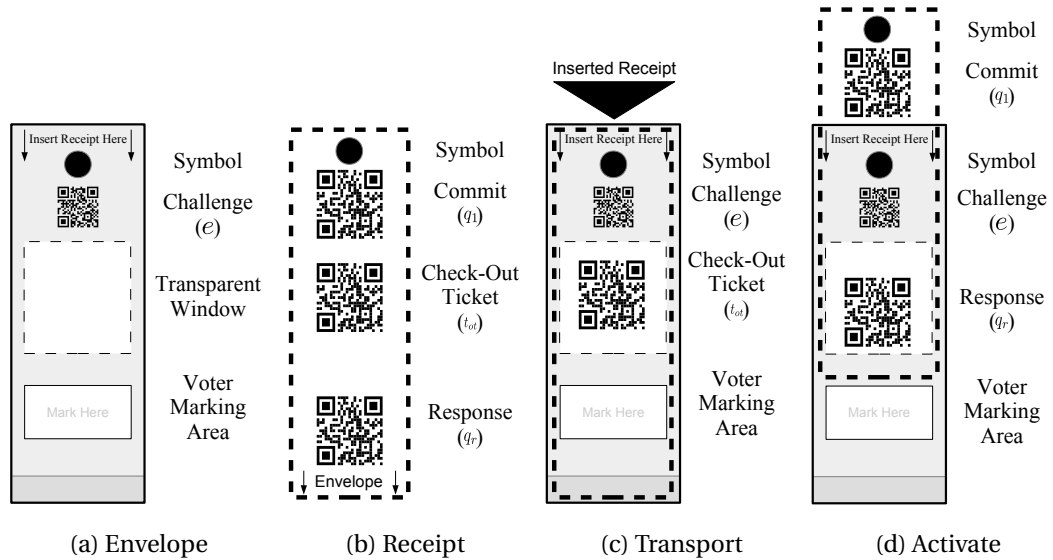


Figure 6.1: **Receipt and Envelope Design.**

A paper credential consists of an envelope (a) and receipt (b). Voters mark envelopes to discern their real credential from fake ones. They transport each credential by placing the receipt into the envelope (c) and activate it by lifting the receipt one-third of its length (d).

compromised registrar could duplicate envelopes to make challenges more predictable, but an activation-time check for duplicate challenges detailed in §6.3.1 ensures high-probability detection if many envelopes are duplicated.

Above and beyond an instructional video, we would like the credentialing process itself to help train voters to learn and expect the correct process, especially for printing real credentials. To ensure in particular that voters do not hastily choose or present an envelope too early, before the kiosk has printed the IZKP commit, an honest kiosk first chooses one of a few symbols at random and prints it just above the commit QR code; the voter must then choose any envelope with the matching symbol. When interacting with an honest kiosk, therefore, the voter *must* wait to see the symbol (and commit) printed before picking an envelope, otherwise the honest kiosk gently rejects the voter’s choice of envelope. If a voter accustomed to this process later encounters a compromised kiosk that asks them to present an envelope first while supposedly printing a “real” credential, we hope that the voter’s normal-case training will make such an irregularity more noticeable.

To create fake credentials, in contrast, the kiosk asks the voter to choose and present an envelope first. The kiosk thus obtains the challenge before choosing its commit, thus enabling the kiosk to fake a “proof” of the credential’s “realness.” Again, we hope that even voters completely unaware of the reasons for this curious process will nevertheless be able to follow it and, at least with moderate probability, notice and report if a compromised kiosk ever attempts to use the fake-credential process to print a “real” credential.

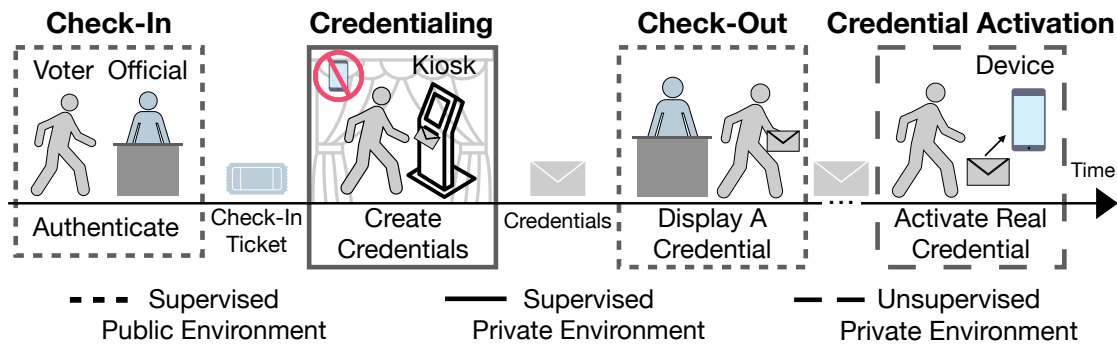


Figure 6.2: **TRIP Voter Workflow**

The voter (1) authenticates themselves to an official at a check-in desk to obtain a check-in ticket (2) enters a supervised private environment to create a real and any potential fake credentials, (3) presents one of their credentials to an official at the check-out desk, and (4) activates their real and any fake credentials on a device they trust, which could belong to them or a trusted third party.

Votegral's envelope and receipt design secures transport and activation. Fully inserting the receipt into the envelope places the credential in the *transport* state (fig. 6.1c). In this state, the QR code necessary for check out appears through the envelope's window, but the envelope's opaque lower portion hides the area of the receipt containing the credential's secret key. Only after the voter transports the credential to whatever device the voter trusts to activate it on, the voter pulls the receipt out of the envelope just enough to reveal the two other QR codes needed for activation, including the credential's secret key, as shown in fig. 6.1d.

6.1.3 Voter-Facing Design

Registering to use Votebral involves the following main steps, which fig. 6.2 illustrates at a high level.

Instructional Video. In the registrar's office, voters first watch an instructional video, covering the credential creation processes and the purpose of fake credentials.

Check-In. A registration official verifies each voter's eligibility, then gives the voter a *check-in ticket*, which permits the voter to enter a privacy booth and use the kiosk inside. Depending on applicable anti-recording policy, voters might be asked to turn off or check in personal devices before entering the privacy booth.

Privacy Booth. Inside the booth is a touchscreen kiosk that can scan and print machine-readable codes, along with a pen and a large supply of envelopes. Each envelope has a see-through area and printed markings as depicted in fig. 6.1a.

Real Credential Creation. Voters follow a 4-step process to create their real credential. The voter first scans the barcode on their check-in ticket (Step 1). The kiosk prints a symbol and a

QR code on receipt paper (Step 2). The voter selects an envelope with a matching symbol and scans its QR code (Step 3). The kiosk then prints two additional QR codes (Step 4), completing the receipt shown in fig. 6.1b. This four-step process directly instantiates the sound interactive proof from Section 6.1.1, where the voter provides the challenge (the envelope) after the kiosk makes its commitment (the first printout). The voter then inserts the receipt inside the scanned envelope for transport (fig. 6.1c), forming their real paper credential. The voter marks the credential, for instance by labeling it as ‘ $\mathcal{R}$ ’, to distinguish it from any fake credentials. If a coercer learns or guesses that the voter does this, then at the next registration the voter can mark a fake credential ‘ $\mathcal{R}$ ’ and mark their real credential ‘ $\mathcal{R}\mathcal{R}$ ’ or in any memorable way.

Fake Credential Creation. A voter wishing to create a fake credential does so in 2 steps: the voter picks and scans an envelope (Step 1, fig. 6.1a), then the kiosk prints the entire receipt (Step 2, fig. 6.1b). This two-step process deliberately executes the proof protocol in the wrong order, allowing the kiosk to generate an unsound but visually identical transcript. The voter then inserts the receipt into the newly-chosen envelope and marks it differently from the real one. Voters can create as many fake credentials as desired. Officials may inquire if a voter spends excessive time in a booth, thus imposing an informal, non-deterministic limit. Once finished, the voter leaves the booth and checks out.

Check-Out. The voter presents any one of their credentials to the official, who scans the receipt’s second QR code through the envelope’s window. This completes the voter’s visit to the registrar. To ensure prompt detection and rectification of any successful impersonation of voters, the registrar subsequently notifies voters digitally and/or by mail of the completed registration session.

Activation. The voter activates their real credential on any device they trust, whether that is their own or a friend’s device. The voter lifts the receipt one-third out of the envelope to the *activate* state (fig. 6.1d). They scan the three visible QR codes: the receipt’s top and bottom QR codes, and the envelope’s QR code. The voter can now discard this credential, perhaps shredding it if the voter faces a coercion threat.

After Registration. The voter can now use their device to cast their intended vote, verifying it with existing cast-as-intended methods [15, 27] to ensure the integrity of the ballot’s contents. Only the vote cast with the real credential will count, however. A voter who loses their device can re-register for new credentials. After each election, the voter’s device downloads, verifies, and shows the election results to the voter.

6.1.4 Additional Design Considerations

Beyond the primary design choices, several deployment considerations influence more subtle nuances.

Public $\mathbb{L}_R$: Impersonation Defense vs. “Surprise” Coercion

A design decision in TRIP is to make registration updates public on the registration log $\mathbb{L}_R$. This transparency directly counters impersonation by an integrity adversary ($\mathcal{I}$), such as a malicious registrar attempting to register in a voter’s name: changes on $\mathbb{L}_R$ are visible to the voter (via VSD notifications). A tension arises, however, with “surprise” coercion. A voter who previously registered without creating fake credentials might later face a coercer; re-registering to obtain a new real credential publicly signals a change on $\mathbb{L}_R$, revealing the voter’s later defiance.

We prioritize verifiability, while mitigating the impact of this trade-off. In particular, natural credential expiration can help victims acquire a new real credential or cast a standing vote without arousing suspicion, thereby limiting the period during which coercion can succeed. The election authority can additionally implement a proactive credential-expiry lottery. By periodically selecting a random subset of voters for mandatory re-registration, the system creates a constant, non-suspicious reason for anyone to be visiting a registration site. This provides “cover” for the authority to act on anonymous tips about coercion. A victim, or their friend, could call in an anonymous tip that triggers a government-initiated re-registration summons, giving the victim a plausibly deniable reason to visit the registrar and report the coercion in safety. This is similar to a bank’s duress PIN, which silently alerts authorities when a user is forced to withdraw money at an ATM [51].

In addition, if an adversary aims to change election outcomes, they must target many voters. Broader targeting increases public awareness as more voters are subjected to coercion, accelerating the adoption of (and familiarity with) fake credentials and reducing attack efficacy over time. Because TRIP lets any voter secretly create fake credentials, a coercer never knows whether a given voter holds only a real credential.

Proactive Preparedness vs. On-Demand Models

The proactive stance of TRIP invites comparison to JCJ-style on-demand fake credential creation [56, 26]. In realistic deployments, there are nuances with on-demand fake credentials.

Suggested on-demand deployments typically rely on per-voter trusted hardware (e.g., smart cards) with duress PINs [37, 40, 82] that unlock fake credentials. In surprise-coercion settings, the coercer can simply demand to confiscate the smart card, nullifying the on-demand option. Purely software on-demand creation would likely require a voter’s device to carry out additional steps that are visible to the coercer (e.g., forging a proof on a new credential). Consequently, issuing, storing, and replacing high-assurance trusted hardware at scale is necessary and costly. By contrast, paper credentials are inexpensive, single-use (activation-only), and disposable after activation, making many of them economical.

Work by Koenig et al. [61] on constraining ballots to registrar-issued credentials suggests issuing voters a mandated minimum of fake credentials. However, such a minimum enables a

6.2 The TRIP Registration Protocol

Symbol	Description
G, q, g	A cyclic group G of order q with generator g
$\mathbb{A}, \mathbb{O}, \mathbb{K}, \mathbb{P}, \mathbb{V}$	Authority, Officials, Kiosks, Envelope Printers, Electoral roll
n_A, n_O, n_K, n_P	Number of Authorities, Officials, Kiosks, Envelope printers
$\mathbb{L}, \mathbb{L}_R, \mathbb{L}_E, \mathbb{L}_V$	Ledger and Registration, Envelope & Voting (sub-)ledgers
$V_{id}, c_{pc}, c_{pk}, c_{sk}$	Voter's identifier, Public Credential, Public & Private Keys
$c, \tilde{c}_{pk}, \tilde{c}_{sk}$	A credential, A fake credential's Public and Private Keys
$\mathbf{E}, n_E, n_c$	Envelope challenges and number of envelopes & credentials
τ, s_{rk}	MAC authorization tag, Official & kiosk shared secret key
OSD, VSD	Registration official and Voter's supporting devices
$t_{in}, t_{out}, q_c, q_r$	Check-In & Check-Out Tickets, Commit & Response Codes

Table 6.1: **Scheme Notations.**

simple coercion strategy: demand the (minimum +1) credentials. Instead, we allow voters to choose the number of fake credentials, including zero, since some voters may judge them unnecessary and prefer to avoid any chance of misidentifying their real credential.

Rather than enforcing the creation of fake credentials, TRIP aims to make them socially normalized. For example, we re-frame “fake credentials” as “test credentials” in the user-facing instructions of the user study in part III. This encourages voters to create them for legitimate, non-coercion-related reasons, such as for testing the system or for educational purposes — like showing children how e-voting works. Therefore, by making test credentials useful and “fun,” the aim is to have few voters leave the registration booth with only a real credential and create an unofficial minimum of fake credentials.

6.2 The TRIP Registration Protocol

This section presents the formal scheme of TRIP. We begin by defining our notation and the required cryptographic primitives.

Notation. For a finite set S , $s \leftarrow S$ denotes that s is sampled independently and uniformly at random from S . The symbol $\ominus$ represents exclusion from a collection of elements. We denote $a \leftarrow b$ as appending b to a , $a \parallel b$ as concatenating b with a , $\mathbf{x}$ as a vector of elements of type x , and $\mathbf{x}[i]$ as the i th entries of the vector $\mathbf{x}$. We use $\top$ and $\perp$ to indicate success and failure, respectively. Variables used throughout the TRIP scheme are summarized in table 6.1.

Primitives. TRIP requires (1) the ElGamal encryption scheme EG, (2) a distributed key generation scheme DKG, (3) a EUF-CMA signature scheme Sig, (4) a secure hash function H, (5) a message authentication code scheme MAC, and (6) an interactive zero-knowledge proof of equality of discrete logarithms ZKPoE. We defined these primitives in chapter 4.

Setup($\lambda, \mathbb{V}, (G, p, g), n_A, n_O, n_K, n_P, n_E$)
1: $\mathbb{L} \leftarrow \emptyset$
2: $\{A_i^{\text{sk}}\}, \mathbb{A}_{\text{pk}} \leftarrow \text{DKG}(G, p, g, n_A)$
3: $\{O_i, K_i, P_i \leftarrow \text{Sig.KGen}(1^\lambda)\}_{0 \leq i < n_O, 0 \leq i < n_K, 0 \leq i < n_P}$
4: $\mathbb{L}_R \leftarrow \{V_i^{\text{id}}\}_{i \in \mathbb{V}}$
5: $\mathbf{E} \leftarrow \{P_j \xleftarrow{\$} \mathbb{P}; e_i \xleftarrow{\$} \mathbb{Z}_q; \mathbb{L}_E \leftarrow (P_j^{\text{pk}}, H(e_i), \text{Sig.Sign}(P_j^{\text{sk}}, H(e_i)))\}_{0 \leq i < n_E}$
6: $s_{rk} \leftarrow \{0, 1\}^\lambda$

Figure 6.3: **Setup** Procedure for the ledger, the authority members, the officials, the kiosks and the envelope printers with envelope issuance. The secret s_{rk} is shared between officials and kiosks.

6.2.1 Setup.

The Setup algorithm, detailed in fig. 6.3, initializes the system's core actors and public parameters. While prior works often include registration as part of a broader setup process, we separate it to delineate the registration protocol cleanly. Setup proceeds as follows:

- **Initialize Ledger:** The global ledger $\mathbb{L}$ is initialized. Its sub-ledgers — $\mathbb{L}_R$ for registration records, $\mathbb{L}_E$ for envelope challenge commitments, and $\mathbb{L}_V$ for votes — are made available publicly.
- **Initialize Authority:** The members of the Election Authority $\mathbb{A}$ jointly execute a distributed key generation protocol (DKG). This produces a single collective public key $\mathbb{A}_{\text{pk}}$ for encryption, while the corresponding secret key is secret shared among the authority members (A_i^{sk}). $\mathbb{A}_{\text{pk}}$ is made publicly available. $\mathbb{A}_{\text{pk}}$ must be a generator of G_q .
- **Initialize Registrar Actors:** Each entity involved in the registration process — every official (O_i), kiosk (K_i), and printer (P_i) — generates its own long-term signing key pair using $\text{Sig.KGen}(1^\lambda)$. Their public keys are made known. The registrar uses the electoral roll $\mathbb{V}$ to populate $\mathbb{L}_R$ with each voter's unique identifier V_i^{id} .
- **Populate Voter Roll:** The initial electoral roll $\mathbb{V}$ is used to populate the registration ledger $\mathbb{L}_R$ with the unique identifier V_i^{id} for each eligible voter. This creates a public list of who is eligible to vote. In practice, the registrar may update $\mathbb{V}$ accordingly.
- **Generate Envelopes:** The printers $\mathbb{P}$ generate a large supply of envelopes, $\mathbf{E}$. The total number, n_E , must be sufficient for all voters to create real and fake credentials, with a buffer for security, as defined by

$$n_E > c|\mathbb{V}| + \lambda_{\mathbf{E}}|\mathbb{K}|$$

The parameter $c > 1$ is an estimate of envelope consumption per voter, and $\lambda_{\mathbf{E}}$ is a security parameter detailed in §6.3.2. Unlike paper ballots, envelopes do not expire and can be reused across elections.

For each envelope, a printer P_j generates a fresh nonce $e \xleftarrow{\$} \mathbb{Z}_q$, which will be used as the

ZKP challenge by the voter inside the booth.

- *Materialization*: The printer P_j prints the envelope containing the printer's public key P_j^{pk} , the nonce e , and a signature $\sigma_p \leftarrow \text{Sig.Sig}_{P_j^{\text{sk}}}(P_j^{\text{pk}} \parallel H(e))$.
- *Commitment*: To commit to the envelope's contents, the printer publishes a tuple containing its public key, the hash of the nonce, and the signature — $(P_j^{\text{pk}}, H(e), \sigma_p)$ — to the envelope ledger $\mathbb{L}_E$. During Activate, VSD will reveal the nonce e to prevent it from being reused.
- **Establish Shared Secret**: The registration officials $\mathbb{O}$ and kiosks $\mathbb{K}$ establish a shared symmetric secret key s_{rk} . This key is used to generate and verify MACs on the check-in tickets that authorize a voter to use a kiosk.

6.2.2 Voter Check-In and Kiosk Authorization

CheckIn(O, K, s_{rk}, V_{id})	
OSD (O, s_{rk}, V_{id})	Kiosk (K, s_{rk}, t_{in})
1: $\tau_r \leftarrow \text{MAC.Sig}(s_{rk}, V_{id})$	$(V_{id}, \tau_r) \leftarrow t_{in}$
2: $t_{in} \leftarrow (V_{id}, \tau_r)$	$\text{MAC.Vf}(s_{rk}, \tau_r, V_{id}) \stackrel{?}{=} \top$

Figure 6.4: **Check-In**. The official's device issues check-in ticket and kiosk verifies authenticity of check-in ticket.

The in-person registration process begins when a voter presents their identity to an official at the check-in desk. As detailed in fig. 6.4, the official first verifies the voter's eligibility. Upon successful authentication, the official issues the voter a single-use check-in ticket t_{in} .

This ticket consists of the voter's identity, V_{id} and a message authentication code (MAC), τ_r , computed over V_{id} using the shared secret key s_{rk} .

For usability, we have the ticket be a physical barcode printed on paper. Due to the limited data capacity of barcodes compared to QR codes, we use a compact MAC (MAC) signature rather than a digital signature (Sig). This is further discussed in §8.5.

The voter takes this ticket into the privacy booth and presents it to the kiosk (e.g., by scanning the barcode). The kiosk validates the authorization tag τ using the same shared key s_{rk} .

6.2.3 Interactive Credential Creation

The core of the TRIP protocol is the interactive creation of credentials, shown in fig. 6.5 and fig. 6.6. The goal is to issue a single public key credential tag, c_{pc} , and then use an interactive process to prove it corresponds to a real key. A different interactive process falsely proves c_{pc} corresponds to a fake key.

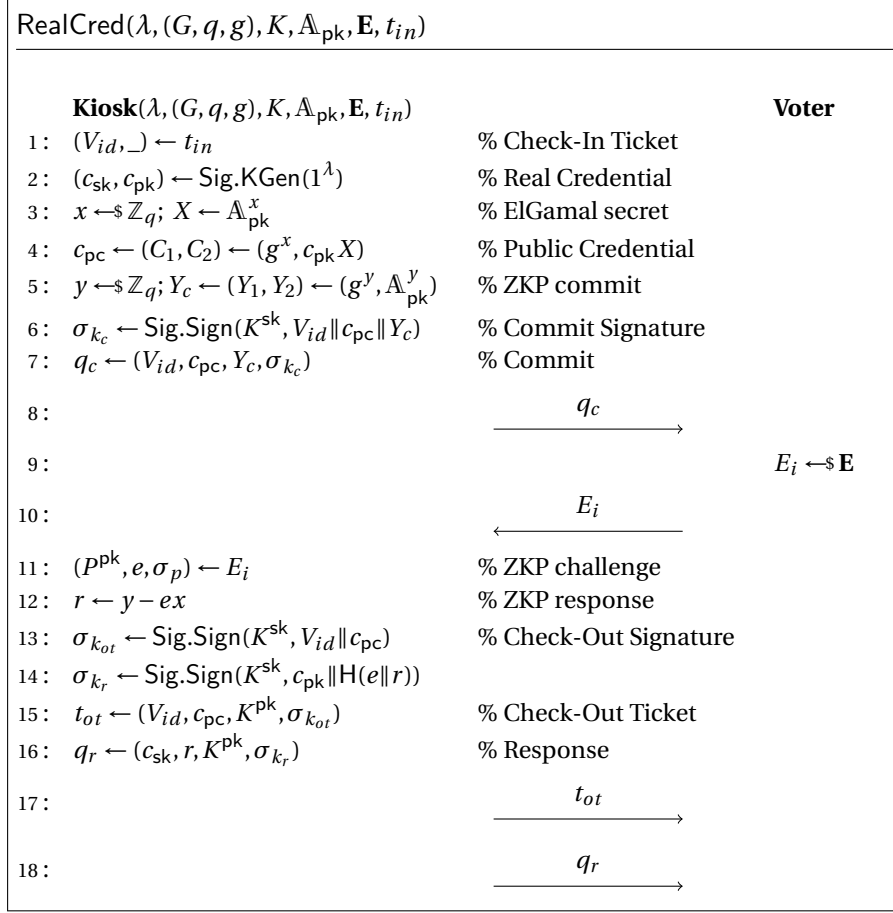


Figure 6.5: **Real Credential Creation Process.** Voter and kiosk follow a sound zero-knowledge proof construction.

The process begins right after the voter's check-in ticket t_{in} is validated. The kiosk first generates the voter's real key pair (c_{sk}, c_{pk}) . It then creates the public credential tag by encrypting the public key: $c_{pc} = \text{EG.Enc}_{\mathbb{A}_{pk}}(c_{pk}; x)$ where x is the secret ElGamal-generated randomness. This tag c_{pc} will be used for all credentials, real and fake. The voter learns which credential is real only by observing the interactive proof process that follows.

Creating a Real Credential (The Sound Proof).

To issue a real credential, the kiosk and voter engage in a sound interactive proof (ZKPoE that c_{pc} correctly encrypts c_{pk}):

$$\text{ZKPoE}_{C_1, C_2, X} \{(x) : C_1 = g^x \wedge X = \mathbb{A}_{pk}^x\}$$

The process strictly follows the three steps of a Σ -protocol.

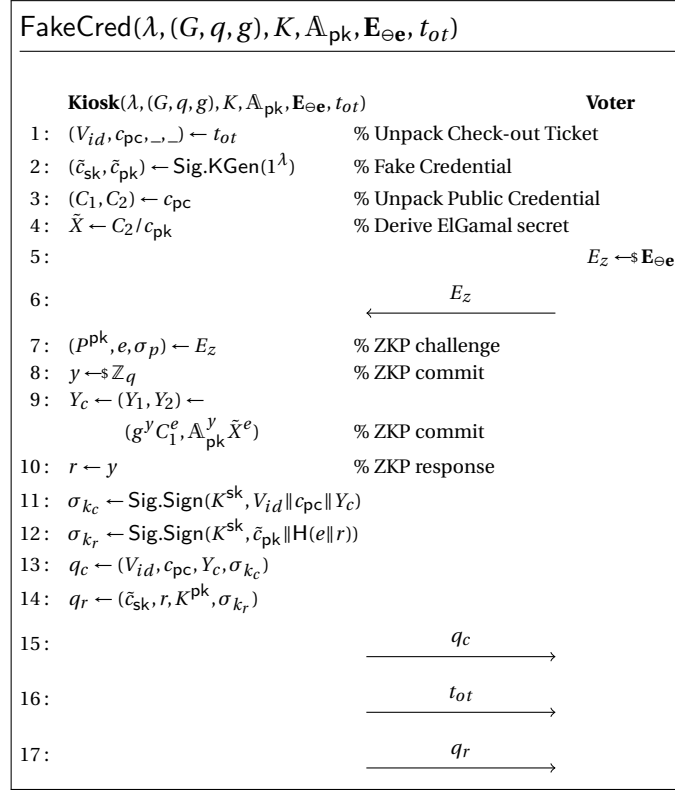


Figure 6.6: **Fake Credential Creation Process.** Voter and kiosk follow an unsound zero-knowledge proof construction. Envelopes cannot be reused.

1. **Commit:** The kiosk generates fresh randomness y and computes the ZKP commitments Y_c (line 5). It signs the voter's ID, the public tag c_{pc} and the commitments together to create σ_{k_c} (line 6). This data is then printed as the first QR code, q_c . *The voter waits for this printout before proceeding.*
2. **Challenge:** The voter selects an envelope E_i from the available supply of envelopes $\mathbb{E}$, and presents it to the kiosk (line 9). The kiosk unpacks the envelope's contents to retrieve the nonce e , which serves as the ZKP challenge (line 11).
3. **Response:** The kiosk, now having both the commit and the challenge, computes the final ZKP challenge $r = y - ex$ (line 12). It then prints two final items: the check-out ticket t_{ot} (line 15) and the response QR code q_r (line 16). The check-out ticket is for the official and contains the public tag c_{pc} and a signature. The response QR code contains the voter's secret key c_{sk} , and the ZKP response r .

This strict commit-challenge-response sequence constitutes a sound proof. The voter's confidence comes from observing the correct sequence of physical actions (along with the output from their VSD during activation).

Creating a Fake Credential To create a fake credential, the kiosk falsely “proves” that the same public tag c_{pc} encrypts a new, useless key $\tilde{c}_{pk}$, as shown in fig. 6.6. This requires breaking the

Chapter 6. TRIP: Trust-Limited Coercion-Resistant In-Person Voter-Registration

soundness property of the proof protocol by inverting its order.

1. **Challenge:** The process begins with the voter providing the challenge. They select a new envelope E_z and present it to the kiosk (lines 5-7). The kiosk now knows the challenge e before it has to commit.
2. **“Cheating” Commit:** Knowing e in advance now allows the kiosk to cheat. It generates a new key pair $(\tilde{c}_{sk}, \tilde{c}_{pk})$ (line 2), and derives the ElGamal exponent $\hat{X}$ (line 4). It then uses $\hat{X}$, along with the known challenge e to construct the cheating commit Y_c that is tailored to satisfy the proof equation (line 9).
3. **Response** The ZKP response is now computed as $r = y$ (line 10). The kiosk then prints the cheating commit and the response into QR codes q_c and q_r , respectively (line 13-14). The check-out ticket QR code t_{ot} remains the *same* as the one printed for the real credential (line 16), as the contents are identical.

This challenge-commit-response sequence constitutes an *unsound* proof. The voter can repeat the fake credential creation process for any number of desired fake credentials within reasonable limits. The voter then leaves the booth with their real and fake credential(s).

6.2.4 Check-Out.

CheckOut($O, \mathbb{L}, \mathbb{K}, t_{ot}$)	
OSD ($O, \mathbb{L}, \mathbb{K}, t_{ot}$)	
1: $(V_{id}, c_{pc}, K^{pk}, \sigma_{kot}) \leftarrow t_{ot}$	% Check-Out Ticket
2: $K^{pk} \stackrel{?}{\in} \mathbb{K}^{pk}$	% Authorized?
3: $\text{Sig.Vf}(K^{pk}, \sigma_{kot}, V_{id} \ c_{pc}) \stackrel{?}{=} \top$	% Verify Signature
4: $\sigma_o \leftarrow \text{Sig.Sign}(O^{sk}, V_{id} \ c_{pc} \ \sigma_{kot})$	% Official Approval
5: $\mathbb{L}_R[V_{id}] \leftarrow (c_{pc}, K^{pk}, \sigma_{kot}, O^{pk}, \sigma_o)$	% Update Ledger on V_{id}
VSD ($\mathbb{L}, V$)	
6: $\text{Notify}(V_{id})$	% Notify Voter

Figure 6.7: **Check-Out.** The official approves the voter’s registration session, publishing on the ledger their signature, the kiosk’s signature and the voter’s public credential. The ledger verifies signatures and the VSD monitoring the ledger notifies the voter.

The check-out process, detailed in fig. 6.7, finalizes the voter’s in-person registration session by posting an immutable, publicly verifiable record on the ledger. This step is critical as it binds the voter’s identity to their new public credential tag.

First, the voter presents any one of their credentials — real or fake — to the official. Because the check-out ticket t_{ot} is identical across all credentials, the choice reveals no information. The voter keeps the credential in the transport state (fig. 6.1c), ensuring only the t_{ot} portion is visible to the official’s scanner.

The official’s device (OSD) first verifies the kiosk’s signature on the check-out ticket to ensure

Activate($\mathbb{L}, V, c$)	
VSD ($((G, q, g), \mathbb{L}, V, c)$)	
1: $((V'_{id}, c_{pc}, Y_c, \sigma_{k_c}), _, (c_{sk}, r, K^{pk}, \sigma_{k_r}), (p^{pk}, e, \sigma_p)) \leftarrow (q_c, _, q_r, e) \leftarrow c$	% Unpack Credential
2: $c_{pk} \leftarrow \text{Sig.PubKey}(c_{sk})$	% Get Public Key
3: $\text{Sig.Vf}(K^{pk}, \sigma_{k_c}, V'_{id} \parallel c_{pc} \parallel Y_c) \stackrel{?}{=} \top$	% Receipt Integrity Check 1
4: $\text{Sig.Vf}(K^{pk}, \sigma_{k_r}, c_{pk} \parallel H(e \parallel r)) \stackrel{?}{=} \top$	% Receipt Integrity Check 2
5: $\text{Sig.Vf}(p^{pk}, \sigma_p, H(e)) \stackrel{?}{=} \top$	% Envelope Integrity Check
6: $(C_1, C_2) \leftarrow c_{pc}; X \leftarrow C_2 / c_{pk}$	% Derive ElGamal Secret
7: $(Y_1, Y_2) \leftarrow Y_c$	% Extract ZKP commitments
8: $Y_1 \stackrel{?}{=} g_1^r C_1^e; Y_2 \stackrel{?}{=} A^r X^e$	% Verify ZKP on C_1, C_2, C_3
9: $(c'_{pc}, K'^{pk}, \sigma_{k_{ot}}, O^{pk}, \sigma_o) \leftarrow \mathbb{L}_R[V_{id}]$	% Voter Reg. Session
10: $c'_{pc} \stackrel{?}{=} c_{pc} \wedge K'^{pk} \stackrel{?}{=} K^{pk} \wedge V'_{id} \stackrel{?}{=} V_{id}$	% Verify Public Cred & Actors
11: $e \notin \mathbb{L}_E[H(e)]; \mathbb{L}_E[H(e)] \leftarrow e$	% Challenge Unused & Append

Figure 6.8: **Credential Activation.** Verifies the integrity of the credential: if any procedure with fails, then the process aborts. Upon success, VSD stores the credential's private key c_{sk} .

its contents are valid. OSD then adds its own attestation by signing the details of the session as σ_o : the voter's ID, their public credential tag, and the kiosk's signature. This creates a chain of accountability, with the official formally vouching for the completion of the voter's session. Finally, the OSD publishes the complete registration record – $(c_{pc}, K^{pk}, \sigma_{k_{ot}}, \sigma_o)$ – to the registration ledger $\mathbb{L}_R$.

The voter's VSD, monitoring the ledger, will detect this update and notify the voter of this registration session.

6.2.5 Activation.

Activation is the final step of the TRIP protocol, where the voter's trusted device (VSD) acts as an automated auditor to verify the authenticity of the credential for which the voter is incapable of (e.g., cryptographic elements). To begin activation, the voter places the credential in the *activate state* (fig. 6.1d, which reveals three QR codes: the commit QR code (q_c), the envelope (E_i), and the response QR code (q_r). The VSD then performs a series of checks to validate the credential's integrity:

1. **Credential Contents Attestation (lines 2-5):** First, VSD checks all the signatures: the kiosk's signatures on the commit (σ_{k_c}) and response (σ_{k_r}) QR codes, along with the printer's signature on the envelope (σ_p). A failure here indicates that one of these two parties acted maliciously since the voter chose a registrar-supplied envelope and was present when the kiosk produced the signatures.
2. **ZKP Proof Verification (lines 6-8):** Next, the VSD verifies the interactive proof. It uses the given public credential tag c_{pc} to derive the ElGamal secret component X (line 6). It then

Chapter 6. TRIP: Trust-Limited Coercion-Resistant In-Person Voter-Registration

uses the commitments Y_c , the challenge e , and the response r to verify the ZKP equation. This check confirms that the proof is cryptographically valid. A failure here, when activating the *real* credential, proves that the kiosk was malicious and cheated during the interactive protocol.

3. **Public Registration Record Check (lines 9-10):** VSD then cross-references the credential with the public record on the ledger $\mathbb{L}_R$. It fetches the voter's registration record and confirms that public credential tag c_{pc} on the paper credential matches the one on the record. This ensures that the credential being activated is the current one for the voter.
4. **Challenge Uniqueness Check (line 11):** Finally, VSD ensures that the envelope's challenge has not yet been used. It looks up the hash of the envelope's challenge on $\mathbb{L}_E$ and checks that no e is present, and then publishes e publicly. This check addresses an attack where envelopes are duplicated with the same challenge to increase the kiosk's odds of success. More details are in §6.3.1.

These checks lead to one of two outcomes.

On Success. If all checks pass, VSD stores the secret key c_{sk} for future voting. The paper credential can now be discarded. This outcome provides the final piece of assurance for the voter. The VSD has confirmed the cryptographic validity of the credential. The voter, having observed the “procedural” soundness of its creation (i.e., commit-then-challenge sequence), now has end-to-end verification that this credential will cast real votes. VSD, of course, remains unaware of whether the credential activated is real or fake; only the voter knows this.

On Failure. If any of the checks fail, VSD alerts the voter and presents the failure message (e.g., “Invalid signature by the kiosk”, “Duplicate challenge”) to report to election officials. It also instructs the voter to re-register for a new set of credentials.

6.3 Security Analysis and Formal Proofs

In this section, we analyze the security properties of the TRIP protocol, as depicted in fig. 6.9 combining all sub-processes. We prove that TRIP satisfies individual verifiability, even against a malicious registrar. We then turn to proving that TRIP achieves coercion-resistance. As this is an end-to-end property that relies on voting and tallying, we adapt the JCJ voting and tallying schemes to prove that TRIP satisfies coercion-resistance.

Formal API. For the purpose of our security analysis, we model the Votegral system using the formal API presented in Figure 6.10. This API abstracts the interactive protocol steps into a set of cryptographic algorithms.

- **RealCred and FakeCred:** These functions model the core registration process.
 - The RealCred function encapsulates the entire successful registration session for a single real credential, including CheckIn and CheckOut. It takes the registrar's secret key, and a voter's ID and outputs the voter's secret key c_{sk} , the public credential tag c_{pc} and a proof

TRIP $(\lambda, (G, q, g), V, \mathbb{L}, \mathbb{A}, O, \mathbb{K}, \mathbb{P}, \mathbf{E}, s_{rk})$ <pre> 1: $O, K \leftarrow V(O, \mathbb{K})$ % Voter-chosen/given official and kiosk 2: $t_{in} \leftarrow (V_{id}, _) \leftarrow \text{CheckIn}(O, K, s_{rk}, V_{id})$ 3: $c \leftarrow (q_c, t_{ot}, q_r, e) \leftarrow ((c_{pc}, Y_c, \sigma_{kc}), (V_{id}, c_{pc}, K, \sigma_{k_{ot}}),$ $(V_{id}, c_{sk}, r, K, \sigma_{kr}), (P, e, \sigma_p))$ $\leftarrow \text{RealCred}(\lambda, (G, q, g), K, \mathbb{A}_{pk}, \mathbf{E}, t_{in})$ 4: $\mathbf{e} \leftarrow \{e\}; \mathbf{c} \leftarrow \{c\}$ % Used challenges & vector of credentials 5: for 1 to $(n_c \leftarrow V()) - 1$ do % Chosen number of envelopes 6: $\tilde{c} \leftarrow \text{FakeCred}(\lambda, (G, q, g), K, \mathbb{A}_{pk}, \mathbf{E}_{\ominus \mathbf{e}}, t_{ot})$ 7: $\mathbf{c} \leftarrow \tilde{c}; \mathbf{e} \leftarrow \tilde{c}[e]$ 8: $c_v \leftarrow V(\mathbf{c})$ % Voter-chosen credential for check-out 9: $\mathbb{L} \leftarrow \text{CheckOut}(O, \mathbb{L}, \mathbb{K}, c_v[t_{ot}])$ 10: for i to n_c do 11: $\mathbb{L} \leftarrow \text{Activate}(\mathbb{L}, V, \mathbf{c}_i)$ </pre>

 Figure 6.9: **TRIP** Registration process for a prospective voter.

of correctness P .

- The FakeCred function models the creation of one or more fake credentials, which are all linked to the public tag c_{pc} generated by RealCred.
- Activate: This function models the verification step performed by the voter's VSD. It takes a credential c and its proof P as input and outputs $\top$ (success) or $\perp$ (failure), updating the ledger with the used challenge.
- Vote: This function models the casting of a ballot using a set of activated credentials. For the coercion-resistance proof, it takes a probability distribution D_{n_H, n_M} as input. This distribution, as defined in the JCJ model [56], represents the uncertainty of an honest voter's choice, which is essential for making the coercer's view of the election indistinguishable from a random one.
- Tally and Ideal-Tally: These functions model the final tallying phase.
 - Tally represents the real-world algorithm that computes the final tally X and a proof of correctness P , ensuring the result is verifiable.
 - Ideal-Tally is an idealized function used in the coercion-resistance proof. It is an idealized version of Tally that has the power to ignore ballots cast by the adversary, which is the technique used to prove that the adversary cannot determine whether the targeted voter complied with their demands.

6.3.1 Individual Verifiability

In this section, we provide a formal security analysis of the individual verifiability (IV) property for TRIP. Informally, IV ensures that an honest voter can detect any adversarial attempt to

Chapter 6. TRIP: Trust-Limited Coercion-Resistant In-Person Voter-Registration

- $c_{sk}^i, c_{pc}^i, P^i, \mathbb{L}_R \leftarrow \text{RealCred}(\mathbb{L}_R, R_{sk}, \mathbb{V}_i^{id}, \lambda)$: takes as input the ledger $\mathbb{L}_R$, the registrars' private key R_{sk} , a voter's identifier $\mathbb{V}_i^{id}$ and a security parameter λ and outputs the voter's private and public credential c_{sk}^i and c_{pc}^i , correctness proofs P^i and an updated registration ledger $\mathbb{L}_R$. For simplicity, RealCred incorporates the CheckIn and CheckOut processes required in TRIP.
- $c_f^i, \mathbf{P}^i \leftarrow \text{FakeCreds}(R_{sk}, \mathbb{V}_i^{id}, c_{pc}^i, n_f, \lambda)$: takes as input the registrar's private key R_{sk} , the voter's identifier $\mathbb{V}_i^{id}$, the voter's public credential c_{pc}^i , the number of fake credentials $n_f \in \mathbb{N}$, and a security parameter λ and outputs n_f fake credentials c_f^i and n_f proofs of correctness $\mathbf{P}^i$.
- $out, \mathbb{L}_R \leftarrow \text{Activate}(\mathbb{L}_R, c, P)$: takes as input the registration ledger $\mathbb{L}_R$, a credential c and the credential's correctness proof P and outputs $out \in \{\top, \perp\}$ and an updated registration ledger $\mathbb{L}_R$.
- $\mathbb{L}_V \leftarrow \text{Vote}(c_{sk}, T_{pk}, n_M, D_{n_H, n_M}, \lambda)$: takes as input the set of credentials c_{sk} , the talliers' public key T_{pk} , the candidate list n_M , and a probability distribution D_{n_H, n_M} over the possible (voter, candidate) pairs^a. It appends a TRIP-formatted ballot for each credential in c_{sk} to the ledger $\mathbb{L}_V$.
- $(X, P) \leftarrow \text{Tally}(T_{sk}, \mathbb{L}, n_M, \lambda)$: takes as input the talliers' private key T_{sk} , the ledger $\mathbb{L}$, the candidates n_M , and a security parameter λ and outputs the tally X and a proof P showing that the talliers computed the tally correctly.
- $(X, P) \leftarrow \text{Ideal-Tally}(T_{sk}, \mathbb{L}, n_M, \lambda)$: takes as input the talliers' private key T_{sk} , the ledger $\mathbb{L}$, the candidates n_M , and a security parameter λ and outputs an ideal-tally X and a proof P . This algorithm, as defined in JCJ and only used in the ideal game, differs from Tally by not counting any ballots cast by the adversary if the bit $b = 0$.

^aThis distribution captures the uncertainty of honest voters' choices, which impairs the adversary's ability to detect coercion.

Figure 6.10: **TRIP API.**

tamper with either (1) the credential produced during registration or (2) the ballot that will eventually be tallied. We first revisit the threat model tailored to individual verifiability. We then formalize the IV security game and map TRIP's procedures to the formal model. We then state the main security theorem followed by its proof. We then present the implications of the adversary's advantage that bring us to our new security definitions, *iterative individual verifiability*, and strong iterative individual verifiability, which take into account the adversary's explicit goal of altering the election outcome. We also show that TRIP satisfies strong iterative individual verifiability.

Threat Model

We revisit the threat model described in §5.2.2. The adversary's objective is to cause the final registration or ballot associated with an honest voter on the ledger to be inconsistent with that voter's actions, without this inconsistency being detected by the voter or their VSD.

The honest parties are the voter and their personal device, VSD, which follow the protocol exactly. This includes the voter following the procedural steps required during in-person registration, such as completing the correct interactive sequence of the Σ -protocol.

The integrity adversary $\mathcal{I}$ is computationally bounded and controls every actor other than the voter and their VSD. This includes other voters, registration kiosks, printers, officials, the election authority and the public ledger. In essence, $\mathcal{I}$ has complete, active control over the entire election infrastructure, and consequently possesses all long-term secret keys. The adversary, however, cannot interfere with the physically printed envelopes once the voter has

entered the booth for their registration session without being detected.

The public ledger $\mathbb{L}$ is modeled as an ideal, append-only functionality accessible to all parties. The adversary $\mathcal{J}$ may append arbitrary entries but cannot delete, modify or reorder existing ones.

Formal Definition.

We now present our formal definition, building on prior work such as Swiss Post E-Voting System [117], and CH-Vote [18]. We first define the syntax of a generic voting scheme Π — an election scheme without tallying — and then use it to construct our security game, IV.

A voting scheme Π is a tuple of four algorithms:

- $\text{Setup}_{\mathcal{J}}(\text{params}) \rightarrow (\text{p}, \mathbb{L}, \mathbb{V}, \text{st}_{\mathcal{J}})$: An algorithm run by the adversary $\mathcal{J}$ that takes as input the game's public parameters params which are assumed to be honestly generated and hardcoded into the game. It outputs scheme-specific public parameters p , the initial ledger state $\mathbb{L}$, the list of the eligible voters $\mathbb{V}$ and the adversary's state $\text{st}_{\mathcal{J}}$ containing any secret keys (like the authority's secret key $\mathbb{A}_{\text{sk}}$ to determine the game's outcome).
- $\text{VoterInteraction}(V \leftrightarrow \text{infra})$: An interactive protocol between the honest voter's agency (represented by an oracle, V) and the malicious infrastructure *infra* controlled by the adversary $\mathcal{J}$. This protocol encapsulates registration and voting, resulting in a voter receipt π , and a possibly modified ledger $\mathbb{L}$.
- $\text{VoterVerify}(\text{params}, \text{p}, \mathbb{L}, \pi) \rightarrow \{\top, \perp\}$: A deterministic algorithm executed on VSD. It takes as input the public parameters params , the scheme-specific parameters p , the final public ledger $\mathbb{L}$ to be used for tallying, and the voter's receipt π . It outputs $\top$ (accept) if the voter's interactions are correctly reflected on the ledger, and $\perp$ (reject) otherwise.
- $\text{Extract}(\text{params}, \text{p}, \mathbb{L}, \text{st}_{\mathcal{J}}, V_{id}) \rightarrow \text{goal}$: A deterministic algorithm used only inside the security game that takes as input the public parameters params , scheme-specific parameters p , the final ledger $\mathbb{L}$, the adversary's state $\text{st}_{\mathcal{J}}$ and a voter identifier V_{id} . It outputs the recorded information associated with that voter V_{id} on the ledger or $\perp$ if no valid record exists.

We formalize individual verifiability using the security game Game IV, shown in fig. 6.11. The game begins with the adversary $\mathcal{J}$ running Setup. Following this, $\mathcal{J}$ chooses a target voter V_{id}^* , an honest intent for that voter, and a malicious goal it wishes to have recorded on the ledger, where $\text{goal} \neq \text{intent}$.

The core of the game is the interaction phase, where $\mathcal{J}$ is given oracle access to $V(V_{id}^*, \text{intent})$ which perfectly simulates the honest voter's actions. At the end of the interaction, $\mathcal{J}$ outputs the voter's receipt π and the final ledger $\mathbb{L}$.

The adversary wins if its malicious goal is recorded on the ledger (Extract returns goal) *and* the voter's local verification on VSD passes (VoterVerify returns $\top$). We define the advantage

Game $IV_{\Pi, \mathcal{J}}(\text{params})$	
1:	$(p, \mathbb{L}, \mathbb{V}, \text{st}_{\mathcal{J}}) \leftarrow \Pi.\text{Setup}_{\mathcal{J}}(\text{params})$
2:	$(V_{\text{id}}^*, \text{intent}, \text{goal}) \leftarrow \mathcal{J}(\text{Choose Target \& Goal})$
3:	$(\pi, \mathbb{L}) \leftarrow \mathcal{J}^{V(V_{\text{id}}^*, \text{intent})}(\mathbb{L})$
4:	$\text{happy} \leftarrow \Pi.\text{VoterVerify}(\text{params}, p, \mathbb{L}, \pi)$
5:	$\text{adv_outcome} \leftarrow (\text{Extract}(\text{params}, p, \mathbb{L}, \text{st}_{\mathcal{J}}, V_{\text{id}}^*) = \text{goal})$
6:	return 1 if $(\text{happy} \wedge \text{adv_outcome} \wedge (\text{goal} \neq \text{intent}))$ else 0

Figure 6.11: **Individual Verifiability (Ind-Ver)**. An adversary wins if it can make the ledger reflect its malicious goal for an honest voter, without the voter’s verification procedures detecting the manipulation.

of an adversary $\mathcal{J}$ against individual verifiability of scheme Π as:

$$Adv_{\Pi, \mathcal{J}}^{IV} = \Pr[\text{Game } IV_{\Pi, \mathcal{J}} = 1]$$

Protocol Instantiation.

We now map the abstract syntax to the concrete procedures of Π_{TRIP} .

The global public parameters are $\text{params} = (G, q, g, H)$ (G, q, g) is a cyclic group of prime order q where the DDH problem is hard, and $H: \{0, 1\}^* \rightarrow \mathbb{Z}_q$ is a hash function. These are assumed to be honestly set to prevent the adversary from winning vacuously.

Setup is executed by $\mathcal{J}$, taking params as input. $\mathcal{J}$ may generate arbitrary keys for the election authority, officials, kiosks, and printers, and may populate the ledger $\mathbb{L}$ and voter list $\mathbb{V}$ in any way. It outputs the scheme-specific parameters p which would include the public keys of all parties.

The VoterInteraction abstract protocol is instantiated by two concrete interactions: the in-person TRIP registration procedure (fig. 6.9, lines 1-9), and the subsequent “at-home” Vote procedure (fig. 7.1). The adversary $\mathcal{J}$ controls the infrastructure side of both interactions. For the registration procedure, the voter — not their VSD— performs the interactive choices, such as selecting the number of credentials n_c (line 4), and the final credential for check-out c_v (line 8). The pick of n_c envelopes is a sequential process, where the voter picks one envelope for RealCred and then $n_c - 1$ envelopes for FakeCred. The voter aborts the game, returning 0, if the adversary deviates from the Σ -protocol during RealCred. For the voting procedure, VSD also keeps the computed ballot B in memory for the VoterVerify procedure.

The voter receipt π corresponds to the full set of data contained in the n_c physical paper credentials (QR codes) the voter takes home after registration.

The abstract VoterVerify($\text{params}, p, \mathbb{L}, \pi$) protocol is instantiated as a procedure run on the

voter's trusted VSD, taking the final public ledger $\mathbb{L}$ to be tallied and the registration receipt π as input. It returns $\top$ if and only if both of the following checks succeed:

- **Registration Verification:** VSD runs the Activate procedure for every credential contained in the receipt π (fig. 6.9, lines 10-11). This includes verifying the correctness of the zero-knowledge proof constructed during RealCred, and the first-time redemption of the envelope's nonce against $\mathbb{L}_E$. This check succeeds only if this loop completes for all credentials created without aborting.
- **Vote Verification:** VSD retrieves the locally stored ballot B_{local} that it cast during the Vote procedure. It queries the ledger $\mathbb{L}$ for the ballot associated with the voter's credential c_{pk} . Let this be B_{posted} . This check succeeds only if the ballot posted on the ledger B_{posted} is bit-for-bit identical to B_{local} . If multiple ballots are posted for c_{pk} it retrieves the one that will be tallied — based on some policy, which for this instantiation, we set as the last ballot cast counts.

The game's Extract algorithm determines the final outcome for a voter as recorded on the ledger. It first parses the registration ledger $\mathbb{L}_R[V_{id}]$ to find the public credential c_{pc} . It then decrypts c_{pc} using the authority key $\mathbb{A}_{\text{sk}}$ from $\text{st}_{\mathcal{J}}$ to recover the associated credential public key, c'_{pk} . It continues by finding the ballot that counts (i.e., last ballot cast) B_{posted} on the voting ledger $\mathbb{L}_V$ associated with c'_{pk} . Finally, it decrypts B_{posted} using $\mathbb{A}_{\text{sk}}$ to recover the vote message m' . If any of the steps fail, it returns $\perp$. Otherwise, it outputs the tuple (c'_{pk}, m') . The intent and goal are thus defined as tuples of the voter's credential public key and the choice of vote.

Security Theorem.

We now state our main theorem regarding the individual verifiability of our protocol.

Theorem 1. *Let Π_{TRIP} be instantiated as described in §6.3.1. If the underlying Σ -protocol is computationally sound (which relies on the hardness of the Discrete Logarithm Problem), and the ElGamal scheme is perfectly binding, then, for any PPT adversary $\mathcal{J}$ playing Game IV, its advantage $\text{Adv}_{\Pi_{\text{TRIP}}, \mathcal{J}}^{\text{IV}}(\lambda)$ is bounded by:*

$$\text{Adv}_{\Pi_{\text{TRIP}}, \mathcal{J}}^{\text{IV}}(\lambda) \leq \max_{1 \leq k \leq n_E} E_{n_c \sim D_C} \left[\frac{k}{n_E} \cdot \frac{\binom{n_E - k}{n_c - 1}}{\binom{n_E - 1}{n_c - 1}} \right] + \text{negl}(\lambda)$$

where n_E is the total number of physical envelopes present in the booth during the voter's registration, k is the number of those envelopes maliciously prepared by $\mathcal{J}$, n_c is the number of credentials the voter chose to create, and D_C is the distribution of the voter's choice for n_c .

Proof. Let Win be the event that $\mathcal{J}$ wins the Game IV. In other words, Win means that VoterVerify returns $\top$ and Extract returns the adversary's goal. The adversary can win by tampering with registration or by tampering with the vote. Let RegSub be the event that

Chapter 6. TRIP: Trust-Limited Coercion-Resistant In-Person Voter-Registration

the adversary's goal for the credential key, $\text{goal}.c_{pk}$, differs from the honest intent, $\text{intent}.c_{pk}$: $\text{goal}.c_{pk} \neq \text{intent}.c_{pk}$. Let VoteSub be the event that $\text{goal}.m \neq \text{intent}.m$ where m represents the vote.

For the adversary to win, the goal must differ from intent, which means that either RegSub or VoteSub (or even both) must occur. Therefore, the Win event is a sub-event of $(\text{RegSub} \vee \text{VoteSub})$, allowing us to partition the Win event into two disjoint cases based on whether registration was tampered. By the law of total probability, we get:

$$\begin{aligned} \Pr[\text{Win}] &= \Pr[\text{Win} \wedge (\text{RegSub} \vee \text{VoteSub})] \\ &= \Pr[\text{Win} \wedge \text{RegSub}] \\ &\quad + \Pr[\text{Win} \wedge \neg \text{RegSub} \wedge \text{VoteSub}] \end{aligned}$$

We bound each term in a separate lemma, lemma 1 and lemma 2, respectively.

Lemma 1. *The probability of an adversary $\mathcal{J}$ winning Game IV via registration tampering is bounded by:*

$$\Pr[\text{Win} \wedge \text{RegSub}] \leq \max_{1 \leq k \leq n_E} E_{n_c \sim D_C} \left[\frac{k}{n_E} \cdot \frac{\binom{n_E - k}{n_c - 1}}{\binom{n_E - 1}{n_c - 1}} \right] + \text{negl}(\lambda)$$

Proof. The proof proceeds via a game hop argument. Let Win_i be the event that the adversary wins Hybrid i .

Hybrid 0. This is the real Game IV. The adversary $\mathcal{J}$ interacts with the game as defined in the TRIP protocol. The adversary's success probability is $\Pr[\text{Win}_0 \wedge \text{RegSub}]$.

Hybrid 1. This hybrid is identical to hybrid 0 except that it aborts if the adversary $\mathcal{J}$ produces two different plaintexts (e.g., credential public keys) that are valid openings to the same ElGamal ciphertext c_{pc} . Since ElGamal is perfectly binding, this event is information-theoretically impossible. Thus,

$$|\Pr[\text{Win}_0 \wedge \text{RegSub}] - \Pr[\text{Win}_1 \wedge \text{RegSub}]| = 0$$

Hybrid 2. This hybrid is identical to hybrid 1 except we replace the Σ -protocol with an idealized proof system that aborts if $\mathcal{J}$ tries to produce a proof for a false statement. The statement being proven in RealCred is knowledge of the randomness x used to create c_{pc} as the encryption of c_{pk} .

$$\text{ZKPoE}_{C_1, X} \{(x) : C_1 = g^x \wedge X = \mathbb{A}_{pk}^x\}$$

where $c_{pc} = (C_1, C_2)$ and $c_{pk} = \frac{C_2}{X}$. An adversary attempting to tamper with registration wants to get on the ledger c_{pc} that encrypts some other c'_{pk} — known only to $\mathcal{J}$ — instead of the voter's intended public key c_{pk} . This is a false statement, so the probability that the adversary wins in Hybrid 2 is

$$\Pr[\text{Win}_2 \wedge \text{RegSub}] = 0$$

We now wish to bound the advantage of $\mathcal{J}$ between Hybrid 1 and Hybrid 2. To do this, we must adapt the soundness analysis of the Schnorr Σ -protocol to our setting where adversarially chosen envelopes are used for the challenge instead of a uniformly random group element. The adversary can break soundness by either guessing the envelope challenge or finding the corresponding discrete log that allows them to produce the desired ciphertext. This second case is oblivious to how the challenge is chosen, and puts us back in the standard computational soundness analysis of the Schnorr Σ -protocol. By reduction to the hardness of the Discrete Logarithm Problem, the probability of generating a valid proof in this case is negligible.

Now we must bound the probability that $\mathcal{J}$ is able to guess the challenge e that will be selected by the voter. Recall that our threat model allows $\mathcal{J}$ to adversarially select the set of envelopes available to the voter. We first show that the best strategy that $\mathcal{J}$ can adopt is to have a malicious set of credentials $\mathcal{M}$ containing the same repeated nonce, and an honest set $\mathcal{H}$ consisting of envelopes with a unique nonce.

Consider an arbitrary strategy where the adversary prepares n_E envelopes consisting of a malicious set $\mathcal{M}$ and an honest set $\mathcal{H}$. We partition $\mathcal{M}$ into subsets $\mathcal{M}_1, \dots, \mathcal{M}_k$ where $\mathcal{M}_i$ contains various envelopes with the same nonce e_i^* . Now, when guessing the nonce the voter will choose, let p_i be the probability that the adversary guesses e_i^* . Then, let $P = \sum p_i$, so the probability that the adversary tries to guess an envelope from $\mathcal{H}$ is $1 - P$. The adversary wins if it correctly guesses the envelope picked by the voter for their real credential *and* the voter only picks envelopes with unique nonces. We show that this probability is maximal when all the envelopes in $\mathcal{M}$ have the same repeated nonce.

Now assume the voter creates n_c total credentials. We can upper bound the probability that the adversary correctly guesses the voter's real credential without getting caught as follows. Let EnvGuess be the event where the adversary correctly guesses the nonce and let DupCatch be the event where the voter finds duplicate nonces. First, we lower bound the probability that the adversary gets caught. Note that for all $\mathcal{M}_i$, this probability is at least the probability that it gets caught on a specific $\mathcal{M}_i$. This tells us that

$$\Pr[\text{DupCatch} \mid \text{EnvGuess}] \geq \max_i \Pr[\mathcal{J} \text{ caught on } \mathcal{M}_i \mid \text{EnvGuess}]$$

which gives us that

$$\Pr[\neg \text{DupCatch} \mid \text{EnvGuess}] \leq 1 - \max_i \Pr[\mathcal{J} \text{ caught on } \mathcal{M}_i \mid \text{EnvGuess}]$$

Then, the probability of success is

$$\Pr[\neg \text{DupCatch} \wedge \text{EnvGuess}]$$

$$\leq (1 - \max_i \Pr[\mathcal{J} \text{ caught on } \mathcal{M}_i \mid \text{EnvGuess}]) \cdot \Pr[\text{EnvGuess}]$$

By Union Bound:

$$\begin{aligned} &\leq (1 - \max_i \Pr[\mathcal{J} \text{ caught on } \mathcal{M}_i \mid \text{EnvGuess}]) \cdot \left(\sum_{i=1}^k p_i \frac{|\mathcal{M}_i|}{n_E} \right) \\ &\leq (1 - \max_i \Pr[\mathcal{J} \text{ caught on } \mathcal{M}_i \mid \text{EnvGuess}]) \cdot \left(\sum_{i=1}^k \frac{|\mathcal{M}_i|}{n_E} \right) \\ &\leq (1 - \max_i \Pr[\mathcal{J} \text{ caught on } \mathcal{M}_i \mid \text{EnvGuess}]) \cdot \frac{|\mathcal{M}|}{n_E}. \end{aligned}$$

Now, note that if $\mathcal{M}$ consists of envelopes with a single repeated nonce, then this actually becomes an equality. We can therefore conclude that an adversary gains nothing from using various repeated nonces.

Now, we analyze the optimal strategy, which is:

1. $\mathcal{J}$ prepares n_E envelopes composed of two sets: a malicious set $\mathcal{M}$ of k envelopes containing the same, pre-determined nonce e^* , and an honest set $\mathcal{H}$ of $n_E - k$ envelopes containing unique nonces.
2. During the RealCred interaction, $\mathcal{J}$ must commit to Y_c *before* seeing the voter's challenge e . Therefore, to prove a false statement that c_{pk} encrypts $\text{intent}.c_{pk}$ when it actually encrypts $\text{goal}.c_{pk}$, $\mathcal{J}$ must guess the challenge e that the voter will choose. Thus, $\mathcal{J}$ hopes that the voter chooses an envelope from $\mathcal{M}$ (containing e^*) for this step.
3. The VoterVerify procedure checks for reused nonces across all envelopes posted on the ledger $\mathbb{L}_E$. Therefore, to pass this check, $\mathcal{J}$ must hope that the voter during FakeCred picks only envelopes from the honest set $\mathcal{H}$.

The adversary wins if the voter chooses an envelope from $\mathcal{M}$ for RealCred — probability of $\frac{k}{n_E}$ *and* chooses the subsequent $n_c - 1$ envelopes for FakeCred entirely from $\mathcal{H}$. Given the first choice was from $\mathcal{M}$, there are $n_E - 1$ envelopes left, of which $n_E - k$ are in $\mathcal{H}$. The probability of the second event is then:

$$\frac{\binom{n_E - k}{n_c - 1}}{\binom{n_E - 1}{n_c - 1}}$$

Given that these are two distinct and ordered probabilistic events, the total advantage of the adversary in Hybrid 1 over Hybrid 2 (and thus its probability of success) is, maximizing over $\mathcal{J}$'s choice of k and averaging over the voter's choice of n_c , is:

$$\Pr[\text{Win}_1 \wedge \text{RegSub}] = \max_{1 \leq k \leq n_E} E_{n_c \sim D_C} \left[\frac{k}{n_E} \cdot \frac{\binom{n_E - k}{n_c - 1}}{\binom{n_E - 1}{n_c - 1}} \right] + \text{negl}$$

This probability represents the adversary's optimal strategy in a game of uncertainty. The

adversary must choose the number of malicious envelopes, k , without knowing how many credentials, n_c , the voter will choose to create. A greedy strategy of setting $k = n_c$ (all envelopes being malicious) would guarantee a win if the adversary knew that $n_c = 1$, as the nonce-reuse check in VoterVerify would never detect duplicates assuming the adversary issues new envelopes after each registration session. However, this strategy fails spectacularly if $n_c > 1$, as the VSD would immediately detect a nonce-reuse on the activation of the second credential. Since $\mathcal{J}$ does not know n_c , it must balance the potential reward of subverting the RealCred interaction against the risk of detection during FakeCred procedures. This formula captures the maximum expected success for $\mathcal{J}$, averaged over the voter's behavioral distribution D_C .

Summing this statistical term with the negligible terms from the game hops conclude the proof of this lemma. $\square$

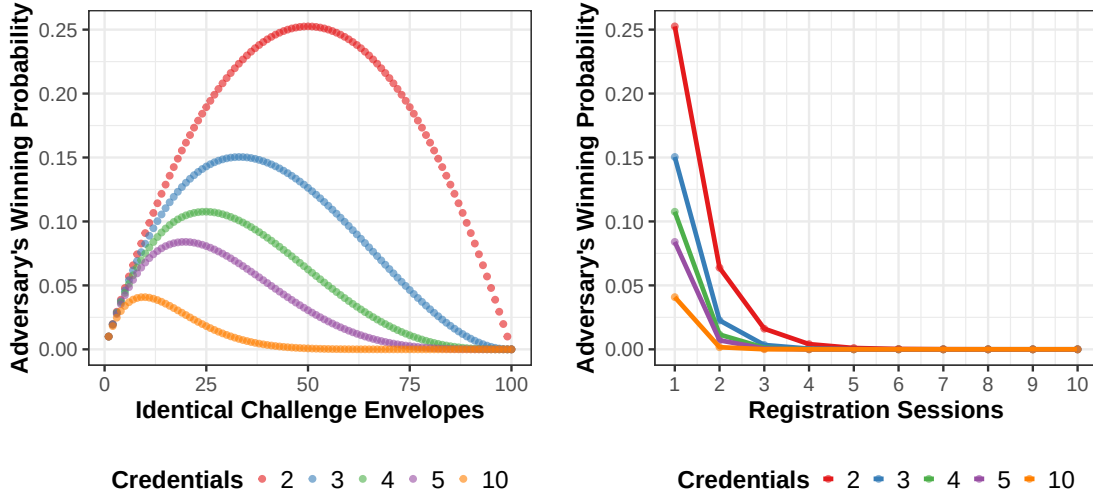
Lemma 2. *Given that registration was not subverted ($\neg \text{RegSub}$), the probability of an adversary winning via vote manipulation is 0. $\Pr[\text{Win} \wedge \neg \text{RegSub} \wedge \text{VoteSub}] = 0$*

Proof. We proceed by direct argument. The condition $\neg \text{RegSub}$ implies that the public credential c_{pc} on the ledger correctly encrypts the honest credential key c_{pk} that the voter possesses. The adversary $\mathcal{J}$, however, knows the credential's secret key as it generated the key pair (c_{sk}, c_{pk}) on the kiosk during RealCred. The adversary's goal is to have the final tallied vote be $\text{goal}.m$ while the voter's intent was $\text{intent}.m$, without the VoterVerify check failing. We consider all possible strategies for $\mathcal{J}$.

The first strategy is to submit a ballot on the ledger given $\mathcal{J}$ knowledge of c_{sk} . $\mathcal{J}$ constructs and signs a ballot $B_{\mathcal{J}}$ for the vote $\text{goal}.m$. $\mathcal{J}$ then sends this ballot to ledger L_V such that it will be the one extracted for the tally. The game requires $\mathcal{J}$ to output the final ledger L before VoterVerify is run, as a means of mimicking continuous checking by VSD. If $\mathcal{J}$ posts only its ballot $B_{\mathcal{J}}$, the voter's VSD will detect a ballot B associated with c_{pk} on L_V when no vote was cast and fail. If the voter's ballot is posted B_V , and then $\mathcal{J}$ posts $B_{\mathcal{J}}$, VoterVerify will compare the last ballot $B_{\mathcal{J}}$ with its locally stored ballot B_V . Since $B_V \neq B_{\mathcal{J}}$, the check will fail. If $\mathcal{J}$ posts $B_{\mathcal{J}}$ and then the voter's ballot is posted B_V , the last ballot will be B_V , and the Extract function will find $\text{intent}.m$, not the adversary's $\text{goal}.m$. Therefore, $\mathcal{J}$ does not win in all cases.

The other strategy is to modify the decryption of the ballot. $\mathcal{J}$ intercepts or appends a new ballot based on the voter's honestly generated ballot B_V (containing an encryption of $\text{intent}.m$) and attempts to modify it into $B_{\mathcal{J}}$ such that $B_{\mathcal{J}}$ decrypts to $\text{goal}.m$. For VoterVerify to pass, $B_{\mathcal{J}}$ must be bit-for-bit identical to the original B_V that the VSD stored locally. However, since the ElGamal encryption is perfectly binding, $\mathcal{J}$ cannot get the ciphertext to decrypt to anything but the voter's plaintext $\text{intent}.m$. It is information-theoretically impossible to find a different message $\text{goal}.m$ that is a valid opening of the same ciphertext. Therefore, any modification that changes the underlying vote will necessarily change the ciphertext, causing the bit-for-bit check in VoterVerify to fail. The probability of success for this strategy is 0.

In all scenarios, the combination of the VSD's bit-for-bit check and the perfect binding property



(a) Adversary's Winning Probability based on The Number of Identical Envelopes & Number of Credentials Voter Creates

(b) Adversary's Winning Probability across Voter Registration Sessions & Number of Credentials Voter Creates.

Figure 6.12: **Adversary's Winning Probability for 100 Envelopes.**

of the encryption scheme ensures that any attempt by $\mathcal{A}$ to change the vote after a successful registration will be detected. The probability of success is therefore 0. $\square$

We now combine the results of Lemma 1 and Lemma 2 to complete the proof. The total advantage of the adversary $\mathcal{A}$ is the sum of its advantages in the two disjoint scenarios we analyzed:

$$\begin{aligned}
 \text{Adv}_{\Pi_{\text{TRIP}}, \mathcal{A}}^{\text{IV}} &= \Pr[\text{Win}] \\
 &= \Pr[\text{Win} \wedge \text{RegSub}] + \Pr[\text{Win} \wedge \neg \text{RegSub} \wedge \text{VoteSub}] \\
 &\leq \left(\max_{1 \leq k \leq n_E} \mathbb{E}_{n_c \sim D_C} \left[\frac{k}{n_E} \cdot \frac{\binom{n_E - k}{n_c - 1}}{\binom{n_E - 1}{n_c - 1}} \right] + \text{negl}(\lambda) \right)
 \end{aligned}$$

This concludes the proof of the theorem. $\square$

Implications of the Security Bound

Beyond the proof, we can build intuition for the adversary's security bound by analyzing their strategic options. To subvert a voter's registration without being detected cryptographically, the adversary must forge a zero-knowledge proof during the RealCred procedure. This requires knowing the voter's ZKP challenge *before* committing to the proof, as is done for FakeCred. The adversary's only viable strategy to achieve this is to "poison" the deck of envelopes in such

a way that they can predict which challenge the voter will select.

The Adversary’s Dilemma. The core of the adversary’s strategy is to place multiple envelopes containing the same, pre-selected nonce e^* in the booth: the malicious set $\mathcal{M}$. Their hope is that the voter, when creating their real credential, will happen to pick up one of these malicious envelopes. This strategy, however, presents a fundamental dilemma, a trade-off between maximizing the chance of success and minimizing the risk of immediate detection. This is precisely the dynamic captured by the security bound in theorem 1 and visualized in Figure 6.12.

Consider the adversary’s choice of how many malicious envelopes, k , to place in the booth out of a total of n_E . As an adversary increases k , the probability that the voter selects a malicious envelope for their real credential, $\Pr[e^* \in \mathcal{M}]$, increases linearly k/n_E . This is the adversary’s path to a successful tampering. The voter’s primary defense is the creation of fake credentials — not just individual verifiability, but also for coercion-resistance! For each fake credential, the voter picks another envelope from the remaining pool and if *any* of these additional draws also comes from the malicious set, then VSD will detect a reused nonce during activation and the attack will fail. As k increases, so does the probability of VSD’s detection.

This tension is illustrated in Figure 6.12a. For any given number of credentials the voter creates, the adversary’s winning probability initially rises with k , but then decreases as the risk of detection increases. The peak of each curve represents the adversary’s optimal choice of k , but the adversary does not know which curve they are on when selecting k as it depends on the voter’s choice of n_c . The expectation over the voter’s choice distribution D_X captures this uncertainty and significantly limits the adversary’s real-world advantage. The user study (part III) demonstrates the soundness of this notion: 76% of participants chose to create at least one fake credential, with 53% stating they would create fake credentials if such a system existed today.

Large-Scale Fraud. The adversary’s difficulty in winning compounds dramatically when attempting to influence an entire election. To succeed, they must win the registration gamble not just for one targeted voter, but for *every* single voter they target, as a single detected failure could expose the entire operation. If the maximum probability of deceiving a single voter is $p_{\max}$, the probability of deceiving N independent voters is $p_{\max}^N$.

This exponential decay is visualized in fig. 6.12b. Even with an optimistic success probability of $\sim 25\%$ for a single voter (the peak for $n_c = 2$), the chance of successfully subverting just three such voters undetectably drops to less than 2%. This shows that the success probability makes large-scale fraud infeasible against a population of even moderately cautious voters. This fact, where security against one is amplified into security against many, is what motivates our concept of *iterative individual verifiability* (§6.3.1).

Additional Adversarial Behaviors. An adversary might exploit human biases — humans being non-random — by, for example, placing their malicious envelopes at the top of the stack.

Game I-IV$^{\lambda_v}_{\Pi, \mathcal{J}}$ (params) <pre> 1: $\mathbb{V} \leftarrow \mathcal{J}$ (% Compile electoral roll) 2: successes $\leftarrow 0$ 3: for V_{id} in $\mathbb{V}$ do 4: successes $+=$ Game IV$^*_{\Pi, \mathcal{J}}$(params, V_{id}) 5: endfor 6: return 1 if (successes $\geq \lambda_v$) else 0 </pre>

Figure 6.13: **Iterative Individual Verifiability (I-IV)** Definition of iterative individual verifiability to account for an adversary’s goal of altering an election outcome.

However, even if this could double or triple the success rate against one voter, it provides only a constant-factor advantage against an exponential problem when targeting many voters.

The adversary could also program the kiosk to abort if the voter selects an “un-guessed” envelope. This, however, is not an undetectable subversion but rather an observable and undeniable failure. A single voter reporting such behavior is sufficient to prove misconduct. The authorities are capable of challenging the kiosk to reveal the ZKP commitment secret y for the Y_c it printed in the first QR code (fig. 6.5, line 5) — a computationally infeasible task if it cheated. If the kiosk refuses to reveal y or provides an incorrect value, authorities gain unequivocal proof of misconduct.

Ultimately, while the adversary has a non-negligible advantage of winning against a single voter, this advantage becomes negligible when attempting to scale the attack.

Iterative Individual Verifiability

While the IV game establishes the security for a single voter, the adversary’s true objective is not to deceive one voter but to deceive enough voters to alter an election’s final outcome. This requires a model of security that accounts for an adversarial campaign across a population of voters. To change an election outcome, they must successfully and undetectably subvert a number of votes λ_v equal to or greater than the election’s margin of victory.

We explore this requirement through two complementary views. First, we discuss a general model applicable to any voting protocol, resulting in a standard binomial security bound. Second, we leverage a specific property of TRIP— the ability to produce non-repudiable proof of misbehavior — to define a “high-stakes” game, S-I-IV, which yields a much stronger exponential security bound for TRIP.

Formal Definition: Iterative Individual Verifiability. We define the I-IV property using the security game Game I-IV shown in fig. 6.13. This game is a wrapper that repeatedly calls a voter-targeted version of our base IV game, which we now formally define.

The I-IV game requires a sub-routine that forces the adversary to attack a specific, pre-determined voter. We denote this game as Game IV*. It is identical in all respects to Game IV from Figure 6.11 but with two modifications: First, the game now takes as an additional input parameter V_{id} and hardcodes the target voter as $V_{id}^* \leftarrow V_{id}$. Then, the step $(V_{id}^*, \text{intent}, \text{goal}) \leftarrow \mathcal{J}(\text{Choose Target \& Goal})$ becomes $(\text{intent}, \text{goal}) \leftarrow \mathcal{J}(\text{Choose Goal})$.

With this sub-game defined, we now present the main game in fig. 6.13. In this game, the adversary first commits to an electoral roll $\mathbb{V}$. The challenger then executes $|\mathbb{V}|$ independent rounds of IV*, one for each voter $V_{id} \in \mathbb{V}$. The adversary wins if they accumulate at least λ_v successful attacks.

General Model for Iterative Attacks For any voting scheme where a detected failure might be deniably attributed to a “systems fault”, an adversary’s campaign can be modeled as a sequence of Bernoulli trials. An adversary might therefore attack $n_v > \lambda_v$ voters, tolerating some failed attempts to maximize their chance of reaching λ_v wins. Each attempt against a single voter is a trial with maximum success probability $p_{\max}$. The adversary’s success in this scenario is bounded by the tail property of the binomial distribution $B(n_v, p_{\max})$:

$$\Pr[\text{successes} \geq \lambda_v] \leq \sum_{i=\lambda_v}^{n_v} \binom{n_v}{i} (p_{\max})^i (1 - p_{\max})^{n_v-i}$$

This bound reflects a weaker security guarantee, as an adversary can trade a higher number of attempts for a greater probability of success.

Strong Iterative Individual Verifiability TRIP provides a much stronger guarantee that allows us to move beyond the general binomial model. Specifically, any detected attempt at tampering provides non-repudiable proof of the adversary’s malicious actions. We now show this in the following lemma.

Lemma 3. *Let $\perp$ be the event that VoterVerify returns reject for an honest voter’s interaction in TRIP. This event constitutes non-repudiable cryptographic proof of misbehavior, which an adversary can only deny culpability for this event by breaking the hardness of the Discrete Logarithm Problem or the EUF-CMA security of the underlying signature scheme.*

Lemma 3 Proof Sketch. A rejection by VoterVerify($\perp$) during the registration phase of TRIP arises from a failed attempt to deceive the voter. We can disregard failures during the voting phase, as lemma 2 of theorem 1 shows that the adversary’s success probability is 0 in that phase. The registration failures can be categorized into two main types: ZKP soundness failure and nonce-reuse detection.

As established in theorem 1, to cheat the RealCred ZKP, the adversary must commit to Y_c before seeing the voter’s challenge e . This forces the kiosk to guess a challenge e^* . If the voter selects an envelope with a different challenge $e \neq e^*$, the adversary has two options:

First, it may abort the protocol. The voter is left with an incomplete but signed receipt (i.e.,

Chapter 6. TRIP: Trust-Limited Coercion-Resistant In-Person Voter-Registration

the first QR containing the commitment Y_c). Upon reporting the voter reporting the kiosk's malfunction, authorities can challenge the kiosk to reveal the ZKP secret y corresponding to Y_c . A malicious kiosk, having computed Y_c to cheat, will be unable to produce a valid y that satisfies the ZKP commitment equation: $Y_c = (Y_1, Y_2) = (g^y, \mathbb{A}_{pk}^y)$, which is a failure reducible to the hardness of the DLP.

The adversary may also complete the protocol with an invalid proof. If the adversary provides an invalid ZKP response r on the receipt, the voter's VSD will detect this during the Activate procedure, triggering the $\perp$ event. The voter now possesses, at the minimum, a receipt containing a signed ZKP commitment in the first QR code and an invalid ZKP response r in the third QR code. This is also non-repudiable proof that the kiosk acted maliciously unless the adversary can shift the blame to a third party by demonstrating that they can break the EUF-CMA security of the signature scheme to claim forgery.

The second class of attacks is the nonce-reuse detection. Suppose the adversary successfully guesses the challenge for RealCred (i.e., the voter picks an envelope with e^*). They must still pass the $n_c - 1$ FakeCred checks. If the voter selects another malicious envelope containing e^* , the kiosk similarly has two options: abort or complete the procedure.

If the kiosk aborts during FakeCred, the authorities check that each envelope used has a unique challenge. Furthermore, for additional verification, the authorities can perform the same ZKP soundness check as described earlier using the voter's stated real credential. An honest kiosk can always comply with the challenges faced by the authorities by retaining the secrets produced during their interaction with the voter until the end of that interaction.

If the kiosk completes the protocol, the VSD will later detect a nonce re-use during the Activate procedure, triggering a $\perp$. This detection is non-repudiable because each physical envelope is supposed to contain a unique challenge, signed by an (adversary-controlled) envelope printer. The voter can now present these distinct envelopes with the same challenge to the authorities, providing a direct proof of malicious behavior.¹

In all scenarios, a $\perp$ event provides the voter and the authorities with cryptographic evidence of misbehavior. Therefore, the adversary's failure to tamper is non-repudiable, bounded only by the negligible probability of breaking DLP and the EUF-CMA property of the underlying signature scheme. $\square$

This lemma 3 allows us to model a game where the entire campaign fails upon a single detection. Furthermore, it implies that a rational adversary has no incentive to continue attacking once their goal of λ_v wins is met, as any further attempts only add risk for any $p_{\max} < 1$. We formalize this with the game S-I-IV.

Formal Definition: Strong Iterative Individual Verifiability. We now formalize our notion of

¹Our threat model assumes voters are honest and will not duplicate envelopes to cause disruption. To mitigate such an attack in practice, physical security measures like unique watermarks on the envelopes could be employed by the envelope printer.

Game S-I-IV $^{\lambda_v}_{\Pi, \mathcal{J}}$ (params)
1: $\mathbb{V} \leftarrow \mathcal{J}$ (% Compile electoral roll) 2: successes $\leftarrow 0$ 3: for V_{id} in $\mathbb{V}$ do 4: $r = \text{Game IV}^\dagger_{\Pi, \mathcal{J}}(\text{params}, V_{id})$ 5: if $r = \perp$ then return 0 % Adversary caught 6: successes $+= r$ 7: endfor 8: return 1 if (successes $\geq \lambda_v$) else 0

Figure 6.14: **Strong Iterative Individual Verifiability (S-I-IV)** Definition of strong iterative individual verifiability to account for an adversary's immediate loss if caught.

strong iterative individual verifiability using the following game S-I-IV shown in fig. 6.14. This game models a sequential and adaptive attack where the adversary $\mathcal{J}$ must achieve a target, λ_v wins, without a single detection.

To define the outcome of each round in IV, we define another sub-game $\text{IV}^\dagger$, which adopts the same modifications as IV^* with the one additional modification: IV returns three values: 1 for win, $\perp$ for detection, and 0 for benign loss. Concretely, for detection ($\perp$), this means that goal $\neq$ intent, adv_outcome is $\top$ and happy (VoterVerify) is $\perp$.

Theorem 2. Let Π_{TRIP} be instantiated as described in §6.3.1, and $p_{\max} = \text{Adv}^{\text{IV}}_{\Pi_{TRIP}, \mathcal{J}}$ be the maximum advantage in the IV game. Under the hardness of the Discrete Logarithm problem and the EUF-CMA security of the signature scheme, for any PPT adversary $\mathcal{J}$ playing the S-I-IV game with a success threshold of λ_v , its advantage is bounded by:

$$\text{Adv}^{\text{S-I-IV}}_{\Pi_{TRIP}, \mathcal{J}}(\lambda_v, \lambda) \leq (p_{\max})^{\lambda_v} + \text{negl}(\lambda)$$

Proof. Let Win be the event that Game S-I-IV returns 1. Our goal is to bound $\Pr[\text{Win}]$. Let $R_i \in \{1, 0, \perp\}$ be the random variable for the outcome of the $\text{IV}^\dagger$ game against voter V_i . The adversary wins if

$$\left(\bigwedge_{i=1}^{|\mathcal{V}|} R_i \neq \perp \right) \wedge \left(\sum_{i=1}^{|\mathcal{V}|} \mathbf{1}_{R_i=1} \geq \lambda_v \right)$$

The proof proceeds via two lemmas.

Lemma 4. For any round $i \in \{1, \dots, |\mathcal{V}|\}$ and for any PPT adversary $\mathcal{J}$, the probability of success in that round is bounded by $p_{\max}$, conditioned on the event that no prior round resulted in a detection, and on the full transcript $\mathbb{T}_1^{i-1}$. Formally,

$$\Pr[R_i = 1 \mid \bigwedge_{i=1}^{i-1} (R_i \neq \perp) \wedge \mathbb{T}_1^{i-1}] \leq p_{\max} + \text{negl}(\lambda)$$

Proof. We proceed via a standard reduction. Assume for contradiction that there exists a PPT adversary $\mathcal{J}$ for which this inequality does not hold for some round i . That is, for some history $T_{i-1} = \wedge_{j < i} S_j \neq \perp \wedge T_{i-1}$, $\mathcal{J}$'s success probability $p_i = \Pr[S_i = 1 | T_{i-1}]$ is greater than $p_{\max} + \text{negl}(\lambda)$. We construct an adversary B that uses $\mathcal{J}$ to win IV game with probability p_i , contradicting the definition of $p_{\max}$.

B receives the parameters of $IV^\dagger$ game targeting an honest voter oracle V . B internally runs $\mathcal{J}$ and perfectly *simulates* the history T_{i-1} by playing the role of the honest voters for rounds $\{1, \dots, i-1\}$ (i.e., creates a fake history for all previous voters). At round i , B connects $\mathcal{J}$ to the voter oracle V in the $IV^\dagger$ game. From $\mathcal{J}$'s perspective, this interaction is indistinguishable from a real round i of S-I-IV, as the sources of randomness (the voter's choice for n_c from the distribution D_C) are fresh and independent of past transcripts. B then forwards $\mathcal{J}$'s output to the challenger.

B 's advantage in its $IV^\dagger$ game is exactly p_i . Our assumption $p_i > p_{\max} + \text{negl}(\lambda)$ thus implies B can break the underlying cryptography underpinning the $IV^\dagger$ game, a contradiction. Therefore, the lemma holds. $\square$

Lemma 5. *For any PPT adversary $\mathcal{J}$, $\Pr[\text{Win}]$ is bounded by $(p_{\max})^{\lambda_v} + \text{negl}(\lambda)$*

Proof. For the Win event to occur, the adversary must achieve λ_v wins before any $\perp$ outcome occurs, at which point the game terminates. Let's analyze the adversary's probability of success. The adversary's strategy involves choosing which voters to attack. Let the sequence of attacked voters be $V_{i_1}, V_{i_2}, \dots$. To win, the the adversary must succeed in the first λ_v of these attacks.

Let E be the event that the adversary succeeds in its first λ_v attempted attacks. The Win event is a sub-event of E , so $\Pr[\text{Win}] \leq \Pr[E]$. We can express $\Pr[E]$ using the chain rule of probability:

$$\begin{aligned} \Pr[E] &= \Pr[R_{i_1} = 1] \cdot \Pr[R_{i_2} = 1 | R_{i_1} = 1] \cdot \dots \\ &\quad \cdot \Pr[R_{i_{\lambda_v}} = 1 | \wedge_{k=1}^{\lambda_v-1} R_{i_k} = 1] \\ &= \prod_{k=1}^{\lambda_v} \Pr[R_{i_k} = 1 | \wedge_{l=1}^{k-1} R_{i_l} = 1] \end{aligned}$$

For each term in this product, the conditioning event $(\wedge_{l=1}^{k-1} R_{i_l} = 1)$ implies no prior detections occurred. By lemma 4, each term is bounded by $p_{\max} + \text{negl}$.

<i>Notation</i>	<i>Description</i>
$\mathbb{R}$	Registrar (combines kiosks, envelope printers and officials)
$\mathbb{L}_V, X, P, \text{st}$	Voting (sub-)ledger, Tally & Tally Proofs, (Adversarial) state
D^c, D^v	Probability distribution of fake credentials and votes
β, n_j^c, M	Target Vote, Target number of total credentials, Voting options
$\mathcal{C}, \mathbf{V}_C, j, \beta$	Coercer, $\mathcal{C}$ -controlled voters, $\mathcal{C}$ -target voter, $\mathcal{C}$ -intended ballot
$n_V, n_{\mathcal{C}}, n_M$	Number of voters, controlled voters and voting options

 Table 6.2: **Proof Notations.**

$$\begin{aligned}
 \Pr[\text{Win}] &\leq \Pr[\text{E}] \leq \prod_{k=1}^{\lambda_v} (p_{\max} + \text{negl}) \\
 &= (p_{\max} + \text{negl})^{\lambda_v} \\
 &\leq (p_{\max})^{\lambda_v} + \text{negl}
 \end{aligned}$$

□

We now conclude the proof. Lemma 4 establishes a bound on the adversary's per-round success probability, showing that past interactions do not increase its chances beyond $p_{\max}$. Lemma 5 then leverages this per-round bound to analyze the probability of the global Win event. By substituting the final result of lemma 5, we have:

$$\text{Adv}_{\Pi_{\text{TRIP}}, \mathcal{G}}^{\lambda_v} = \Pr[\text{Win}] \leq (p_{\max})^{\lambda_v} + \text{negl}(\lambda)$$

This completes the proof.

□

This result demonstrates that with TRIP's non-repudiable proofs of malicious behavior, the security against scalable attacks on individual verifiability scales exponentially with the number of votes the adversary must alter, as shown in Figure 6.12b.

6.3.2 Coercion-Resistance Proof

We prove that VoteGral is coercion-resistant by showing that the difference between $\mathcal{C}$'s winning probability in a real game – representing the adversary's interactions with our system – and in an ideal game – representing the desired level of coercion-resistance – is negligible. In both games, $\mathcal{C}$'s goal is to determine whether the targeted voter evaded coercion and cast a real vote. Like the total number of votes cast in an election, we treat the total number of credentials created as public information: $\mathcal{C}$ could trivially win if it knew exactly how many (fake) credentials *all other* voters created. The adversary's uncertainty about the target

Game C-Resist-TRIP-Ideal ^{$\mathcal{C}', b$} ($\lambda, \mathbb{V}, \mathbb{R}, \mathbb{A}, M, n_{\mathcal{C}}$)

```

1:  $\mathbb{V}_{\mathcal{C}}, \{n_c^i\}_{1 \leq i \leq |\mathbb{V}_{\mathcal{C}}|}, \text{st}_{\mathcal{C}'} \leftarrow \mathcal{C}'(\mathbb{V}, \text{"Choose controlled voter set"})$ 
2:  $\mathbb{L}^{JCJ}, \mathbb{L}_E, n_V, n_M, n_{\mathcal{C}}' \leftarrow \emptyset, \emptyset, |\mathbb{V}|, |M|, |\mathbb{V}_{\mathcal{C}}|$ 
3:  $(j, \beta, \text{st}_{\mathcal{C}'}) \leftarrow \mathcal{C}'(\text{st}_{\mathcal{C}'}, \text{"Pick target voter and their parameters"})$ 
4: if  $n_{\mathcal{C}}' \neq n_{\mathcal{C}}$  or  $j \notin \{1, 2, \dots, n_V\} \setminus \mathbb{V}_C$  then abort
5: for  $i = 1$  to  $n_V$  do % All voters — real credential
6:    $c_{\text{sk},-}^i, c_{\text{pc},-}^i, \mathbb{L}_R^{JCJ} \leftarrow \text{RealCred}(\mathbb{L}_R^{JCJ}, \mathbb{R}_{\text{sk}}, \mathbb{V}_{id}^i, \lambda)$ 
7:   if  $i \in \mathbb{V} \setminus (\mathbb{V}_C \cup \{j\})$  do % Honest voters — fake credentials
8:      $\tilde{c}_{\text{sk},-}^i \leftarrow \text{FakeCreds}(\mathbb{R}_{\text{sk}}, \mathbb{V}_{id}^i, c_{\text{pc}}^i, D_{1,[0,\text{inf}]}^c, \lambda)$ 
9:      $\_, \mathbb{L}_E \leftarrow \{\text{Activate}(\mathbb{L}_E, \tilde{c}_{\text{sk}}^i)\}_{1 \leq i \leq |\tilde{\mathbb{C}}|}$ 
10:  if  $b = 0$  then % Target voter evades — casts real vote
11:     $\tilde{c}_{\text{sk},-}^j \leftarrow \text{FakeCreds}(\mathbb{R}_{\text{sk}}, \mathbb{V}_{id}^j, c_{\text{pc}}^j, 1, \lambda)$ 
12:     $\_, \mathbb{L}_E \leftarrow \{\text{Activate}(\mathbb{L}_E, c_{\text{sk}}^j), \text{Activate}(\mathbb{L}_E, \tilde{c}_{\text{sk}}^j)\}$ 
13:     $\mathbb{L}_V^{JCJ} \leftarrow \text{Vote}(c_{\text{sk}}^j, \mathbb{A}_{\text{pk}}, M, \beta, \lambda)$ 
14:   $c_{\mathcal{C}'} \leftarrow c_{\text{sk}}^j$  % Target voter always releases their real credential
15:   $\mathbb{L}_V^{JCJ} \leftarrow \{\text{Vote}(c_{\text{sk}}^i, \mathbb{A}_{\text{pk}}, M, D_{n_V - n_C, n_M}^v, \lambda)\}_{i \in \mathbb{V} \setminus \{\mathbb{V}_{\mathcal{C}} \cup j\}}$ 
16:   $\mathbb{L}_V^{JCJ} \leftarrow \mathcal{C}'(\text{st}_{\mathcal{C}'}, \mathbb{L}_R^{JCJ}, c_{\mathcal{C}'}, \{c_{\text{sk}}^i\}_{i \in \mathbb{V}_C}, \lambda, \text{"Coercer casts ballots"})$ 
17:   $(\mathbf{X}, \_) \leftarrow \text{Ideal-Tally}(\mathbb{A}_{\text{sk}}, \mathbb{L}_V^{JCJ}, M, \mathbb{L}_R^{JCJ}, \lambda)$ 
18:   $b' \leftarrow \mathcal{C}'(\text{st}_{\mathcal{C}'}, \mathbf{X}, \mathbb{L}_E, \text{"Guess } b\text{"})$ 
19:  return  $b' = b$ 

```

Figure 6.15: **Game C-Resist-TRIP-Ideal**. The TRIP ideal game for coercion-resistance, adapted from JCJ, that considers the adversary’s probabilistic knowledge of honest voters’ fake credentials. Notations are defined in tables 6.1 and 6.2, Votegral API in fig. 6.10.

voter thus derives from the other honest voters, each of whom creates an unknown (to the adversary $\mathcal{C}$) number of fake credentials, which we model as a probability distribution² D^c . To achieve statistical uncertainty also on the voting choice, we adopt the same approach for the content of the ballot with the distribution D^v . This “anonymity among the honest voters” mimic the reasoning by which votes themselves are considered to be (statistically) protected once anonymized. We present the ideal game in fig. 6.15 to highlight Votegral’s level of coercion-resistance and defer the proof to §6.3.2.

In this ideal game, adapted from JCJ [56], the coercer chooses the target voter j and $n_C < n_V$ controlled voters who abide by the coercer’s demands. All voters obtain their real credential while honest voters also create and activate fake ones. The envelope ledger $\mathbb{L}_E$ ’s content (line 18) releases to $\mathcal{C}$ the number of total credentials created during Activate. The distribution D^c reflects the statistical uncertainty pertaining to the number of fake credentials created and

²In practice, to artificially increase this uncertainty, envelope printers can post challenges on the ledger *without* printing a corresponding envelope and gradually release these values, similar to the JCJ option of voting authorities or third parties intentionally injecting fake votes to add noise.

Game C-Resist $^{\mathcal{C},b}(\lambda, \mathbb{V}, \mathbb{R}, \mathbb{A}, M, n_{\mathcal{C}})$

```

1:  $\mathbf{V}_{\mathcal{C}}, \{n_f^i\}_{i \in \mathbf{V}_{\mathcal{C}}}, \text{st}_{\mathcal{C}} \leftarrow \mathcal{C}(\mathbb{V}, \text{"Choose controlled voter set"})$ 
2:  $\mathbb{L}, n_V, n_M, n_{\mathcal{C}}' \leftarrow \emptyset, |\mathbb{V}|, |M|, |\mathbf{V}_{\mathcal{C}}|$ 
3:  $(j, n_f^j, \beta, \text{st}_{\mathcal{C}}) \leftarrow \mathcal{C}(\text{st}_{\mathcal{C}}, \text{"Pick target voter and their parameters"})$ 
4: if  $n_{\mathcal{C}}' \neq n_C$  or  $j \notin \{1, 2, \dots, n_V\} \setminus \mathbf{V}_{\mathcal{C}}$  then abort endif
5: for  $i = 1$  to  $n_V$  do
6:    $c_{\text{sk}}^i, c_{\text{pc}}^i, P_c^i, \mathbb{L}_R \leftarrow \text{RealCred}(\mathbb{L}_R, \mathbb{R}_{\text{sk}}, V_i^{id}, \lambda)$ 
7:   if  $i = j$  and  $b = 0$  then % Target voter evades coercion
8:      $\tilde{c}_{\text{sk}}^j, \tilde{P}_c^j \leftarrow \text{FakeCreds}(\mathbb{R}_{\text{sk}}, V_{id}^j, c_{\text{pc}}^j, n_f^j + 1, \lambda)$ 
9:   elseif  $i \in (\mathbf{V}_{\mathcal{C}} \cup j)$  then %  $\mathcal{C}$ -Controlled Voters; target voter submits
10:     $\tilde{c}_{\text{sk}}^i, \tilde{P}_c^i \leftarrow \text{FakeCreds}(\mathbb{R}_{\text{sk}}, V_{id}^i, c_{\text{pc}}^i, n_f^i, \lambda)$ 
11:   else % Honest voters
12:     $\tilde{c}_{\text{sk}}^i, \tilde{P}_c^i \leftarrow \text{FakeCreds}(\mathbb{R}_{\text{sk}}, V_{id}^i, c_{\text{pc}}^i, D_{1,[0,\text{inf}]}^c, \lambda)$ 
13:   endif
14:    $\mathbf{c}^i \leftarrow (\tilde{c}_{\text{sk}}^i, \tilde{P}_c^i); \mathbf{c}^i \leftarrow (c_{\text{sk}}^i, P_c^i)$ 
15: endfor
16:  $\mathbf{c}_{\mathcal{C}} \leftarrow \mathbf{c}^i$  % Target voter releases all credentials
17: if  $b = 0$  then % Target voter evades coercion
18:    $\_, \mathbb{L}_E \leftarrow \text{Activate}(\mathbb{L}_E, c_{\text{sk}}^j)$ 
19:    $\mathbb{L}_V \leftarrow \text{Vote}(c_{\text{sk}}^j, \mathbb{A}_{\text{pk}}, M, \beta, \lambda)$ 
20:    $\mathbf{c}_{\mathcal{C}} \leftarrow (\tilde{c}_{\text{sk}}^j, \tilde{P}_c^j)$  % Target voter releases only fake credentials
21: endif
22: for  $i \in \mathbb{V} \setminus (\mathbf{V}_{\mathcal{C}} \cup \{j\})$  do % Honest voters cast vote
23:    $\_, \mathbb{L}_R \leftarrow \{\text{Activate}(\mathbb{L}_R, c_{\text{sk}}^i)\}_{1 \leq i \leq |\mathbf{c}^i|}$ 
24:    $\mathbb{L}_V \leftarrow \{\text{Vote}(c_{\text{sk}}^i, \mathbb{A}_{\text{pk}}, M, D_{n_V - n_{\mathcal{C}}, n_M}^v, \lambda)\}_{1 \leq i \leq |\mathbf{c}^i|}$ 
25: endfor
26:  $\mathbb{L} \leftarrow \mathcal{C}(\text{st}_{\mathcal{C}}, \mathbb{L}, \mathbf{c}_{\mathcal{C}}, \{\mathbf{c}^i\}_{i \in \mathbf{V}_{\mathcal{C}}}, \lambda, \text{"Activate and cast votes"})$ 
27:  $(\mathbf{X}, P_t) \leftarrow \text{Tally}(\mathbb{A}_{\text{sk}}, \mathbb{L}, n_C, \lambda)$ 
28:  $b' \leftarrow \mathcal{C}(\text{st}_{\mathcal{C}}, \mathbf{X}, P_t, \mathbb{L}, \text{"guess } b")$ 
29: return  $b' = b$ 
    
```

 Figure 6.16: **Game C-Resist.**

activated by honest voters, which could potentially be zero. If the challenge bit $b = 0$, the voter uses the real credential to cast a vote. The challenger then gives $\mathcal{C}$ the real credential of each controlled voter and the target voter. The honest voters and the coercer then proceed to cast ballots, where honest votes—and thus the adversarial statistical uncertainty—are drawn from the distribution D^v . The degree of coercion-resistance is thus bounded by the uncertainty the adversary faces about honest voters' behaviors. For tallying, we use *Ideal-Tally*, an algorithm adopted from JCJ [56], which tallies all votes from honest and controlled voters, but only tally the adversary's vote using the voter's real credential if $b = 1$. Given the tally along with the previously given credentials, the adversary guesses the bit b , i.e., whether the victim voter has cast a ballot.

Theorem 3 (Coercion-resistance, informal). *The Vote_{gral} is coercion-resistant under the decisional-Diffie-Hellman assumption in the random oracle model.*

Definition 1 (Coercion-resistance). A scheme is coercion-resistant if for all PPT adversaries $\mathcal{C}$, all security parameters $\lambda \in \mathbb{N}$, and all parameters $\mathbb{V}, \mathbb{R}, \mathbb{A}, M, n_{\mathcal{C}}$, the following holds:

$$\begin{aligned} \text{Adv}_{\mathcal{C}, \text{C-Resist}}^{\text{coer}}(\lambda, \cdot) = \\ | \Pr_{\mathcal{C}, b} [\text{C-Resist}^{\mathcal{C}, b}(\cdot) = 1] - \Pr_{\mathcal{C}', b} [\text{C-Resist-Ideal}^{\mathcal{C}', b}(\cdot) = 1] | \\ \leq \text{negl}(\lambda), \end{aligned}$$

where the probability is computed over all the random coins used by the algorithms in the scheme.

We introduce the following three major changes from JCJ games to model VoteGral's behavior.

Change #1: Voter Registration Algorithms. To model information beyond the credential (e.g., proofs of correctness), we replace JCJ algorithms `register` and `fakekey` with `RealCred` and `FakeCreds`, respectively. We also employ `Activate` to model additional registration data on the ledger (e.g., envelope challenges). In our C-Resist games, `Activate` consistently returns *out* as $\top$ since the registrar $\mathbb{R}$ is trusted.

Change #2: Fake Credentials Issuance. In contrast to the JCJ game where $\mathcal{C}$ has no influence over the voter before the voting phase, VoteGral allows $\mathcal{C}$ to interact with voters before registration. Specifically, $\mathcal{C}$ can demand voters to generate $n_f \in \mathbb{N}$ fake credentials during registration. We use the probability distribution D^c to model the uncertainty around fake credentials created and activated by honest voters.

The coercion adversary $\mathcal{C}$ has no control on the probability distribution D^c and we enforce this in the system through the parameter λ_E . This security parameter represents the minimum number of envelopes that each booth requires to ensure that coerced voters cannot accurately count all the envelopes that a booth contains. This system property—that is, a minimum number of envelopes per booth—ensures in practice that the distribution D^c is out of the coercion adversary's control, as we assume for the rest of the proof.

Change #3: Ledger Entries. While JCJ uses a trusted registration algorithm to generate a voting roster with each voter's public credential c_{pc} , VoteGral incorporates a more complex protocol to achieve individual verifiability. Specifically, during `Activate`, the envelope challenge is disclosed on the envelope ledger, permitting the adversary to discern the total number of fake credentials created. Additionally, the registration ledger includes digital signatures from both the kiosk and officials responsible for issuing the voter's credential. Despite these additions, we show that $\mathcal{C}$'s winning probability is negligibly affected, outside of the added probability distribution of fake credentials represented in the ideal game. Similar to JCJ, the winning probability of the ideal game is $\gg 0$ as coercion-resistance is bounded by the adversary's uncertainty over the behavior of honest voters.

We present our formal definition for coercion-resistance under definition 1, and use the ideal

(C-Resist-Ideal) and real (C-Resist) games in figures 6.15 and 6.16, respectively. The coercer wins if they can correctly guess the bit b , representing whether or not the targeted voter gives in to coercion. While C-Resist represents the coercive adversary $\mathcal{C}$, to prove security we must compare $\mathcal{C}$ with another adversary $\mathcal{C}'$ who plays C-Resist-Ideal, which embodies the security we want to achieve against coercion. We show that the difference between the real and ideal games is negligible.

Proof. We use three hybrid games to transition from the real to the ideal game, with each game involving a protocol change.

1. **Eliminate Voting Ledger View:** Eliminate $\mathcal{C}$'s access to the Voteal voting ledger $\mathbb{L}_V$, as $\mathcal{C}$ is incapable of differentiating between a ledger filled with honest voter ballots and a randomly generated set of ballots, assuming the decisional-Diffie-Hellman assumption holds.
2. **Number of Fake Credentials:** $\mathcal{C}$'s ability to demand voters to create a specific number of fake credentials is equal to that of an adversary $\mathcal{C}'$ who *cannot* demand voters to create a specific number of fake credentials, given the distribution of the honest voters' fake credentials.
3. **Eliminate Registration Ledger View:** Eliminate $\mathcal{C}$'s access to Voteal roster $\mathbb{L}_R$ by introducing a new ledger $\mathbb{L}_R^{JCJ}$ where we can decouple V_i^{id} and $(c_{pc}^i, K, \sigma_k, R, \sigma_r)$ via semantic security: $\mathbb{L}_R: (V_i^{id}, c_{pc}^i, K, \sigma_k, R, \sigma_r)$ & $\mathbb{L}_R^{JCJ}: (c_{pc}^i)$

Hybrid 1. We replace Tally (fig. 6.16, l. 27) with Ideal-Tally (fig. 6.15, l. 17) by proving that $\mathcal{C}$ has no longer access to $\mathbb{L}_V$. We use a simulation-based approach to show that if an adversary with access to $\mathbb{L}_V$ has a non-negligible advantage over an adversary who does not have access to $\mathbb{L}_V$, then the decisional-Diffie-Hellman assumption is broken. The simulator gets as input a tuple of group elements that are either a Diffie-Hellman tuple or uniformly random and must output a guess. It interacts with $\mathcal{C}$, simulates the honest parties of the protocol, and makes a guess based on its output.

The simulator then proceeds as follows:

1. **Setup:** Choose two group elements x uniformly at random and broadcast the public key (g, g^x) . Broadcast a list of candidate identifiers $\{M_i\}_{i=1}^{n_M}$ where each identifier is a random group element.
2. **Coin Flip:** Flip the coin b .
3. **Adversarial Corruption:** The adversary chooses the set V_C of controlled voters and a target voter j . For each controlled voter and the target voter, the adversary chooses the number of fake credentials n_f to create. For the target voter, $\mathcal{C}$ sets the target vote β . If the number of controlled voters is not equal to $n_{\mathcal{C}}$, or j is not an appropriate index then the simulator aborts.
4. **Registration:** For each voter i , the simulator runs the Voteal registration process while acting as the registrar. The simulator first issues each voter their real credential c_{sk}^i along

with their public credential c_{pc}^i . The simulator then continues with generating fake credentials for each group of voters. For each controlled voter i in V_C , the simulator issues n_f^i fake credentials as specified by the adversary. For the target voter, if $b = 0$, the simulator generates $n_f^i + 1$ fake credentials for the target voter; otherwise, it generates n_f^i fake credentials. For each uncontrolled honest voter, the simulator creates fake credentials, for which the amount is sampled from a probability distribution that models the adversary's knowledge of the number of fake credentials that these voters intend to create. Finally, the simulator carries out check-out for all voters.

5. **Credential Release:** The simulator gives $\mathcal{C}$ the real and fake credentials of voters in V_C . If $b = 0$, the simulator gives $\mathcal{C}$ the target voter's $n_f^i + 1$ fake credentials; otherwise, the simulator gives $\mathcal{C}$ the voter's real credential and n_f^i fake credentials.
6. **Honest Ballot Casting:** For each honest voter i , the simulator samples a random vote $\beta_i \leftarrow \$ D_{n_V - n_C, n_M}$ and posts a ballot for this vote on the voting ledger $\mathbb{L}_V$. The simulator forms the ballot using the input (g, h) as follows. The simulator generates two random group elements r_i, k_i . Next, the simulator computes the encryption of the public credential c_{pc}^i as $E_1 = (g^{r_i}, h^{r_i} c_{pc}^i)$, and the encryption of the vote as $E_2 = (g^{k_i}, h^{k_i} \beta_i)$. This way, if the input is Diffie-Hellman, the result is a valid encryption, whereas if the input is random, the ballot is random and contains no information about the vote or the credential used to cast it. The simulator then simulates the required NIZK proofs and signatures using standard techniques.
7. **Adversarial Ballot Posting:** The adversary now posts a set of ballots onto $\mathbb{L}$.
8. **Decryption of Ballots:** The simulator can now check the NIZK proofs and discard the ballots with incorrect proofs. Then, since the simulator plays the role of the honest talliers, it can decrypt the ballots to prepare for the tallying process.
9. **Tallying Simulation:** This is carried out as in JCJ's simulator [56]. Namely, the simulator eliminates duplicates, mixes, removes fake votes and finally decrypts the remaining real votes:
 - *Duplicate elimination:* The simulator removes duplicates.
 - *Mixing:* To simulate the MixNets from the real tally protocol, the simulator outputs an equal-length list of random ciphertexts.
 - *Credential Validity:* The simulator now checks, for each ballot, whether it was cast with a valid credential. This check is possible since the simulator can decrypt all the entries in $\mathbb{L}_R$ and $\mathbb{L}_V$.
 - *Output final count:* The simulator can then use the decrypted values to compute the final tally and output it.
10. **Output Guess:** The simulator uses $\mathcal{C}$'s output to make its guess about whether the input was a DDH triplet or a random triplet.

If we can show that $\mathcal{C}$ has a non-negligible advantage in the real game over hybrid 1, then this implies that the simulator can break the DDH assumption. The key to this argument lies in how the simulator constructs the ballots in Step 6. When the input is a DDH tuple, where $X = g^a$, $Y = g^b$, and $Z = g^{ab}$ for unknown a, b , the public key given to $\mathcal{C}$ is $h = g^a$. The

ciphertext E_1 is $(g^b, g^{ab} \cdot c_{pc}^i)$. This is a mathematically valid ElGamal encryption of c_{pc}^i under the public key $h = g^a$, using b as the randomness. The same holds for E_2 . Thus, when the input is DDH, the view of $\mathcal{C}$ corresponds to the actual game, and the adversary receives the contents of $\mathbb{L}_V$. However, if the input tuple is a random one, where $Z = g^c$ for a random c , then the ciphertext E_1 is $(g^b, g^c \cdot c_{pc}^i)$. The second component, $g^c \cdot c_{pc}^i$, is a uniformly random group element from the adversary's perspective, as c is unknown and independent of a and b . In this scenario, the ciphertext perfectly conceals the credential, making the adversary's view equivalent to denying them access to the meaningful encrypted content of the ledger. Hence, if the adversary $\mathcal{C}$ holds a significant advantage in distinguishing the real game from the hybrid, then the probability that the simulator correctly guesses whether its input was DDH or random will also be significant, leading to a contradiction of the DDH assumption.

Hybrid 2. In this Hybrid, we demonstrate that the adversary's ability to demand a specific number of fake credentials from voters is lost, as compared to hybrid 1. In C-Resist (fig. 6.16, l. 16 and 20), the voter gives their credentials to the adversary while in C-Resist-Ideal, the adversary always gets the voter's single real credential (fig. 6.15, l. 14).

First, we show that real and fake credentials are indistinguishable: both real and fake credentials contain a ZKP transcript, although the ZKP transcript for the fake credentials are simulated. The zero-knowledge property of the proof system implies indistinguishability.

Next, from hybrid 1, $\mathcal{C}$ does not get access to $\mathbb{L}_V$, thus $\mathcal{C}$ cannot use any real or fake credentials to cast valid ballots, or see ballots cast with these. As a result, since real and fake credentials are indistinguishable, always giving the adversary the real credential gives them no advantage since the credentials cannot influence the tally outcome with it. Now the value of b only determines whether or not the target voter casts a ballot.

For the same reason, $\mathcal{C}$ can only use the n_f fake credentials for determining whether the target voter cast a ballot or not and to influence the number of envelope challenges on $\mathbb{L}_E$.

Yet, the signing key pair $(\tilde{c}_{sk}, \tilde{c}_{pk})$ corresponding to each fake credential is sampled independently from the real pair (c_{sk}, c_{pk}) , and these cannot be used by the adversary or the target voter to cast a valid vote. Regarding the number of challenges on $\mathbb{L}_E$, the only difference in the case where the target voter resists coercion is that they create an additional fake credential. As a result, the number of challenges will only differ by one. To detect this, the adversary needs to distinguish between the following distributions:

- Number of envelope challenges when target complies: $n_C + n_T + n_H + n_f + D_{n_H}^{fake}$
- Number of envelope challenges when target resists: $n_C + n_T + n_H + n_f + D_{n_H}^{fake} + 1$

Since everything but $D_{n_H}^{fake}$ is known to the adversary, this is equivalent to just distinguishing between $D_{n_H}^{fake}$ and $D_{n_H}^{fake} + 1$, so they get no advantage from requesting a certain number of fake credentials. Therefore, the adversary does not gain an advantage from specifying the number of fake credentials, since these fake credentials will not help them identify whether the voter gave them a real credential.

Chapter 6. TRIP: Trust-Limited Coercion-Resistant In-Person Voter-Registration

Hybrid 3. In this hybrid, we replace the Voteal roster $\mathbb{L}_R$ initialized on the first line of Figure 6.16 with the JCJ Roster $\mathbb{L}'_R$ from the Ideal Game in Figure 6.15.

To prove that the advantage of the adversary is negligible between these hybrids, we show that given a JCJ roster, a simulator can output a Voteal roster that is indistinguishable from a real one. This is possible due to the semantic security of ElGamal.

We describe the simulator: **Input:** JCJ roster $\mathbb{L}'_R$, List of voter IDs V_{id}

1. Create a kiosk key pair (k, K) and a registrar key pair (r, R) . If there are multiple kiosks and registrars, these keys contain all the individual keys.
2. Initialize $\mathbb{L}'_R$ to contain each V_{id} in a different entry, along with a random timestamp d .
3. Apply a random permutation to the JCJ roster.
4. Append one entry V_e of the JCJ roster to each entry of $\mathbb{L}_R$.
5. For each entry, add the necessary signatures. First, use k to simulate the kiosk signature $\sigma_2 = \text{Sig.Sig}_k(V_{id}^i || d || V_e)$ and append K, σ_{k_2} to the entry. Then, use r to simulate the registrar's check-out signature $\sigma_r = \text{Sig.Sig}_r(V_{id} || d || V_e || \sigma_{k_2})$ and append R, σ_r to the entry.
6. Output $(K, R, \mathbb{L}'_R)$. Recall that we consider a single logical entity for all the registration actors.

As stated earlier, $\mathbb{L}'_R$ is indistinguishable from a real Voteal roster for the same list of voters due to the semantic security of ElGamal. If we start with a real Voteal roster, we could create intermediate rosters by swapping two encryptions at a time and updating the digital signatures until we get a random permutation. Each of these swaps will yield indistinguishable rosters by the semantic security of ElGamal. At the end, the distribution will be the same as that of our simulator.

Recall that the view of $\mathcal{C}$ is the list of valid verification keys for the kiosks and registrars, along with the Voteal roster. Since we have shown that the additional elements contained in the Voteal roster but not in the JCJ roster can be simulated, this means that the advantage of $\mathcal{A}$ is negligible.

With this, we have reached the Ideal JCJ game and have thus shown that the advantage of $\mathcal{A}$ in the C-Resist game is negligible over the advantage in C-Resist-Ideal.

□

6.3.3 Privacy

The privacy adversary seeks to uncover a voter's vote by identifying and decrypting the voter's real ballot. See our threat model for the capabilities of such an adversary (§5.2.2). Unlike the coercion adversary, this adversary cannot directly interact with or influence voters; it must achieve its objective solely via electronic means. A voter's ballot is electronically accessible in three locations: on the voter's device, on the voting sub-ledger $\mathbb{L}_V$, and in the final tally. In

what follows, we informally argue that the privacy adversary is unable to decrypt the voter's real ballot in these three locations.

Voter's device. Per our threat model (§5.2.2), this adversary cannot compromise the device containing the voter's real credential. Compromising devices that hold fake credentials offers no advantage because ballots cast from these devices are not counted: the adversary cannot deduct the voter's real vote.

Voting sub-ledger $\mathbb{L}_V$. The voter's real ballot on the voting sub-ledger is encrypted, i.e., the has only access to the ciphertext of the real ballot. The adversary can identify the voter's real ballot on the ledger because it can compromise the registrar, including the kiosk that issued the target voter's credentials. Decrypting this ballot, however, would require the adversary to obtain the election authority's private key, which is impossible since the adversary can only compromise all but one authority member. Secret keys from compromised members provide no information into the collective key of the member that remains uncompromised, thus preventing reconstruction of the election authority's private key.

Final tally. In the final tally, ballots are revealed in decrypted form but through a mixing process that unlink ballots from voters. While the adversary can acquire the mixing permutation from all the compromised authority members, it cannot obtain it from the one member it cannot compromise. Each authority member generates and keeps their permutation secret, leaving no information for the adversary to deduce the permutation of the uncompromised member.

The privacy adversary is therefore unable to achieve its goal of revealing any voter's vote, thereby preserving privacy.

Excluded attack. As the Swiss Post e-voting system [117, Section 18.2], we exclude trivial attacks where all voters vote for the same option, which would indicate to the adversary what the targeted voter voted. Also, we do not consider the adversarial strategy to delete from the voting sub-ledger and final tally all but one vote, to learn the targeted ballot from the tally's result. We assume this attack to be detectable and thus inapplicable by the privacy adversary.

In chapter 7, we provide a proof sketch that VoteGral as a whole satisfies privacy.

6.4 Implementation

Our full VoteGral prototype consists of 9,182 lines of code as counted by CLOC [30].

TRIP is 2,633 lines of Go [78], and uses Kyber [65] for cryptographic operations, gozxing [47] for QR code processing, gofpdf [57] and pdfcpu [101] for printing and reading QR codes. We use Schnorr signatures and SHA-256 for the signing and hashing algorithms, respectively. For both signatures and encryption, we use the Ed25519 elliptic curve.

6.5 Evaluation

To evaluate whether TRIP represents a practical approach to registration for coercion-resistant online voting, we examine the following key questions:

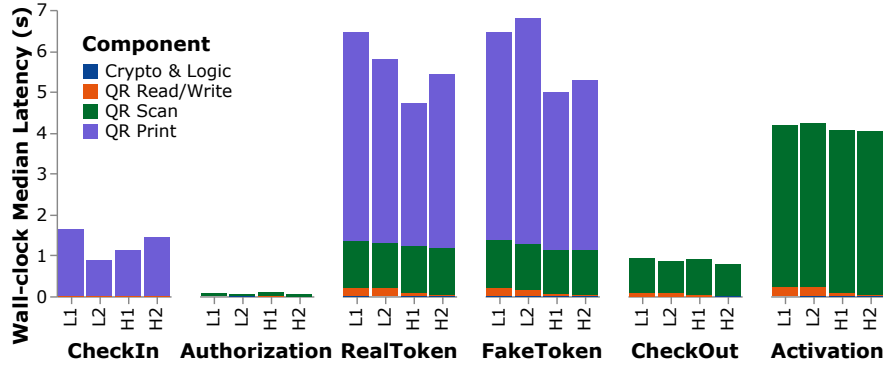
- Q1: Is TRIP fast enough in practice to accommodate voters who typically spend only a few minutes in a booth?
- Q2: How does TRIP’s registration-phase performance compare to that of state-of-the-art e-voting systems?

To compare the computational cost of TRIP and answer Q2, we compare TRIP against three e-voting protocols: (1) Swiss Post [117]—a verifiable secret ballot system used in Switzerland—(2) VoteAgain [73]—a coercion-resistant voting system based on deniable re-voting—and (3) Civitas [26]—a verifiable and coercion-resistant voting system. We include Swiss Post’s voting system as it represents the state-of-the-art in verifiable secret-ballot systems, despite lacking coercion resistance. We chose VoteAgain for its efficient tallying process, despite lacking end-to-end verifiability. We compare against Civitas because it is a well-known coercion-resistant, end-to-end verifiable system based on JCJ [82, 83, 81, 37, 61, 62]. We omit coercion-resistant systems that rely on cryptographic primitives still deemed impractical [104], or that were unfortunately not runnable despite our efforts, including Estonia’s system based on deniable re-voting [34].

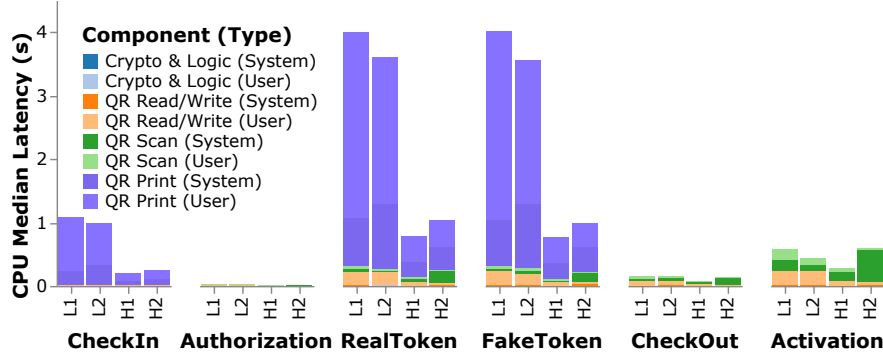
6.5.1 Experimental Setup and Benchmarks

For §6.5.2, we run TRIP across four distinct hardware setups: (L1) a Point-of-Sale Kiosk (Quad-core Cortex-A17, 2GB RAM, Linaro), (L2) a Raspberry Pi 4 (Quad core Cortex-A72, 4GB RAM, Raspberry Pi), (H1) a Parallels VM on Macbook Pro (M1 Max 4 cores, 8GB RAM, Ubuntu 22.04.2), and (H2) a Beelink GTR7 (AMD Ryzen 7840HS, 32GB RAM, Ubuntu 22.04.4). (L) devices represent resource-constrained setups.

The Macbook Pro serves as our baseline, while the Raspberry Pi and Beelink are compact computational units suitable for integration into devices with QR printing and scanning capabilities. The Point-of-Sale Kiosk, used in our user study part III, include both computational elements and QR peripherals (receipt printer and barcode/QR scanner). However, due to issues with the kiosk’s built-in receipt printer, particularly with tearing receipts, we replaced it with a dedicated EPSON TM-T20III printer featuring an automatic cutter for easier receipt handling. To ensure consistency across all devices, we equipped each configuration with this printer and a Bluetooth-connected barcode/QR scanner, as wired connections were not compatible across all four configurations.



(a) Wall-clock Time incurred by each sub task in TRIP



(b) CPU Time incurred by each sub task of TRIP

Figure 6.17: **Execution latency of TRIP:** (L) represents resource-constrained devices while (H) represents resource-abundant devices.

6.5.2 Voter-Observable Registration Latencies

This experiment addresses Q1: whether TRIP’s performance is adequate for realistic use in environments where voters typically spend a few minutes each in the privacy booth. This experiment also explores whether TRIP can operate on low-cost, resource-constrained, or battery-powered devices.

We scripted TRIP to issue one real and one fake credential without human involvement, measuring all user-observable wall-clock delays across all registration phases, including cryptographic operations, QR code scanning (“QR Scan”), encoding/decoding (“QR Read/Write”), and receipt printing (“QR Print”). Since QR printing and scanning are mechanical components, they introduce unique challenges in latency measurement. For QR printing, we adapted CUPS [87] to capture CPU and wall-clock latency from job initiation to completion. For QR scanning, we measured the time when TRIP starts to receive data to when it captures the terminating character. We ran TRIP for 10 consecutive registrations on each hardware, recording wall-clock and CPU latencies for each registration component and phase (figs. 6.17a and 6.17b).

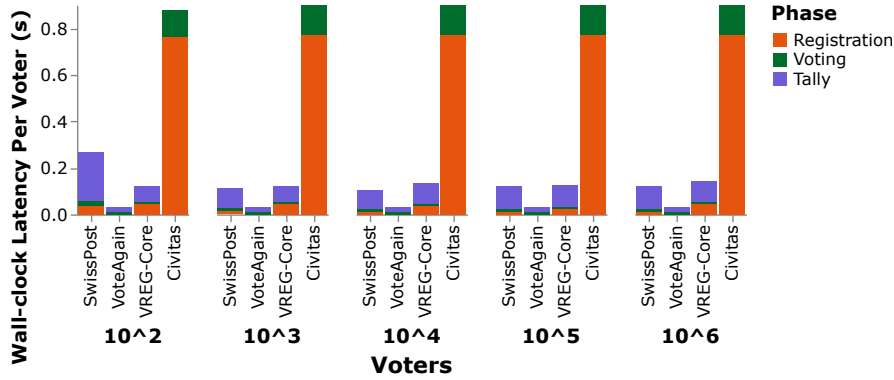


Figure 6.18: **Latency per voter for registration, voting, and tallying phases.** We exclude the Civitas tally phase to avoid skewing the graph's scale.

First, the maximum wall-clock latency perceived by voters during registration is under 19.7 seconds on all deployments, with the longest wall-clock latency for any specific registration process being under 6.5 seconds. In in-person voting, for comparison, the duration that a voter stays inside a booth varies greatly, depending on the complexity of the ballot. A voting time estimator tool [120] states that marking a single-race ballot—where a voter chooses one candidate from many—takes between 19 and 44 seconds (90% confidence interval). Assuming voters instantly execute the mechanical tasks, QR printing and scanning, TRIP meets the lower end of this time range, even on resource-constrained devices.

Second, QR-related tasks significantly contribute to the total wall-clock latency: QR Printing and scanning account for at least 69.5% of the time, with a median overhead of 97.5%. In particular, it takes about 948 milliseconds on average to scan each QR code across devices. This lengthy duration is primarily due to QR data (13-356 bytes) transfer times from the scanner to the computational devices. The check-in ticket's barcode, which contains only 13 bytes of data, further supports this point by taking on average 60 milliseconds across devices. We expect that a more suitable barcode/QR scanner would be in use, such as the kiosk's embedded scanner, which scans even 300-byte QR codes almost instantly. Therefore, using a more efficient scanner could reduce the overall registration latency by about a third, on average, as each registration run currently takes 7 seconds, on average, to scan QR codes.

Third, we find that resource-constrained devices (L1 and L2) perform up to 19.8% slower than higher-end devices (H1 and H2). Specifically, L1 is the slowest with a wall-clock latency of 19.7 seconds, while H1 is the fastest at 15.8 seconds. This difference can be attributed to the CPU time distribution shown in fig. 6.17b. On average, the CPU latency for resource-constrained devices is 260% higher than that for higher-end devices. In particular, QR printing on these devices takes 380% higher than that on higher-end devices. Despite these significant increases in CPU latency, the overall wall-clock latency only rises by an average of 16.5%.

In answer to Q1, we find that TRIP appears suitable for environments on time scales compara-

ble to those applicable to in-person voting, even on resource-constrained devices.

6.5.3 Registration Performance Across Systems

We now examine Q2: How does registration performance in TRIP compare to the registration-time costs of other e-voting systems? This experiment focuses on computational costs, excluding the I/O-related latencies such as QR code printing and scanning considered earlier for Q1. We focus on computation cost for two reasons: TRIP's I/O-bound phases lack direct analogs in existing systems, making “apples-to-apples” comparison impractical; and second, computation cost is a key metric in e-voting research, as cryptographic operations, such as shuffling, decryption, and zero-knowledge proof generation and verification, typically dominate and can be prohibitive for deployment. We compare the cryptographic performance between TRIP, Swiss Post, VoteAgain and Civitas while varying the number of voters.

For TRIP, which includes voter-interaction features like QR codes, we use a configuration termed “TRIP-Core” that omits all QR-related tasks to isolate the cryptographic operations. Swiss Post provided us their end-to-end election simulator, enabling us to run their cryptographic functions. VoteAgain and Civitas did not require any changes from the code provided by their respective papers. Figure 6.18 presents our per-voter results, which also include data for voting and tallying phases; these aspects are discussed further in chapter 7.

In the experimental configuration for 1 million voters, the registration latency per voter for TRIP, Swiss Post, VoteAgain, and Civitas is 45 ms, 13 ms, 0.1 ms, and 771 ms, respectively (fig. 6.18). VoteAgain significantly outperforms all other systems due to its lack of verifiability, reducing the process to generating a key pair for each voter. TRIP is 246% slower than Swiss Post but 94% faster than Civitas. The difference between TRIP and Swiss Post reflects the additional overhead needed to provide coercion-resistance. Nevertheless, these results suggest that TRIP lies in the same general performance range as already-deployed e-voting systems.

6.6 Discussion

6.6.1 Mitigating Credential Exposure

A voter's secret signing key (r) for casting real votes is vulnerable at two points in its lifecycle: on the paper credential it is printed on, and on the issuing kiosk.

The most likely threat is the exposure of the secret key on the paper receipt itself, for instance, to a hidden camera or a prying observer. To mitigate this, we developed the two-state receipt-and-envelope design detailed in §6.1.2. When the receipt is fully inserted for transport fig. 6.1c, the QR code containing the secret key is physically obscured. Only when the voter is in a secure location of their choosing do they slide the receipt to the “activate” position, revealing the key for the brief moment needed for scanning. This design minimizes the window of opportunity for physical compromise.

Chapter 6. TRIP: Trust-Limited Coercion-Resistant In-Person Voter-Registration

Another threat comes from the compromise of the kiosk itself. A malicious kiosk could retain a copy of the secret key r it generates for the voter, later using it to cast a fraudulent ballot. Our defense is to make such attacks detectable in VLT. When a ballot is cast, the corresponding public key (e.g., R) is cast alongside it on the voting ledger $\mathbb{L}_V$. VSDs could then monitor the ledger for any ballots cast with its stored public key. If an unexpected ballot appears, the VSD could alert the voter, who could then contest the vote with the election authority or re-register to invalidate the compromised credential.

A proactive alternative (not implemented in VoteGral) is to treat (R, r) as an issuance token and have VSD rotate to (R', r') at activation, posting a binding signed by r that supersedes R with R' . The pair (R, R') would then become a necessary public intermediary table to link ballots $\mathbb{L}_V$ back to the registration ledger $\mathbb{L}_R$. This shrinks the attack window to the interval between issuance and activation, at the cost of an additional linear cost shuffle on this intermediary table $O(C)$, where C is the number of registrar-issued credentials.

6.6.2 Side Channel Attacks and Defenses

While side-channel attacks are beyond this paper's scope and threat model in general, we acknowledge that they represent important and potentially realistic threats in practice.

The most privacy-sensitive stage of TRIP's registration process is of course the time the voter spends in the privacy booth to print their real and fake credentials. A variety of side-channel attacks could compromise the privacy that the booth is intended to afford. We briefly discuss a few such potential side-channel attack vectors below, together with some potential mitigations. This list is by no means comprehensive, however, and any systematic treatment of side-channel attacks we leave to future work.

In general, a coercer need only learn the precise *number of credentials* that the coercer's victim creates in the privacy booth. Given precise knowledge of this number, the coercer can simply demand that the victim hand over exactly that number of credentials after registration, and will know that one of these credentials must be the victim's real credential, even if it is not obvious which one that is. The coercer can simply vote the same way with all of the victim's credentials, knowing that whichever one is real will cast a vote that counts. Some of the discussion below therefore focuses on side-channel attacks that might give a coercer precise knowledge about the number of credentials created by a voter inside the booth.

For further reference, a broader survey on side channels may be found in [68].

Recording. Perhaps the most-realistic side-channel attack against the privacy of TRIP credential printing is the direct analog of the “ballot selfie” attack against privacy booths used for in-person voting: i.e., voters might be coerced to use recording devices in the privacy booth. We discuss this threat vector and potential mitigations more extensively in §6.6.2.

Timing. A coercion adversary $\mathcal{C}$ might be able to infer the number of credentials a voter cre-

ates by analyzing the duration spent in the booth. Kiosks might introduce a random, artificial delay during credentialing to counteract this threat, making the adversary's estimations more challenging. This delay might appear during screen changes, or a static "Please wait..." message without any underlying activity. Further, kiosks could enforce, for each voter, a minimum random number of fake credentials.

Sound. The noise generated by TRIP's credential printer might expose the of precise number of credentials a voter creates, if the coercer is near enough to hear the printer noise. To mitigate this side-channel threat, we suggest minimizing printer sounds using a low-noise printer. Designing privacy booths as separate rooms with a degree of soundproofing could provide a stronger defense against sound-based attacks, at a significantly higher construction cost of course. Additionally, introducing a noise simulation device might help mask the origin of the sound.

Electromagnetics. A coercer located nearby might use more sophisticated scanning technology to detect electromagnetic leakage from the kiosk the victim is using, or to "see through" the privacy booth's protection in other ways. Privacy booths could in principle be TEMPEST-shielded to protect against such attacks [39], but we doubt the cost of such defenses will be seen as justified by hopefully-rare coercion threats in most cases. Some sites where we envision TRIP might be deployed – such as at embassies or military bases for the use of expatriates and military personnel, for example – might already have suitable areas with stronger side-channel protections, thus reducing the incremental cost of stronger defenses.

We now proceed to detail VLT.

7 VLT: A Verifiable Coercion-Resistant Tallying Scheme with Standing Votes

This chapter presents VLT, the verifiable, coercion-resistant tallying scheme used in Votegral. VLT's goal is a practical coercion-resistant tallying scheme that compares to the state-of-the-art while also supporting standing votes.

7.1 Design Overview

7.1.1 Linear-Time Filtering

The filtering step in a coercion-resistant, end-to-end verifiable voting system with fake credentials involves (1) eliminating duplicate ballots, and (2) removing ballots cast with fake (non-roster) credentials, before decrypting the remaining ballots and computing the outcome. In JCJ [56], both tasks are realized with plaintext-equivalence tests (PETs) [55]. JCJ's duplicate ballot removal costs $O(B^2)$ PETs, and roster membership costs $O(S \cdot n_V)$ PETs, where B is the number of cast ballots, S is the number of ballots post-duplicate filtering and n_V is the number of voters. An adversary targeting a denial of service can therefore flood the ledger with admissible fake ballots, since the ledger cannot (and must not) distinguish real ballots from fake ballots without otherwise violating coercion-resistance. Furthermore, this is a superlinear amplification factor given that the adversary only needs to cast X ballots to cause the tallying step to filter on the order of $O(X^2)$.

To eliminate the amplification factor, our scheme based on Koenig et al. [61]'s approach, is to constrain admissible ballots to those whose credential was issued by the registrar. Concretely, the registrar posts an additional sub-ledger $\mathbb{L}_C$ listing every issued credential public key (real and fake). The ledger then accepts a ballot only if its credential public key c_{pk} appears in $\mathbb{L}_C$ and the ballot carries a valid signature under the corresponding secret key c_{sk} . In Koenig et al.'s scheme, talliers must participate during the voting phase to detect duplicates which is not present in our construction due to c_{pk} being a part of the ballot. Because $\mathbb{L}_C$ contains both real and fake credentials (issued from the same distribution by the kiosk), this constraint does not invoke a real vs fake ballot test.

Chapter 7. VLT: A Verifiable Coercion-Resistant Tallying Scheme with Standing Votes

Next, to reduce quadratic filtering to linear time, we use deterministic “tags” [113, 122, 61]. Deterministic tags have been found to allow coercers to perform a compliance test by submitting two ballots whose credentials are algebraically related (e.g., one being σ and another being σ^a), and then testing for this relation post-mixnet [91]. Yet, by constraining the set of eligible ballots to kiosk-issued, independently generated credentials, the coercer is no longer capable of performing this compliance test [61]. Therefore, after the voting phase ends, the authorities can collectively compute a deterministic tag on each shuffled credential on both the roster and the ballot using a single secret exponent. These tags behave like fingerprints: equal credentials results in equal tags, but the underlying credentials remain hidden. Third, anyone can perform a simple set join on the published tag lists to keep exactly those ballots whose credential matches some roster entry. The result is linear-time filtering, $O(B + n_V)$, followed by verifiable decryption of the remaining ballots to compute the election outcome.

7.1.2 Standing Votes

Some voters may face sustained supervision by a coercer that makes ordinary remote voting impractical: they cannot safely keep a real credential after registration (e.g., due to an immediate search), or later access an unsupervised device to cast their vote. Standing votes enable such voters to participate in the democratic process without carrying a real credential out of the booth. The idea is that *during registration* — the moment the election authority knows for certain the voter can act in their own free will — the voter instructs the kiosk to delegate their voting rights to a publicly known political party. At each election, that voter’s “vote” then counts towards their selected political party’s vote. The trade-off is that this voter must fully trust the kiosk, as they do not receive a verifiable receipt for the standing vote.

Registration Workflow

To enable standing votes, political parties publish a credential public key c_{pk} that they will use to cast their party’s ballot. The ledger maintains an additional sub-ledger $\mathbb{L}_P$ that binds each party identifier P to its public key c_{pk} (together with a proof of knowledge of the corresponding c_{sk} and any party-related metadata).

Inside the booth, a voter facing extreme coercion selects to cast a standing vote and chooses the political party they are most closely aligned with. Instead of generating a voter-owned real credential, the kiosk forms this voter’s registration record by encrypting the chosen party’s public key c_{pk} into the voter’s public credential tag c_{pc} . To preserve plausible compliance outside the booth, the kiosk still prints at least one fresh fake credential pair (c'_{sk}, c'_{pk}) together with the usual unsound proof, falsely attesting that c_{pc} encrypts c'_{pk} . When searched, the voter thus holds only valid-looking fake credentials. Importantly, no procedural changes are required outside the booth: at check-out, the voter still presents any one of their credentials (all are fake in this case), and the official publishes the registration record containing c_{pc} to the ledger.

We deliberately do not issue a standing vote receipt, even though it is technically feasible to produce an interactive zero-knowledge proof that c_{pc} encrypts the political party's public key c_{pk} . The reason being that voters who resort to standing votes typically cannot perform receipt checks on an unsupervised device, and introducing a specialized receipt — one where c_{sk} is not present — would introduce significant usability challenges. Assume a coercer asks a voter to cast a standing vote for political party P and the voter wants a real credential for themselves, the voter will have to create their real credential and then be *certain* to tell the kiosk to create a standing vote for political party P . And what happens if the voter accidentally creates a fake credential instead? Or what if the voter in addition to the fake standing vote receipt wants fake credentials because someone else demands it? This becomes confusing, fast; instead, a voter either leaves the booth with one real credential (and any number of fake ones), or if casting a standing vote, leaves with only fake credentials.

Tally Semantics

Standing votes are only taken into account during the tally phase. As usual, the authority verifiably mixes the registration roster and the accepted ballots, computes deterministic tags, and filters accepted ballots to keep those cast with real (roster) credentials, as described earlier. The only difference is that, on the roster side, there will now be duplicate credential tags as they correspond to the same political party's public key c_{pk} . Therefore, each occurrence of the tag on the roster contributes to one vote for the party's ballot. After the decryption step, the vote itself will likely reveal the political party that cast the ballot, but to be certain, the election authority also decrypts the credential part of the ballot. This ensures that political parties vote according to their public statements, adding auditability for those voters who delegated their vote.

Coercion-Resistance

VLT keeps the coercer's advantage to only the results present in the final tally. The materials given to a voter casting a standing vote are only fake credentials containing an IND-CPA secure encryption of the political party's public key inside c_{pc} . There is no standing vote-receipt to demand. The final tally reveals only aggregate standing-vote counts per party and does not identify which voters delegated. Thus, even if a coercer observes that a party received X standing votes, they cannot determine whether their specific target voter delegated or whether someone else did, provided the adversary cannot control the entire electorate (fig. 6.15).

A coercer might attempt to register a fringe “party” and coerce one voter to delegate, then infer compliance from observing exactly one standing vote. The coercer could also try to scale their fringe party by coercing one voter per election. To mitigate this, $\mathbb{L}_P$ consists of only certified major parties (e.g., parties meeting a public membership threshold). Further, renewal of registration records — especially if randomized — would introduce statistical uncertainty. From one election to the next, the coercer would not be able to tell whether a

missing standing vote reflects non-compliance by a newly targeted voter or non-renewal by a previously targeted voter.

Coercion-Evidence

Standing votes also serve as a form of coercion-evidence [49]. Because standing votes are intended mainly for voters under extreme coercion, the aggregate count of standing votes revealed at the end of each election signals how many voters felt too unsafe to vote directly. A rising number of standing votes could then prompt public scrutiny and policy responses aimed at reducing coercion.

A certain number of voters may take advantage of standing votes for convenience, which adds to the coercer's statistical uncertainty, but this should not be actively incentivized to meet coercion-evidence and prevent "super-delegates" [60]. The fact that voters can only change their delegation by re-registering in person likely helps disincentivize this behavior.

7.1.3 Design Goals

In summary, our high-level goal for VLT is to satisfy the following:

- **Universal verifiability:** Each transformation (e.g., mixnet, tag computation, decryption) is publicly verifiable so that anyone can check the correctness of the election outcome.
- **Privacy:** No party without compromising all of the election authority members can learn an honest voter's choice beyond what can be learned from the final tally.
- **Coercion-Resistance:** No party without compromising all of the election authority members or the registrar can learn whether a voter (both honest and malicious) complied with their demands. In particular, the system does not allow any transferable test of how a voter voted nor whether a posted ballot used a real versus a fake credential.
- **Standing Votes:** Voters who face extreme coercion can delegate their voting rights to a third party (e.g., political party) while being capable of auditing that parties' votes.
- **Linear Time Filtering:** $O(B + n_V)$ work to eliminate fake ballots — ones that do not correspond to registrar-issued real credentials — where B is the number of ballots and n_V is the number of voters.
- **Robustness against Fake-Ballot Flooding:** only registrar-issued credentials can be used to cast ballots on the ledger, preventing arbitrary data from becoming a part of the expensive tally steps.

7.2 Scheme

We now describe the registration, voting, and tally procedures. For clarity, we present a single election; extension to multiple elections is possible by including an election context in all authenticated materials (e.g., signatures and proofs).

7.2.1 Registration

Registration follows TRIP, with two additions. First, a credential ledger $\mathbb{L}_C$ that lists every kiosk-issued voter credential public key (real and fake). Second, a party registry ledger $\mathbb{L}_P$ that binds each certified political party identifier P to a public c_{pk} used to cast that party's ballot (together with a proof of knowledge of c_{sk}). Kiosks are provisioned with $\mathbb{L}_P$ in advance if they are operationally airgapped.

For each registration session, the registrar posts to $\mathbb{L}_R$ a public credential ciphertext c_{pc} , which is an ElGamal encryption of a public key:

- Standard: $c_{pc} = \text{EG.Enc}(A^{pk}, c_{pk})$ where c_{pk} is the voter's real credential public key. The registrar also posts the voter's credential keys to $\mathbb{L}_C$ (the real c_{pk} and any fake c'_{pk} the voter chose to create).
- Standing-Vote: $c_{pc} = \text{EG.Enc}(A^{pk}, c_{pk})$ where c_{pk} is the chosen party's key from $\mathbb{L}_P$. To preserve plausible compliance, the kiosk prints only fake credentials (c'_{sk}, c'_{pk}) for the voter. In this case, $\mathbb{L}_C$ is updated only with the voter's fake public keys c'_{pk} .

The timing of $\mathbb{L}_C$ publication is a deployment parameter that trades the timeliness of individual verifiability against potential leakage about registration volume. Naturally, $\mathbb{L}_C$ should not be updated as soon as $\mathbb{L}_R$ is updated as any third party can then know the number of credentials that are issued for a given voter. In contrast, $\mathbb{L}_C$ should not be updated immediately before an election giving voters no time to remediate when their real credential's public key is not present. Although if this were to occur, the voter would have non-repudiable proof that the registrar disenfranchised the voter.

When a voter renews their registration record and receives a new set of credentials, the old credentials remain on $\mathbb{L}_C$; $\mathbb{L}_C$ becomes an endless list of all credentials ever issued. The voter's previous real credential public key c_{pk} essentially now casts fake votes.

7.2.2 Voting

Voter. When a voter intends to cast their ballot, their VSD first synchronizes with the ledger $\mathbb{L}$ and verifies that its locally stored registration view matches $\mathbb{L}_R$. In particular, it checks that the encrypted credential c_{pc} on $\mathbb{L}_R$ equals the locally stored c_{pc} and that the voter-chosen credential $c_{pk} \in \mathbb{L}_C$. VSD then forms a ballot as shown in fig. 7.1 using the voter's selected option β , producing two ElGamal ciphertexts (E_{vt}, E_{cr}) , two NIZK proofs (π_{vt}, π_{cr}) , and a signature σ_{vt} under c_{pk} .

Party Ballot Each party $P \in \mathbb{L}_P$ casts exactly one ballot under its key $c_{pk} \in \mathbb{L}_P$ by running the same procedure as in fig. 7.1, replacing the membership check with $c_{pk} \in \mathbb{L}_P$.

Ledger For each credential c_{pk} , the ledger accepts at most one ballot. Formally, upon accepting a ballot $(c_{pk}, \dots)$, it sets a spent bit $\text{spent}[c_{pk}] \leftarrow 1$. Subsequent ballots with the same c_{pk} are

Vote($\mathbb{L}, c, \mathbb{A}_{pk}, \beta, M$)		
VSD ($((G, q, g), \mathbb{L}, \mathbb{A}^{pk}, V_{id}, c, M, \beta)$		
1: $c'_{pc} \leftarrow L_R[V_{id}]; c'_{pc} \stackrel{?}{=} c_{pc}$	% Latest registration match?	
2: $c_{pk} \in \mathbb{L}_C$	% Voter credential listed?	
3: $E_{vt} \leftarrow \text{EG.Enc}(\mathbb{A}^{pk}, \beta)$	% Encrypt vote option	
4: $E_{cr} \leftarrow \text{EG.Enc}(\mathbb{A}^{pk}, c_{pk})$	% Encrypt credential	
5: $\pi_{vt} \leftarrow \text{NIZK}\{\beta : E_{vt} = \text{EG.Enc}(\mathbb{A}^{pk}, \beta) \wedge \beta \in M\}$	% Well-formed vote option	
6: $\pi_{cr} \leftarrow \text{NIZK}\{c_{sk} : E_{cr} = \text{EG.Enc}(\mathbb{A}^{pk}, g^{c_{sk}}) \wedge c_{pk} = g^{c_{sk}}\}$	% Credential binding	
7: $\sigma_{vt} \leftarrow \text{Sig.Sign}(c_{sk}, E_{vt} \ E_{cr} \ \pi_{vt} \ \pi_{cr})$	% Ballot authentication	
8: $B \leftarrow (c_{pk}, E_{vt}, E_{cr}, \pi_{vt}, \pi_{cr}, \sigma_{vt})$	% Store ballot	
9: $\mathbb{L}_V \leftarrow B$	% Post ballot	

Figure 7.1: **Voting** The voter encrypts their choice and their credential, proves the encryption of their choice and their credential is well-formed, and signs and posts the ballot.

rejected. The ledger verifies that voter-cast ballots have $c_{pk} \in \mathbb{L}_C$ and party-cast ballots have $c_{pk} \in \mathbb{L}_P$. The ledger also verifies σ_{vt} is a valid signature under c_{pk} and that all NIZK proofs verify.

7.2.3 Tally

We adopt the Tally method defined in the TRIP API (fig. 6.10), focusing on the construction of the tally X and proof P .

Inputs

From the registration ledger $\mathbb{L}_R$, the authority collects the vector of public credentials:

$$\mathbf{C}^r = (c_{pc,1}, \dots, c_{pc,n_V})$$

where $c_{pc,i}$ is an ElGamal encryption of a public key $c_{pk,i}$ (either a voter's real credential or a party credential for standing votes).

From the voting ledger $\mathbb{L}_V$, the authority collects the accepted ballots as pairs:

$$(\mathbf{C}^v, \mathbf{V}) = ((c_1, \dots, c_b), (v_1, \dots, v_b))$$

where c_i is the credential ciphertext E_{cr} , v_i is the vote ciphertext E_{vt} .

Verifiable Mixnet

The election authority then collectively runs a verifiable mixnet to unlink inputs from outputs [80, 12]:

$$\mathbf{C}_s^r = \text{Mix}(\mathbf{C}^r), \quad (\mathbf{C}_s^v, \mathbf{V}_s) = \text{MixK}(\mathbf{C}^v, \mathbf{V})$$

where Mix shuffles a vector of ElGamal ciphertexts, and MixK shuffles tuples of ElGamal ciphertexts that preserve within-tuple association (c_i, v_i) .

Deterministic Tags

Over the shuffled outputs, the election authority then collectively runs a distributed deterministic tag protocol [43, 122, 113]:

$$\mathbf{C}_t^r \leftarrow \text{Tag}(\mathbf{C}_s^r), \quad \mathbf{C}_t^v \leftarrow \text{Tag}(\mathbf{C}_s^v)$$

Each tag binds deterministically to the underlying public key $m = c_{\text{pk}}$ without revealing m , with proofs of correct tag computation.

Tag Matching and Standing Vote Weighting.

Let $m[\tau]$ denote the multiplicity of tag τ in $\mathbf{C}_t^r$:

$$m[\tau] = |\{i : \mathbf{C}_t^r[i] = \tau\}|$$

Let $J = \{j \mid \mathbf{C}_t^v[j] \in \text{Unique}(\mathbf{C}_t^r)\}$ be the set of ballot indices whose credential tags appear in the roster tags. We construct the multiset of kept vote ciphertexts by weighting each matched ballot by the multiplicity of its tag:

$$\mathbf{V}_r = \bigsqcup_{j \in J} \underbrace{\{\mathbf{V}_{s_j}, \dots, \mathbf{V}_{s_j}\}}_{m[\mathbf{C}_t^v[j]] \text{ copies}}.$$

Essentially, standard real-credential ballots are kept once (since their roster tag has multiplicity 1) while a political party ballot is kept P_n times where P_n is the number of delegations to that party (the multiplicity of its tag in $\mathbf{C}_t^r$). Ballots cast with fake credentials are excluded because their tags do not appear in $\mathbf{C}_t^r$.

Decryption & Tally.

Finally, the election authority verifiably decrypts $\mathbf{V}_r$ to obtain plaintext votes $\mathbf{V}_d$ and publishes the final tally X .

$$\mathbf{V}_d = \{\text{EG.Dec}(\mathbb{A}_{\text{sk}}, \mathbf{V}_r) \mid \mathbf{V}_r\}$$

Chapter 7. VLT: A Verifiable Coercion-Resistant Tallying Scheme with Standing Votes

Along with the final tally X , the election authority publishes the proof bundle P consisting of:

- mixnet proofs for $\mathbf{C}^r \rightarrow \mathbf{C}_s^r$ and $(\mathbf{C}^v, \mathbf{V}) \rightarrow \mathbf{C}_s^v, \mathbf{V}_s$
- proofs of correct tag computation on $\mathbf{C}_s^r, \mathbf{C}_s^v$, along with the actual tags $\mathbf{C}_t^r$ and $\mathbf{C}_t^v$
- decryption proofs for $\mathbf{V}_r$

Party-credential audit. For any tag τ with $m[\tau] > 1$, the election authority additionally verifiably decrypts and publishes the credential component of the unique ballot $(\mathbf{C}_s^v, \mathbf{V}_s)$ to obtain the political party's public key c_{pk} .

7.3 Security Analysis

This section provides proof sketches, along with the system and cryptographic assumptions, that VLT achieves the properties listed in §5.1. The public transcript of an election consists of the immutable sub-ledgers $\mathbb{L}_R, \mathbb{L}_V, \mathbb{L}_C, \mathbb{L}_P$, together with the posted mixnet transcripts, tag transcripts, and decryption proofs; we denote it as trans .

7.3.1 Adversaries and Assumptions

We analyze each property against the strongest PPT adversary consistent with that property's goal. The integrity adversary targets voter eligibility (Property 1), individual verifiability (Property 4), and universal verifiability (Property 3). The privacy adversary targets ballot privacy (Property 2), and the coercion adversary targets coercion-resistance (Property 5). All adversaries can fully observe the public transcript trans .

System Assumptions

As part of our system assumptions, we abstract registration so that proofs focus on tallying.

Assumption 1 (Ledger). The ledger, including all of its subledgers, is public and append-only. For each incoming ballot, the ledger enforces $c_{pk} \in \mathbb{L}_C$ (for voter ballots) or $c_{pk} \in \mathbb{L}_P$ (for party ballots), verifies NIZK proofs and the signature under c_{pk} , and ensures at most one ballot per c_{pk} appears on $\mathbb{L}_V$ via a spent bit.

Assumption 2 (Standing-Party Registry). $\mathbb{L}_P$ binds each political party identifier P to a public key c_{pk} , together with a proof of knowledge of the corresponding private key c_{sk} .

Assumption 3 (Registration). The verifiable, coercion-resistant registration process satisfies the following:

1. *Roster Correctness.* For each eligible voter V , the registration roster $\mathbb{L}_R$ contains exactly one ciphertext $c_{pc} = \text{EG.Enc}(A^{pk}, m_v)$, where m_v is either the voter's real credential public key c_{pk} or a political party's public key $c_{pk} \in \mathbb{L}_P$.

2. *No Public Link*. There is no public link between a voter's real credential public key c_{pk} and their roster ciphertext c_{pc} in $\mathbb{L}_R$.
3. *Credential Indistinguishability*. To any party other than the voter, real and fake credentials are computationally indistinguishable.
4. *Non-transferable receipts*. Any receipts/transcripts given to the voter are non-transferable (i.e., they can be simulated).
5. *Voter Checks*. An honest voter can verify their own entries in $\mathbb{L}_C$ and $\mathbb{L}_R$.
6. *Publication Policy*. The posting schedule of $\mathbb{L}_C$ follows a public policy and is independent of the registration session (i.e., standard, standing-vote).
7. *Registrar Issuance*. $\mathbb{L}_C$ contains the registrar-issued voter credentials (real and fake).

Assumption 4 (Anonymous Voting Channel). Ballots are submitted over an anonymous channel (no link between a voter and their cast ballot).

Cryptographic Assumptions

Assumption 5 (ElGamal). ElGamal is IND-CPA secure in the chosen group.

Assumption 6 (Signatures). The signature scheme is EUF-CMA secure.

Assumption 7 (NIZK Proofs). Zero-knowledge proofs are complete, sound, and zero-knowledge. Interactive zero-knowledge proofs are made non-interactive when stated using the Fiat-Shamir heuristic.

Assumption 8 (Verifiable Mixnet). Let Mix be a publicly verifiable shuffle for vectors of ElGamal ciphertexts and MixK its tuple variant that preserves within-tuple association (E_{cr}, E_{vt}) . It satisfies:

1. *Correctness*. Accepted proofs certify that outputs are a re-randomized permutation of inputs (for vectors and tuples).
2. *Soundness*. No accepting proof exists for outputs that are not a re-randomized permutation of the inputs.
3. *Zero-Knowledge*. There is a simulator that, given only the relation “output is some re-randomized permutation of input,” produces transcripts indistinguishable from real ones.
4. *Public verifiability*. Interactive proofs are made non-interactive using the Fiat-Shamir heuristic.
5. *Privacy*. If at least one mix server is honest, the final permutation is hidden from any party lacking all servers' permutations.

Assumption 9 (Verifiable Deterministic Tags). A distributed protocol Tag takes an ElGamal ciphertext and outputs a tag T with a publicly verifiable transcript Π_{tag} . It satisfies:

1. *Correctness*. For any two ElGamal ciphertexts C, C' encrypting plaintexts m, m' :

$$\Pr[\text{Tag.tag}(C) = \text{Tag.tag}(C')] = \begin{cases} 1 & \text{if } m = m', \\ \text{negl}(\lambda) & \text{if } m \neq m'. \end{cases}$$

Chapter 7. VLT: A Verifiable Coercion-Resistant Tallying Scheme with Standing Votes

2. *Public verifiability.* Anyone can verify Π_{tag} as correct for T from C .
3. *Privacy.* There exists a simulator that, given only an equality oracle on plaintexts, outputs (T, Π_{tag}) indistinguishable from real, provided at least one authority keeps its tag secret.
4. *Context separation.* Tags across elections are unlinkable.
5. *Robustness.* No fixed-point tags; for any well-formed ciphertext C , $\text{Tag.tag}(C)$ does not result in a public constant with non-negligible probability.

Assumption 10 (Distributed Decryption). Distributed ElGamal decryption requires n parties to collectively decrypt a ciphertext. Decryption is accompanied by publicly verifiable proofs of correct partial decryption.

7.3.2 Voter Eligibility

Theorem 4 (Voter Eligibility, Property 1, Adversary $\mathcal{S}$). *Under A1, A6, and A7, any honest voter who submits a well-formed ballot is accepted exactly once. Malformed or duplicate submissions are rejected.*

Proof Sketch. An honest voter's VSD constructs $B = (c_{\text{pk}}, E_{\text{cr}}, E_{\text{vt}}, \pi_{\text{cr}}, \pi_{\text{vt}}, \sigma_{\text{vt}})$ and posts it over the anonymous channel. The ledger validates that c_{pk} is admissible ($c_{\text{pk}} \in \mathbb{L}_C \vee c_{\text{pk}} \in \mathbb{L}_P$), $\pi_{\text{cr}}, \pi_{\text{vt}}$ verify (by A7), and σ_{vt} is a valid signature under c_{pk} (by A6), then enforces no duplicates with $\text{spent}[c_{\text{pk}}]$. Therefore, an honest voter's well-formed ballot is accepted exactly once. Any acceptance of an ineligible or duplicate ballot would require violating ledger immutability (A1), forging σ_{vt} , or producing accepting NIZK proofs for false statements, each negligible by assumption. $\square$

7.3.3 Privacy

We formalize ballot privacy via a standard indistinguishability game in which the adversary chooses two messages for a set of challenge ballots and must guess which message was used, given the full transcript trans.

Game $_{\text{priv}}^b(\mathcal{S})$.

- **Setup:** The challenger generates the authorities' keys and public parameters. Registration executes and produces $\mathbb{L}_R, \mathbb{L}_C, \mathbb{L}_P$ satisfying A2 and A3.
- **Query:** $\mathcal{S}$ chooses a non-empty set H of honest voters and for each $i \in H$ two equal-length messages $(m_i^0, m_i^1) \in M \times M$.
- **Challenge:** For each $i \in H$, the challenger emulates the VSD posting an honest ballot $B_i = (c_{\text{pk},i}, E_{\text{cr},i}^b, E_{\text{vt},i}^b, \pi_{\text{cr},i}^b, \pi_{\text{vt},i}^b, \sigma_{\text{vt},i}^b)$ with $E_{\text{vt},i}^b = \text{EG.Enc}(\mathbb{A}_{\text{pk}}, m_i^b)$, $E_{\text{cr},i}^b = \text{EG.Enc}(\mathbb{A}_{\text{pk}}, c_{\text{pk},i})$, and valid proofs/signature. The ledger applies its acceptance rules and appends B_i to $\mathbb{L}_V$. After the voting periods, $\mathbb{A}$ runs the verifiable mixnets Mix, MixK over $\mathbf{C}^r$ and $(\mathbf{C}^v, \mathbf{V})$, computes tags $\mathbf{C}_t^r, \mathbf{C}_t^v$, performs tag matching, verifiably decrypts the resulting $\mathbf{V}_r$, and posts the final tally X and proofs into trans.

- **Output:** $\mathcal{P}$ outputs a bit b' .

The adversary's advantage is $\text{Adv}^{\text{priv}}(\mathcal{P}) = |\Pr[b' = b] - 1/2|$.

Theorem 5 (Privacy, property 2, Adversary $\mathcal{P}$). *Under A4, A5 and 7 to 10, the public transcript in $\text{Game}_{\text{priv}}^0$ and $\text{Game}_{\text{priv}}^1$ are computationally indistinguishable; $\text{Adv}^{\text{priv}}(\mathcal{P})$ is therefore negligible.*

Proof Sketch. The proof follows a sequence of hybrids, each changing the challenger's behavior slightly.

Hybrid 1 (Real). This is $\text{Game}_{\text{priv}}^b$ as defined: each honest VSD posts B_i with

$$E_{vt,i}^b = \text{EG.Enc}(\mathbb{A}_{\text{pk}}, m_i^b)$$

and valid $(\pi_{vt,i}^b, \pi_{cr,i}^b, \sigma_{vt,i}^b)$. The ledger performs checks and posts to $\mathbb{L}_V$. At tally time, $\mathbb{A}$ runs Mix, MixK, then Tag on $(\mathbf{C}_s^t, \mathbf{C}_s^v)$, then decryption with proofs, and publishes X and all transcripts.

Hybrid 2 (Encrypt the other message). For each challenge ballot $i \in H$, replace $E_{vt,i}^b$ by

$$E_{vt,i}^{1-b} = \text{EG.Enc}(\mathbb{A}_{\text{pk}}, m_i^{1-b})$$

while keeping all other components and ledger behavior unchanged. By ElGamal IND-CPA (A5) and independence between E_{vt} and the public $c_{\text{pk},i}$, $\mathcal{P}$'s view is indistinguishable.

Hybrid 3 (Simulated ballot-level NIZKs). Replace $(\pi_{vt,i}^b, \pi_{cr,i}^b)$ by simulated proofs for the same statements using NIZK simulators with per-election domain separation (A7). Since statements remain true, completeness and zero-knowledge imply indistinguishability.

Hybrid 4 (Simulated mixnet transcripts). Replace the real mixnet proofs $(\Pi_{\text{mix}}, \Pi_{\text{mixK}})$ with simulator outputs for shuffles that are consistent with the posted inputs/outputs (i.e., some re-randomized permutation), and treat the mixed ciphertexts as coming from an ideal permutation unknown to $\mathcal{P}$. By the mixnet's zero-knowledge and soundness properties with at least one honest mix server (A8), indistinguishable.

Hybrid 5 (Simulate deterministic tags). Replace $(\mathbf{C}_t^t, \mathbf{C}_t^v)$ and the tag proofs by the output of the tag simulator given only equality-of-plaintext oracles; ensure multiplicities and matches reflect the (ideal) permutation's equality pattern. By Tag's privacy and public verifiability with at least one honest authority (A9) and the timing discipline (post-mix only), indistinguishable.

Hybrid 6 (Simulated decryption proofs). Replace decryption proofs by simulated ones for the same ciphertexts and posted plaintexts. By publicly verifiable threshold decryption ZK (A10), indistinguishable.

Chapter 7. VLT: A Verifiable Coercion-Resistant Tallying Scheme with Standing Votes

Since hybrid 2 swaps m_i^b and m_i^{1-b} for all $i \in H$ and the subsequent hybrids preserve indistinguishability, hybrid 1 with bit b is indistinguishable from hybrid 2 with bit $1 - b$, hence from hybrid 1 with bit $1 - b$. Therefore $\text{Adv}^{\text{priv}}(\mathcal{P})$ is negligible. $\square$

The decryption of standing votes post-tally only affects political parties' votes, not honest voters. Revealing c_{pk} at cast time is harmless because E_{vt} is independent of c_{pk} and mix+tags break linkage between cast events and decrypted outputs.

7.3.4 Universal Verifiability

We first capture roster membership with multiplicity (standing votes).

Lemma 6 (Roster Membership with Multiplicity). *Under A7–A10, the publicly verified tally keeps, for each public key k :*

- if k appears once in $\mathbb{L}_R$ (standard registration), exactly one vote ciphertext paired with $\text{Tag}(k)$.
- if k appears $n > 1$ times (party key with n delegations), the unique party ballot under k is kept with multiplicity n .
- if $k \notin \mathbb{L}_R$ (fake), zero ballots.

Proof Sketch. After shuffles, tag correctness implies equal tags if and only if equal plaintext keys. Tag privacy leaks nothing else. The join “keep if and only if $\mathbf{C}_t^v[j] \in \mathbf{C}_t^r$ ” realizes membership. Multiplicity $m[\text{Tag}(k)]$ induces weighting as defined by $\mathbf{V}_r = \bigsqcup_{j \in J} \{\mathbf{V}_{s_j}\}^{m[\mathbf{C}_t^v[j]]}$. Ledger-level per-key state ensures at most one accepted ballot per key, so multiplicity arises only from roster multiplicity, not from duplicate ballots. $\square$

Property 7 (Standing-Delegation Correctness). For each party key $c_{\text{pk}} \in \mathbb{L}_P$, let P_n be the multiplicity of $\text{Tag}(c_{\text{pk}})$ in the roster tags. The tally credits exactly P_n instances of the unique accepted party ballot under c_{pk} , and publishes a proof that the credential component of that ballot decrypts to a key in $\mathbb{L}_P$.

Theorem 6 (Universal Verifiability, property 3, $\mathcal{J}$). *Under A 1 to 3 and 7 to 10, any observer can verify that the outcome X equals tallying all and only eligible contributions recorded on $\mathbb{L}$. In particular:*

1. Every recorded ballot met eligibility rules (Property 1).
2. Only real ballots—those whose credential tags match roster tags—were kept, with party ballots weighted by the number of delegations (Lemma 6, Property 7).
3. Per key, at most one ballot contributes (with multiplicity for party keys only).
4. Decryptions are correct and consistent with the published proofs.

Proof Sketch. Define a deterministic public procedure $\text{PublicVerify}(\text{trans})$:

1. Verify registrar signatures and consistency of $\mathbb{L}_C, \mathbb{L}_R, \mathbb{L}_P$ (A2 and A3).

2. For each ballot on $\mathbb{L}_V$, verify σ_{vt} and ballot-level NIZK proofs (A7) and check for one ballot per c_{pk} (A1).
3. Verify mixnet proofs for $\mathbf{C}^r \rightarrow \mathbf{C}_s^r$ and $(\mathbf{C}^v, \mathbf{V}) \rightarrow (\mathbf{C}_s^v, \mathbf{V}_s)$ (A8).
4. Verify tag proofs for $\mathbf{C}_s^r, \mathbf{C}_s^v$ and compute the join using $\mathbf{C}_t^r, \mathbf{C}_t^v$, check that kept ballots follow Lemma 6, and for tags with multiplicity > 1 , verify the party-credential audit proofs (Property 7).
5. Verify decryption proofs and recompute X from the resulting $\mathbf{V}_d$ (A10).

If and only if all steps accept, output 1. Soundness of each proof system ensures $\text{PublicVerify}(\text{trans}) = 1$ exactly when trans encodes a unique permutation, membership/weights, and decryptions consistent with $\mathbb{L}$.

Importantly, correctness is relative to the posted contents on the sub-ledgers $\mathbb{L}_C, \mathbb{L}_R, \mathbb{L}_P, \mathbb{L}_V$. $\square$

7.3.5 Individual Verifiability

Theorem 7 (Individual Verifiability, property 4, $\mathcal{I}$). *Under A1, A3, A6, and A7, any honest ordinary voter who follows the protocol can verify:*

1. Recorded-as-Registered: *their registration transcript binds their real credential to the exact roster ciphertext posted on $\mathbb{L}_R$.*
2. Recorded-as-Cast: *their ballot appears on $\mathbb{L}_V$ under their c_{pk} with a valid signature and valid proofs, and is accepted.*

Voters registering to cast a standing vote are not subject to individual verifiability, but they can audit the votes cast by the political party (Property 7).

Proof Sketch. We formalize what a voter can check locally and what is guaranteed by the public proofs:

Recorded-as-Registered. During registration in the privacy booth, the kiosk creates the public credential tag $c_{pc} = \text{EG.Enc}(\mathbb{A}_{pk}, c_{pk})$ and runs a sound interactive proof that c_{pc} encrypts the voter's real public key c_{pk} . Let Π_{reg} denote the transcript of this proof on their real credential. At check-out, the official's device posts a signed record containing c_{pc} to $\mathbb{L}_R$ along with the kiosk and official's signatures). The voter verifies locally that

- Π_{reg} is a valid proof that c_{pc} encrypts their c_{pk} .
- the exact c_{pc} from Π_{reg} appears as the voter's last registration record on $\mathbb{L}_R$.

This establishes that the roster contains an encryption of the voter's actual c_{pk} , with c_{pk} having no public link to the voter's identity (A3).

Chapter 7. VLT: A Verifiable Coercion-Resistant Tallying Scheme with Standing Votes

Recorded-as-Cast. When the voter selects their vote, their VSD forms

$$B = (c_{pk}, E_{cr}, E_{vt}, \pi_{cr}, \pi_{vt}, \sigma_{vt})$$

and submits it over the anonymous channel.

Their VSD then checks on $\mathbb{L}_V$ that

- B appears under the same c_{pk} .
- σ_{vt} verifies under c_{pk} (A6).
- π_{cr}, π_{vt} verify (A7).
- The ledger has accepted the ballot according to the policy that only the first vote cast is admissible (A1).

These checks confirm the ballot is recorded exactly as cast under the voter's credential.

For complete proofs about the TRIP registration scheme achieving individual verifiability, see theorem 1.

Committed-As-Cast: Standing Votes. By policy, voters registering for a standing vote receive no standing-vote transcript linking their identity to a posted roster entry. Consequently, they cannot perform recorded-as-registered. Furthermore, since they also do not vote, they have no need to perform recorded-as-cast. Instead, they can perform a committed-as-cast check where they verify that the political party voted as publicly stated.

- Each party P publishes a public statement of its standing vote.
- At tally time, the authority decrypts any tag with multiplicity > 1 , revealing the credential component c_{pk} , along with a proof of correctness.
- The voter checks that the decrypted vote corresponding to that credential c_{pk} matches the public statement.

□

7.3.6 Coercion-Resistance

We consider a JCJ-style game where a coercer interacts with the target voter at any time, similar to Figure 6.15 but with the addition of standing votes. They can demand actions and voter-obtained transcripts (e.g., “vote for β ,” “delegate to party P ,” “show receipts/keys”). They also see the public trans. The adversary's goal is to distinguish a world where the voter complies from one where the voter deviates.

Game $\text{cr}^b(\mathcal{C})$. The game proceeds as follows:

- **Setup:** As in privacy, the construction of the $\mathbb{L}_R, \mathbb{L}_C, \mathbb{L}_P$ sub-ledgers.

- **Interaction:** $\mathcal{C}$ engages the target with an arbitrary script including registration-time and casting-time demands, and post-election inspection.
- **Challenge:** For a hidden bit b , the challenger either enforces the coercer's script ($b = 1$) or has the voter deviate while remaining indistinguishable to $\mathcal{C}$ ($b = 0$), e.g., casting with a real credential or delegating to a different party and leaving with only fake credentials.
- **Output:** $\mathcal{C}$ sees trans and outputs a bit b' .

$\mathcal{C}$'s advantage is $\text{Adv}^{\text{cr}}(\mathcal{C}) = |\Pr[b' = b] - 1/2|$.

Theorem 8 (Coercion-Resistance, property 5, $\mathcal{C}$). *Under A 1 to 5 and 7 to 10, $\text{Adv}^{\text{cr}}(\mathcal{C})$ is negligible.*

Proof Sketch. We build a simulator S that, given $\mathcal{C}$'s script, produces a transcript indistinguishable from a real execution in which the target deviates.

Registration phase. If $\mathcal{C}$ requests target voter's receipts, S simulates them for a standard or standing-vote registration consistent with the contents on $\mathbb{L}_R, \mathbb{L}_C, \mathbb{L}_P$ (A2 and A3). Credential indistinguishability ensures $\mathcal{C}$ cannot tell real from fake credentials (A3).

Casting phase. If $\mathcal{C}$ demands a specific vote (valid or invalid) and proof, the voter can cast with a fake credential from $\mathbb{L}_C$. The ledger accepts at cast time, and indistinguishability prevents any kind of real vs. fake test by even the ledger. If $\mathcal{C}$ demands abstention, the anonymous channel prevents linking the voter to a vote they cast.

Delegation coercion. If $\mathcal{C}$ demands delegation to party P , by policy, no standing vote receipt exists. Later, even if $\mathcal{C}$ observes P_n standing votes for P , it cannot link any particular delegated vote to the target, and renewals introduces statistical uncertainty across elections.

Tally-time linkability. Mix-then-tag with at least one honest authority provides equality-only leakage. Smith/Weber algebraic-relation tests are mitigated because ballots must use registrar-issued keys from $\mathbb{L}_C$ or $\mathbb{L}_P$ (no arbitrary σ, σ^α construction), and tags are computed only after mixing.

Thus, any distinguishing advantage reduces to breaking one of the underlying assumptions or to targeting the entire electorate (excluded by the game).

Importantly, coercion-resistance is relative to the information that the adversary can learn by the tally. $\square$

7.4 Scheme Instantiation & Implementation

This section details the instantiation of the VLT scheme and its implementation.

7.4.1 Instantiation

We instantiate all of our cryptographic operations over the NIST P-256 curve (prime order $q \approx 2^{256}$, 128-bit security). We use ElGamal [36] over elliptic curves for encryption, Schnorr signatures [109] for ballot authentication, the Bayer–Groth verifiable shuffle [12] for both roster and pair-preserving ballot mixes, SHA-256 for all hash-based operations, and the distributed deterministic tag protocol from UnLynx [43] to compute plaintext-equality tags.

ElGamal (§4.2). Public keys are $H = xG$ with base point G . To encrypt a group element M , we output $(C_1, C_2) = (r \cdot G, M + rH)$ with fresh $r \in \mathbb{Z}_q$. For vote choices, we encode each candidate identifier $\beta \in M$ into a fixed group element. This enables a standard disjunctive set membership proof that the plaintext corresponds to one of the vote options in M . For credential encryption, the plaintext is the voter's P-256 public key c_{pk} . We bind E_{cr} to c_{pk} with a Schnorr-style proof of knowledge c_{sk} showing that $c_{pk} = c_{sk} \cdot G$, with a relation tying E_{cr} to c_{pk} . All zero-knowledge proofs (for voting) are made non-interactive using Fiat-Shamir.

Schnorr Signatures (§4.3). The entire contents of the ballot (except the signature itself) are authenticated with Schnorr over P-256.

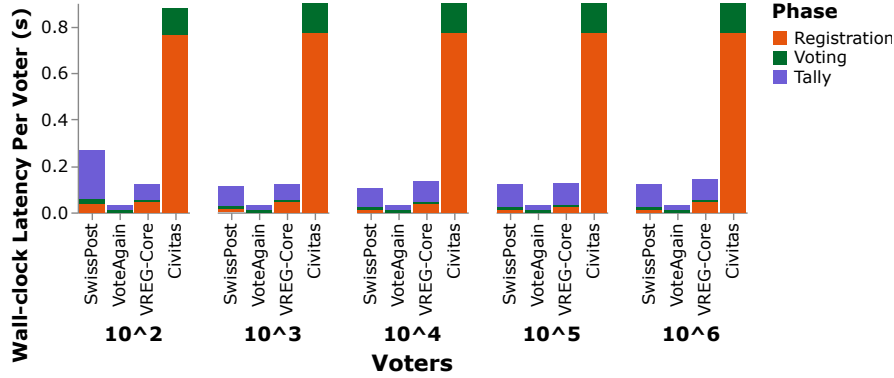
Verifiable Mixnets (§4.7). For the roster ciphertext, we use standard Bayer-Groth. For proving a correct shuffle of multiple ElGamal ciphertexts, we adopt a proof technique from Neff's shuffle paper [80] that provably compresses these ciphertexts into one before reverting back to a standard shuffle.

Deterministic Tags (§4.8). We adopt the UnLynx distributed deterministic tagging: authorities run a distributed protocol that essentially replaces the unique randomness in each ElGamal ciphertext with a single randomness across all ciphertexts.

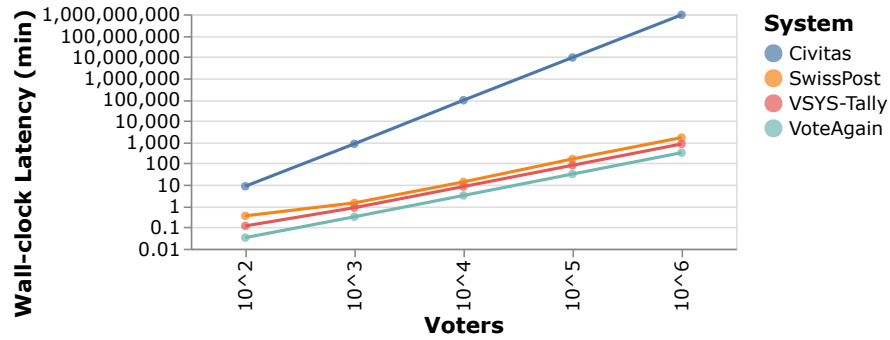
Threshold Decryption (§4.8). Each election authority member produces a decryption share and a proof of correct partial decryption. Shares are then combined to retrieve the plaintext.

7.4.2 Code

We implement VLT in 1,816 lines of Golang [78], and 424 lines of C. We use the Kyber cryptographic library written in Go [31] to implement most of the above primitives. We use a Bayer-Groth library written in C [29] to shuffle and prove the correctness of the shuffle. We enable communication between the Go program and the C program we wrote to interface with the Bayer-Groth library through serializing ciphertexts into files that can be deserialized by the other program.



(a) **Latency per voter for registration, voting, and tallying phases.** We exclude the Civitas tally phase to avoid skewing the graph's scale.



(b) **Tally phase latency**

Figure 7.2: **Comparing Phase Execution across Voting Systems.** For each voting system, we used four election authority members. With VoteAgain, since this was not a configurable parameter, we ran the mixing process four times. Due to Civitas' quadratic time complexity, we extrapolate its latency beyond 10^4 voters.

7.5 Evaluation

In this section, our aim is to answer: how does an online voting system built on VLT and TRIP perform, and scale, measuring the complete “end-to-end” (E2E) voting pipeline from setup through tallying?

We measured the execution latency of each phase for configurations ranging from 100 to 1 million voters while maintaining four talliers. Because tallying in Civitas has quadratic complexity, we extrapolated its results after 1,000 voters due to time constraints. Figure 7.2a (reproduced from Figure 6.18) depicts the total execution latency per voter across each phase (excluding Civitas's tally phase), and Figure 7.2b compares systems' tally latencies.

We observe that during the voting phase alone, the latency per voter for TRIP, Swiss Post, VoteAgain and Civitas is 5ms, 10ms, 1ms, 128ms, respectively. The latency cost of voting is independent of voting population in all systems, unsurprisingly since this phase is “embar-

prisingly parallel.” Voting in Votebral performs twice as fast as Swiss Post and 24 times faster than Civitas but 5 times slower than VoteAgain.

In the tally phase (fig. 7.2b), VLT requires approximately 14 hours for 1 million voters, compared to 5 hours for VoteAgain, 27 hours for Swiss Post and an impractical 1,768 *years* (estimated) for Civitas. While Civitas can enhance its tally performance by dividing ballots into groups, the same method can be used by Votebral when running elections that involve tens if not hundreds of millions of voters; however, this method does weaken coercion-resistance since the statistical uncertainty is confined to a smaller set of voters. Yet, an anonymity set of 1 million voters is better than an anonymity set of a thousand voters.

In summary, we find that Votebral significantly outperforms Civitas in “end-to-end” performance across the pipeline, while also competing closely with the deployed but non-coercion-resistant Swiss Post voting system and the coercion-resistant but non-verifiable VoteAgain system.

7.6 Discussion

7.6.1 Re-Voting

Re-voting can be supported as a policy option (distinct from *deniable re-voting*). Under a “last-ballot-wins” rule, the ledger accepts multiple ballots per c_{pk} but counts only the most recent one (e.g., using a per-credential sequence number or timestamp). This variant does not aim to hide re-voting from a coercer who knows the voter’s real credential; rather, it allows ordinary preference changes during the voting period. If the coercer does *not* know the voter’s real credential, re-voting remains unobservable.

7.6.2 Scaling to Millions of Voters

Tallying one million ballots in our prototype is a significant step, but nationwide deployments (hundreds of millions) require further scale-out. As hinted in evaluation, partitioning voters across multiple instances of VLT is possible, and electoral partitioning naturally supports this. In the United States, there are over 10,000 electoral jurisdictions [2] consisting on average 17,400 voters given 174 million eligible voters [1]. Therefore, on average, each voter could create 56 fake credentials and cast 57 ballots to reach 1 million ballots per electoral jurisdiction. In Switzerland, 20 of 26 cantons have fewer than 500,000 voters, making VLT also suitable.

7.6.3 Voting History

Since Votebral uses fake credentials, a voter could — in principle — view a history of ballots cast with a particular credential (real or fake) without creating a transferable proof of compliance. The history reveals how that *credential* voted, but not whether the credential is real or fake,

and therefore not whether those ballots counted. Being able to see later how one voted is an obvious feature often desired in e-voting systems, but is usually not allowed due to receipt-freeness concerns [17].

This feature does not affect verifiability, Votebral remains sound even if VSD does not expose history. However, allowing voters to view their past votes may improve the *perception* of transparency and trust in an e-voting system. Furthermore, this approach actually offers a verifiability advantage, as each vote can be accompanied by a receipt proving the ballot contains that vote. Voters could use a second device to verify the accuracy of their vote not only for the current election but also for all previous ones. Additionally, even if a second device is not available immediately, a new device purchased later can verify the integrity of votes cast on the previous device(s). Studying such psychological effects, however, remain future work.

Voters can also request the election authority to reveal all previous votes cast with a credential stored on their device. To accomplish this, the voter's device proves ownership of the credential to each election authority member and requests verifiable decryption shares of the ballots cast with it. The voter's device then reconstructs the vote using the decryption shares while verifying its integrity. Importantly, this process does not disclose the vote to any election authority members since the reconstruction is performed locally, on the voter's device.

7.6.4 Voting Methods

Coercion-resistance has to make the assumption that the adversary cannot learn information from the tally X . This, in practice, can be difficult for smaller scale elections (e.g., rural areas and towns) for which the statistical uncertainty is naturally smaller as the number of honest voters and the distribution of votes that they may cast is smaller. To add additional statistical uncertainty to the coercer, instead of decrypting votes, we could run the distributed deterministic tagging step on the real ballot votes themselves after the filtering step that removes fake ballots. With this, we obtain publicly a multiplicity of each vote option without revealing the option itself. Finally, the election authority collectively performs the decryption step only on the vote option that has the highest multiplicity, revealing only the winning option.

In addition, in some jurisdictions, write-in votes may be mandated by law, and this poses a problem since write-in votes could reveal whether a voter complied with the coercer's demands: looking at whether the final tally X contains the unique string that the coercer demanded the voter cast. By not decrypting votes with multiplicity lower than the highest vote option, these write-in votes remain secure.

Votegral Usability Part III

8 Study Methodology

In this chapter, we will primarily investigate the usability of TRIP, the registration component of VoteGral defined in chapter 6, which consists of three central questions along with one meta question:

- Q1. What are voters' perceptions and experiences with coercion and voting systems?**
- Q2. Can voters use the TRIP credentialing process to create their real and fake credentials?**
- Q3. Can voters detect deviations in the credentialing process to achieve verifiability?**
- Q4. Can voters understand and use their fake credentials, while still casting their intended votes with their real credential?**

To answer these questions, we designed and conducted a study with 150 individuals, recruited in a suburban park of Boston, Massachusetts. These participants underwent the TRIP credentialing process and then cast mock votes using a prototype we built specifically for the study. Participants then concluded the study by completing a survey about their experience. Our study received IRB approval and our ethical considerations are presented in §8.2.

Below is an overview of how this study will help us address these questions:

To address Q1, we first ask participants to rate the likelihood of various coercion and vote-buying scenarios, along with potential perpetrators. We also ask participants whether they have experienced or know of someone who has experienced coercion. The wording “or someone you know” is designed to encourage disclosure from those who might otherwise be hesitant to share their experience. To answer perceptions and experiences with existing voting systems, we ask participants to rate the trustworthiness of various voting channels, including the one they just experienced.

To answer Q2, we measured the credentialing success rate among participants. The survey also assessed participants' usability perceptions and user experience using the System Usability Scale (SUS; §3.1) and the User Experience Questionnaire (UEQ; §3.2). The SUS offers a standardized, summative measure of perceived usability, whereas the UEQ provides a more comprehensive, multi-faceted assessment of user experience, detailing aspects such as

Chapter 8. Study Methodology

attractiveness, efficiency, and novelty.

To answer Q3, a subset of participants interacted with a kiosk programmed to act maliciously. This kiosk attempted to misrepresent a fake credential as the participant’s real credential. To achieve this deception, we modified the kiosk’s protocol: participants were instead instructed to select and scan an envelope *before* the commit was printed. This alteration enables the kiosk to generate a fraudulent zero-knowledge proof, designed to deceive verifier devices into accepting it as valid. We then measured the number of participants who detected and reported this anomalous behavior to the facilitator.

To address Q4, we evaluated participants’ understanding of fake credentials through a two-phase approach: an interactive quiz on the kiosk, followed by survey questions. Furthermore, given that fake and real credentials function identically, we instructed participants to vote using their real credential to assess their ability to distinguish their real credential from fake ones.

We now begin by presenting the study groups, including the rationale behind them to enable the detection of statistically significant differences and support conclusive findings. Subsequently, we detail the study materials and describe the study’s setting.

8.1 Study Groups

	No Intervention	Malicious Kiosk	Security Priming	Malicious Kiosk w/ Security Priming
Real Credentials	C			
Real & Fake Credentials	F	M	SF	SM

Table 8.1: **Summary of Study Groups.** Each group consists of 30 participants for a total of 150 participants.

We conduct a between-subjects study, randomly assigning participants equally across five distinct groups, each with their own stimuli. Given our aim to assess a coercion-resistant system using fake credentials, we introduce a control group (C) which exposes participants to only real credentials to assess differences. Next, given our use of a voter-verifiable registration system, we introduce a group which exposes participants to a malicious kiosk (M) that attempts to steal the voter’s real credential by subtly guiding the creation of a fake credential instead of a real one. To understand how an explicit warning could influence the detection rate of a malicious kiosk, we introduce a different instructional video with an explicit warning to participants before undergoing registration, which will be further described in §8.3. We therefore obtain the following five groups, each consisting of 30 participants, also summarized in table 8.1.

- **Control Group (C):** Exposure to only the real credential known as “voting credential”.
- **Fake Credential Group (F):** Exposure to real and fake voting credentials.

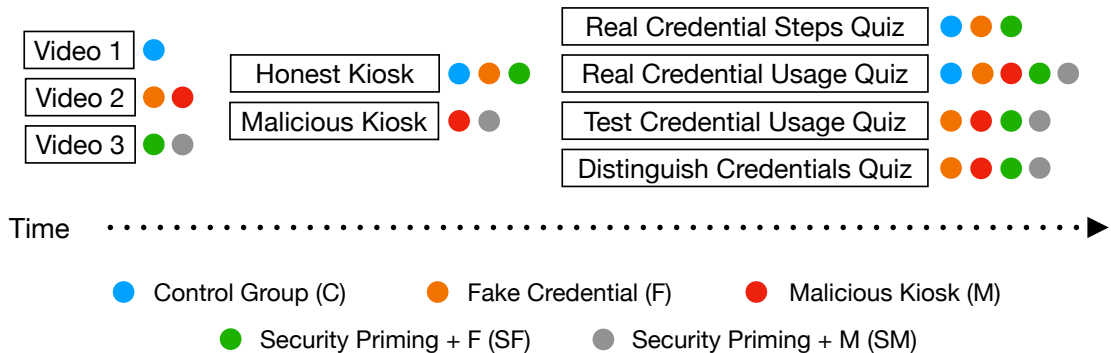


Figure 8.1: **Study Flow Based on Study Group.** Participants first watch one of three instructional videos, then interact with either an honest or malicious kiosk that displays the corresponding quizzes. The distinguish credentials quiz is present for those who created at least one fake credential.

- **Malicious Kiosk Group (M):** Exposure to real and fake credentials, and a malicious kiosk.
- **Security Priming + F (SF):** Exposure to real and fake voting credentials, and an instructional video with security awareness.
- **Security Priming + M (SM):** Exposure to real and fake credentials, a malicious kiosk and an instructional video with security awareness.

Fake Credentials → Test Credentials

For the purposes of the study, we use the term “test credentials” instead of “fake credentials” to avoid the negative connotation that such credentials are inherently bad. Furthermore, the word “test” suggests that they could be used for purposes beyond coercion, such as testing the system or educating friends or family members how to vote online.

8.2 Ethical Considerations

Before participating, each individual received an information sheet outlining the study’s scope, their rights, the expected tasks, and the data we intended to collect. We collected data from participants’ interaction with our devices and paper credentials, supplemented with information from a survey. We respected participants’ privacy by supplying all required materials, including a mobile device to cast a mock vote, and omitting the usual act of presenting an ID when registering to vote. Furthermore, we informed participants that this system is not linked to any real voting systems, and therefore does not actually register them to vote in any elections.

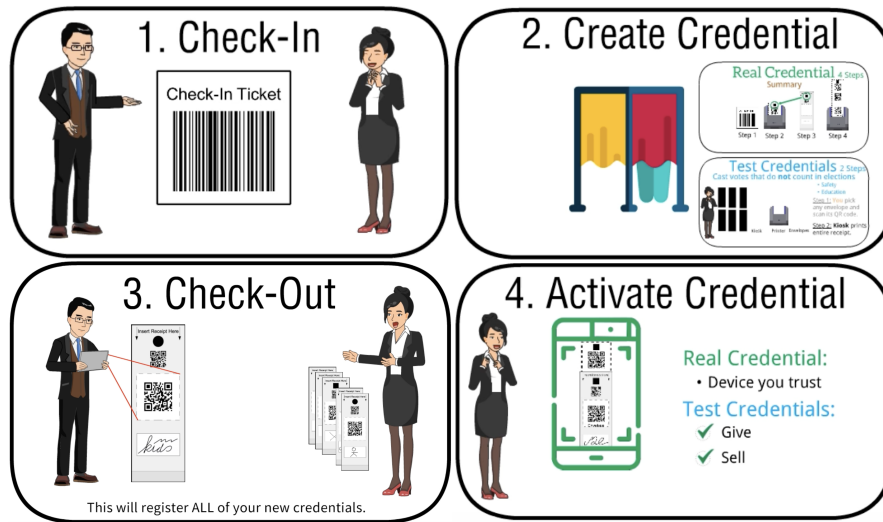


Figure 8.2: **Steps Overview.** Various screenshots of the instructional video that demonstrates each step in the process. For the control group, all signs of test credentials are omitted and “Real Credential” is replaced with “Voting Credential”.

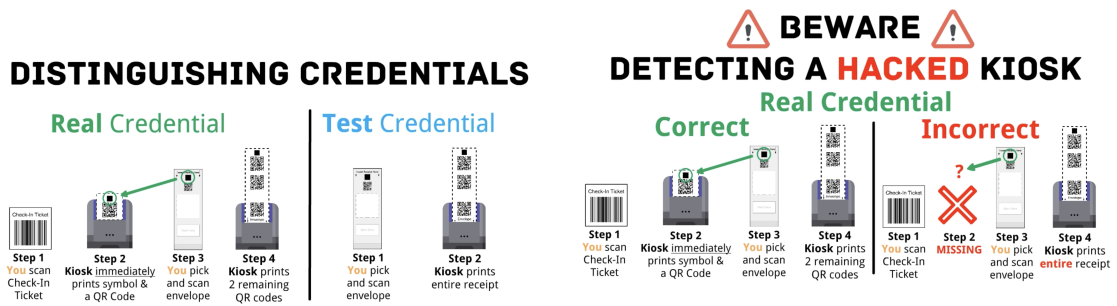
8.3 Study Materials

The study consists of instructional videos to educate the voter, quizzes and a survey to gauge the participants’ understanding of concepts, and a kiosk deliberately set to be malicious to identify whether voters can detect it. We present the study’s physical setup in the following section (§8.4). A visual overview of the study’s flow based on the group the participant is in can be found in fig. 8.1.

Notation. Considering the differences between groups, we adopt specific notations to indicate the relevant text each group encounters. The notation $[X|Y]$ indicates that X is the content present for the control group while Y is the content present for the treatment groups. On the other hand, $[X\$Y]$ indicates that Y is the content present for the malicious kiosk groups (M, SM) while X is the content present for the other groups.

Instructional Videos

We created three videos, available on github.com/dedis/trip-usability/, each illustrating their assigned conditions: control group (video 1), no security priming (video 2) and security priming (video 3). Video 1, lasting 3 minutes and 6 seconds, exclusively demonstrates the creation of a *voting* credential. In contrast, videos 2 and 3, with durations of 5m 50s and 6m 25s respectively, illustrates the creation of real and fake credentials (fig. 8.2). Videos 2 (fig. 8.3a) and 3 (fig. 8.3b) both highlight the differences in creating real and fake credentials. However, video 3 shows, under a conspicuous “BEWARE” sign, how to detect a “hacked” kiosk, should the unlikely event occur.



(a) **Video 2.** Participants are exposed to a slide that shows, side by side, the creation of a real credential and of a fake credential.

(b) **Video 3 (Security Priming).** Participants are exposed to a slide that depicts how a kiosk can trick the voter into creating a fake credential instead of a real credential.

Figure 8.3: **Instructional Video Differences.** Screenshots from the instructional video concerning how to distinguish real and fake credentials.

We use videos as our primary source of instructional material, as individuals learn better from dynamic visuals compared to static images [92]. We select whiteboard animated videos due to their positive impacts on retention, engagement and enjoyment, even when conveying complex material [119]. To enhance the effectiveness of our videos, we incorporate the dynamic drawing principle, adopt a first-person perspective, and include narration with subtitles [76].

Malicious Kiosk

In groups M and SM, we introduced participants to a misbehaving kiosk which follows a deviation to *covertly* violate the integrity of the real credential. The deviation involves having the voter scan an envelope before the kiosk prints the ZKP commit on receipt allowing the kiosk to keep the real credential and trick the voter into creating a fake credential for which the voter believes is their real credential. Other deviations are possible and worth studying in the future but they are either more overt (e.g., skipping the creation of a real credential) or detectable by the voter's personal device activation (e.g., the cryptographic validity of the zero-knowledge proof).

Quizzes

To evaluate participants' understanding of concepts, the kiosk exposes participants to up to four unannounced, multiple choice quizzes shown below, with the correct answer italicized. The selection of quizzes depends on the participant's group assignment and actions, and the quizzes are shuffled for each participant. We expect that at least some of these quizzes would be present in reality to foster active learning and be reminded each time of the steps involved each time voters renew their credentials so that if voters were to ever be exposed to a malicious kiosk they would detect it.

Chapter 8. Study Methodology

Real Credential Steps. While creating your [real] voting credential, when do you pick and scan an envelope to the kiosk? (Check all that apply)

- After a symbol and three QR codes are printed
- *After a symbol and one QR code is printed*
- Before anything is printed

This quiz, given to participants not exposed to the malicious kiosk, assesses their understanding of the correct timing of when to pick and scan an envelope. The quiz is omitted for the malicious kiosk groups because the kiosk is being deceptive of this exact process.

Real Credential Storage. What should you do with your [real] voting credential after leaving the government office? (Check all that apply)

- *Activate it yourself on a device that you trust.*
- Give it to someone else to activate on my behalf.

This question is given to all participants to evaluate their understanding that they must keep and activate their real credential on a trusted device.

Test Credential Usage. Which of the following purposes can you use a test credential for? (check all that apply)

- *To teach kids about voting.*
- *To resist pressure from someone to vote a particular way.*
- *To sell to someone who offers to buy your real credential.*
- To cast a vote that counts in an election.

This quiz is present for all participants in the treatment groups to examine participants' recall of the stated purposes of fake credentials.

Credential Distinguish. Who can distinguish your test credential from your real credential? (Check all that apply)

- Anyone with a device.
- *Only myself with my pen markings.*

This quiz is given to all participants in the treatment groups who elect to create at least one fake credential and whether they understand that only they can differentiate their fake credentials.

Exit Survey

We conduct a survey to gain valuable information about the participants' perceptions and experience with TRIP, in addition to coercion and their level of trust in various voting channels. The average time participants took to complete the quiz was 17 minutes.

Background. *The survey starts by asking voters their demographic information. Participants could choose not to answer.*

1. What is your age?
2. What is your gender?
3. What is your ethnicity?
4. What is the highest degree or level of education you have completed?

System Usability Qualitative. *The survey then asks the following qualitative questions.*

1. Please describe your online voting signup experience in a single word.
2. What elements about this voter registration system did you like most?
3. What elements about this voter registration system did you like least?

System Usability Scale (SUS) *The survey proceeds with asking voters to complete the SUS scale.*

Please rate each statement on a scale ranging from 1 “Strongly Disagree” to 5 “Strongly Agree”:

1. I would like to use this voter registration system whenever I renew my identifications documents (i.e., every 5-10 years)
2. I found the system unnecessarily complex.
3. I thought the system was easy to use.
4. I think that I would need the support of a technical person to be able to use this system.
5. I found the various functions in this system were well integrated.
6. I thought there was too much inconsistency in this system.
7. I would imagine that most people would learn to use this system very quickly.
8. I found the system very cumbersome to use.
9. I felt very confident using the system.
10. I needed to learn a lot of things before I could get going with this system.

Since voters are not expected to register frequently, we rephrase the first prompt from “I would like to use this system frequently” (§3.1) to “I would like to use this voter registration system whenever I renew my identifications documents (i.e., every 5-10 years)” to better align with the expected usage of this system.

User Experience Questionnaire *The survey now asks voters to complete UEQ*

This questionnaire consists of pairs of contrasting attributes. The circles between the attributes represent gradations between the opposites. You can express your agreement with the attributes by ticking the circle that most closely reflects your impression.

Please decide spontaneously. Don't think too long about your decision to make sure that you convey your original impression. *Table 8.2 presents the attributes.*

UEQ Key Performance Indicator Extension (KPI) *We now ask voters to complete the UEQ*

Chapter 8. Study Methodology

Entry	Left	Right	Scale (not shown)
1	annoying	enjoyable	Attractiveness
2	not understandable	understandable	Perspicuity
3	creative	dull	Novelty
4	easy to learn	difficult to learn	Perspicuity
5	valuable	inferior	Stimulation
6	boring	exciting	Stimulation
7	not interesting	interesting	Stimulation
8	unpredictable	predictable	Dependability
9	fast	slow	Efficiency
10	inventive	conventional	Novelty
11	obstructive	supportive	Dependability
12	good	bad	Attractiveness
13	complicated	easy	Perspicuity
14	unlikable	pleasing	Attractiveness
15	usual	leading edge	Novelty
16	unpleasant	pleasant	Attractiveness
17	secure	not secure	Dependability
18	motivating	demotivating	Stimulation
19	meets expectations	does not expectations	Dependability
20	inefficient	efficient	Efficiency
21	clear	confusing	Perspicuity
22	impractical	practical	Efficiency
23	organized	cluttered	Efficiency
24	attractive	unattractive	Attractiveness
25	friendly	unfriendly	Attractiveness
26	conservative	innovative	Novelty

Table 8.2: UEQ Questionnaire

Key Performance Indicators to gauge the voters' importance of various scales of the UEQ (e.g., Attractiveness, Efficiency, Novelty).

Please rate the importance of the following items relative to your use of an ideal voter registration system for online voting from 1 “Not important at all” to 7 “Very Important”. Do not consider how important these items are in your daily life, but please focus specifically on how important they are when you interact with a voter registration system for online voting.

1. The ideal voter registration system should look attractive, enjoyable, friendly and pleasant.
2. I should perform my tasks with the ideal voter registration system fast, efficient and in a pragmatic way.
3. The ideal voter registration system should be easy to understand, clear, simple, and easy to learn.
4. The interaction with the ideal voter registration system should be predictable, secure and meets my expectations.
5. Using the ideal voter registration system should be interesting, exciting and motivating.
6. The ideal voter registration system should be innovative, inventive and creatively designed.

Fake Credentials. *We now ask voters questions related to fake credentials to gain insights as to whether they are comfortable having and using them.*

1. Did you notice anything odd while creating your real credential? Please explain.
2. Do you remember what a test credential is? Please list the potential uses for a test credential.
3. Did you create a test credential? *If Yes, followup question:*
 - (a) After creating and marking the credentials as instructed, how confident are you that you can remember which credential is your real one?
4. If online voting was an option today to cast votes in political elections, do you imagine yourself creating a test credential alongside your real credential? Why or why not?
5. If online voting was an option today to cast votes in political elections, do you image yourself watching an instructional video like the one you watched before completing voter registration? Why or why not?
6. If a government official asks you to leave all of your personal devices in a locker before entering the booth and then retrieve them after you leave the booth, would you be comfortable with this? *Yes/No*

Voting Channels. *We now ask voters about their perceptions on various voting channels.*

Please rate your level of trust (e.g., reliable and accurate counting and reporting of votes) with each of the following voting options from 1 “Not trustworthy at all” to 7 “Completely trustworthy” and explain your reasoning.

1. Voting in-person at a polling place, and filling out a paper ballot by hand.
2. Voting in-person at a polling place, and submitting your choices on an electronic kiosk which then prints a ballot for you to inspect before depositing the ballot in the ballot box.
3. Voting in-person at a polling place, and submitting your choices on an electronic kiosk.
4. Voting remotely by filling out a paper ballot, and sending it in the mail.
5. Voting online using an e-voting system that you also register for online.
6. Voting online using an e-voting system, and registering for it in-person using a process similar to the one you just experienced.

Coercion Scenario. *We finally end the survey asking voters about the perceptions and experiences with coercion.*

1. Which of the following scenarios do you think might plausibly happen (or have happened) to you or someone you know? Please rate the following scenarios from 1 “Not likely at all” to 7 “Extremely likely”
 - Someone - such as an authority figure, employer, or domestic partner - coercing voters to vote in a particular way, threatening harm if they do not.
 - Someone coming to voters’ homes offering them money if they register to vote by mail and give away their mail-in ballot when it arrives.
 - Someone on the Internet offering money to voters to record themselves voting a particular way, for example by taking a “ballot selfie”.
 - An e-voting application that sends voters money if they cast votes that the app instructs.
2. From whom do you think vote buying or coercion is most likely to happen (or have hap-

pened) to you or someone you know? Please rate the following scenarios from 1 "Not likely at all" to 7 "Extremely likely"

- A party operative pressures or coerces you to vote a particular way or buys your vote.
 - An authority (e.g., police, municipal workers) pressuring or coercing voters to cast a vote in a particular way or buys your vote.
 - A family member, such as a domestic partner, pressures or coerces you to vote a particular way or buys your vote.
 - An employer pressures or coerces you to vote a particular way or buys your vote.
3. For each of the above questions (1) and (2), we asked participants whether this item "Has happened to you or someone you know?"
- Yes, please describe
 - No
4. Any additional comments about this user study?

Statistical Methods

To establish statistical significance, we use an alpha level of 0.05. For our between-subjects comparison on non-parametric data (e.g., ordinals), we use the Kruskal-Wallis one-way ANOVA, while for pairwise comparisons, we use the Dunn test. For our within-subjects comparisons on non-parametric data, we use the Friedman rank sum test, while for pairwise comparisons, we use the Durbin-Conover test. To control for the family-wise error rate when conducting pairwise statistical comparisons, we use the Holm-Bonferroni method. We apply Shapiro-Wilk to assess whether the current data follows a normal distribution. We employ Cohen's *d* to measure effect sizes, which estimate the degree of difference between two groups. Typically, an effect size of about 0.2 is considered small, while an effect size near 0.8 is considered large. The term *participants* represent the average views or experiences of those involved in the study.

8.4 Study Setting

This section presents the execution of the study — its location, how we recruited participants and how participants completed the study.

To enhance demographic diversity while adopting a neutral environment, we conduct our study in a suburban park of Boston, Massachusetts in the United States. We verbally invited passerbys to participate in the study over three months, and showcased flyers/posters at our study location to encourage engagement. We limited participants to only those who had prior experience with voter registration.

Our setup (fig. 8.4) consists of a table with two chairs on one side for the participant and a park bench on the other for the facilitator. On the right side of the table, we have the credentialing process with the kiosk, the envelopes and the receipt printer while on the left side, we have



Figure 8.4: **Study Setup:** (1) Starting on the left chair, participant reviews information sheet and signs consent form, (2) watches instructional video on the laptop, (3) handed a check-in ticket from facilitator, (4) moves to the right chair and interacts with the kiosk to obtain their credential(s), (5) returns to the left chair for check-out, (6) activates their credential(s) and casts mock votes on the supplied mobile device, and, (7) completes the exit survey on either the laptop or a touchscreen tablet.

the check-in, check-out, activation and voting processes.

Whenever an eligible participant agrees to participate, we guide them to the left side of the table to review the information sheet and sign the consent form. The facilitator addresses any logistical questions, and refrains from discussing the objectives of the study beyond “assessing the usability of an online voting system.” The participant is then randomly assigned to one of five groups that we presented earlier in §8.1. To enhance the ecological validity of our study [59], the facilitator asks the participant to envision themselves at a government office, as scripted later in this section.

The facilitator then starts playing the appropriate instructional video for the participant to watch, the content of which depends on the participant’s assigned group. After the video concludes, the facilitator hands the participant a check-in ticket and directs them to the booth.

To simulate an authentic booth experience, the facilitator does not interact with the participant, intervening only upon request. Upon completing the credentialing process, the participant returns to the left side of the table and hands one credential (real or fake) to the facilitator. The facilitator then scans the QR code through the visible window and returns the credential.

For activation, the facilitator has the participant envision that they are now at home, and to use the study’s supplied mobile device to activate their real credential and cast a mock vote. Optionally, the participant may activate their fake credential(s) and cast fake votes by clicking on ‘cast vote with another credential’. However, we designed the app for the purpose of this study only. The app is stateless, supporting only one credential at a time and if asked, we

Chapter 8. Study Methodology

explain to participants that, in a real setting, one could unlock a specific credential using its associated PIN.

Once finished, the participant completes the exit survey and is then compensated \$20. The entire study typically takes around 30 minutes.

Facilitator's Scripts

Recruitment. Would you like to participate in a usability study on online voting? The study takes approximately 30 minutes and you will be compensated \$20 for your time. To be eligible, you must have registered to vote in the past. The whole process is fictitious, you would be voting on your favorite color or favorite sport.

Welcome. Thank you for your interest in our study. Online voting is also known as electronic voting or e-voting and enables voters to cast votes using their own device in the comfort of their own home. Here is the information sheet. We also have an instructional video for you to watch. At any time during or after the study, you may withdraw from participation by contacting us using the contact info on the information sheet. Please review the information sheet and sign the consent form if you agree to participate in the study. I will keep the original signed consent form. You may take a copy of the consent form or take a picture for your records. The information sheet is yours to keep.

Groups C & F & M Scenario Priming. Imagine that the year is 2030. Online voting is now commonplace. You are renewing your drivers license at a government office. You see a poster announcing that you can vote online. You are interested, so you watch an instructional video on how to register for online voting. After watching the video, you go to the clerk to get started.

Groups SF & SM Scenario Priming. Imagine that the year is 2030. Online voting is now commonplace. You are renewing your drivers license at a government office. You see a poster announcing that you can vote online. You are interested, so you watch an instructional video on how to register for online voting. You worry that hackers are trying to manipulate U.S. elections, but the video teaches you how to register securely and protect your vote from manipulation. After watching the video, you go to the clerk to get started. Being security conscious, you monitor the registration process carefully and report anything suspicious.

Check-In. Here is your check-in ticket, you may now enter the booth.

Check-Out. Please hand me [any one of] your credential[s] to perform the check-out process.

Activation. Here is your credential. Now imagine that you have left the government office and are at home. Please use this device as the device that you trust for online voting. For the purposes of this study, activate your real credential and then cast a vote in our mock elections. [You may then activate your test credentials if you wish]

Conclusion. That's it. In practice, you would be able to vote in all elections for the next 5-10

years on your device until you need to renew your identification documents, where you would repeat this signup process.

8.5 Preliminary Studies

We initially conducted two preliminary user studies involving 77 participants to gain insights on our design. The first preliminary study involved 41 PhD students, the second involved 36 individuals from varying locations across Boston, Massachusetts.

This led to two enhancements in TRIP. First, we replaced a QR code on the check-in ticket with a barcode, as participants often mistook it for the kiosk's first printed QR code used to create real credentials. Second, to remind participants to pick and scan an envelope after the first QR, we added a matching symbol on the receipt and envelopes, prompting the voter to select an envelope bearing the same symbol.

9 Study Results

9.1 Participants

This section details the demographic profile of our participants and their perceptions and experiences with coercion. Moreover, we also present their views on the various voting channels present in the survey.

9.1.1 Demographics

We recruited 150 participants, aged 19 to 83, with an average of 44 and a median age of 36.5. Figure 9.1 depicts this age distribution. Figure 9.2a provides a breakdown of the participants' age distribution across the five study groups, while Figure 9.2b provides a breakdown of their gender, ethnicity and education. We observed distinct participation patterns: seniors primarily during the day, middle-aged individuals after work, and younger individuals (18-35) throughout the day. During recruitment, the first impression of many individuals is that they could register with us for online voting; we clarified that this is only a usability study on online voting with mock, non-political, elections. Participants' time availability was the primary recruitment challenge; we estimated the study to last around 30 minutes, and found, after the fact, that a typical participant took 35 minutes. A few individuals explicitly declined to participate due to their opposition to online voting, demonstrating the typical difficulties in overcoming selection bias in user studies.

9.1.2 Perceptions & Experiences with Coercion

We now discuss participants' perspectives and experiences with coercion and vote buying based on their survey responses.

Participants rated the following four coercion scenarios on a 7-point Likert scale [70]: (C)oercing by threatening harm (C-Forceful), (P)urchasing absentee ballots (P-Ballot)¹, purchasing

¹Illustrated by the North Carolina ballot fraud incident [118].

Chapter 9. Study Results

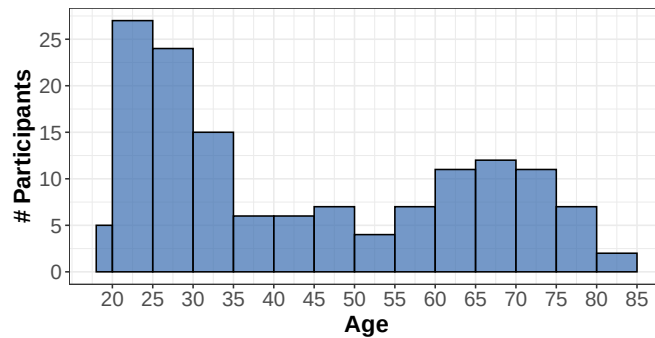
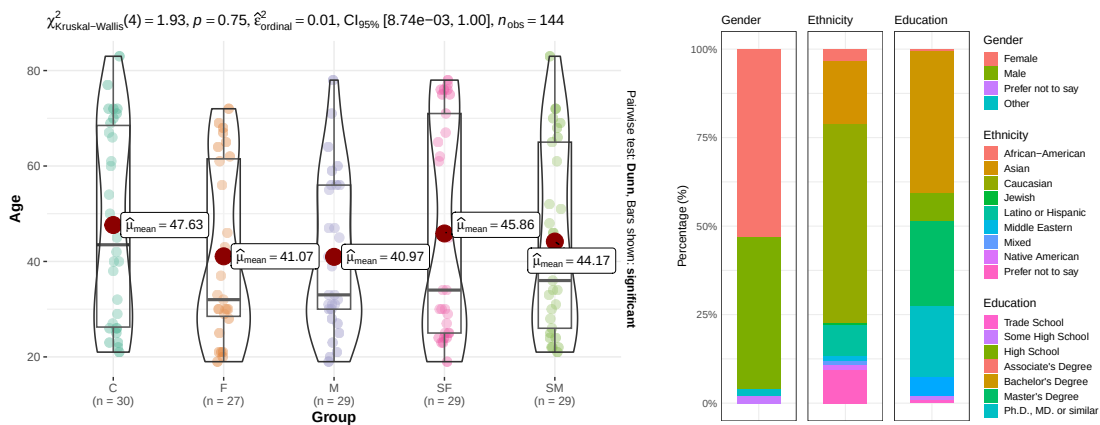


Figure 9.1: **Distribution of Participant Ages.** The minimum, median, mean and maximum ages among all groups were 19, 36.5, 44 and 83, respectively. Six participants did not disclose their age. Visualization created using ggstatsplot [89].

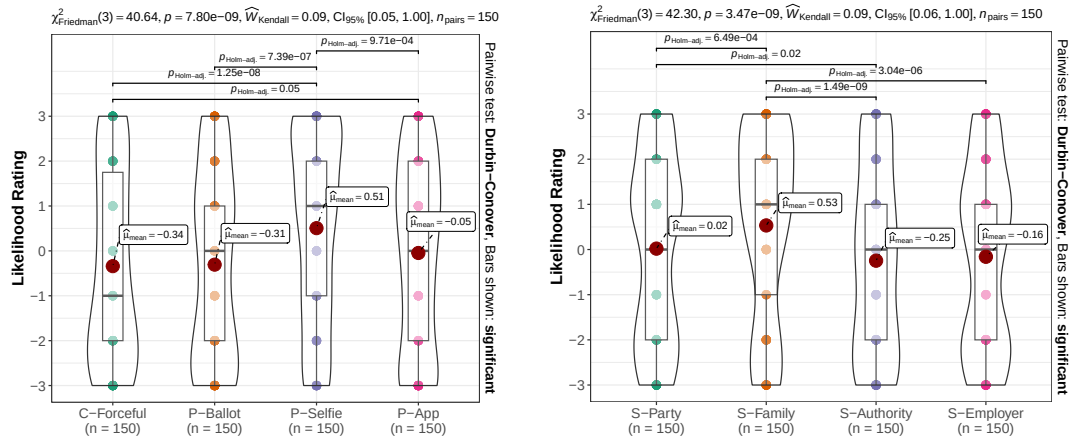


(a) **Age Across Groups.** The age distribution of participants across groups. Visualization created using ggstatsplot [89] (b) **Gender, Ethnicity and Education.** The distribution of participants, categorized by gender, ethnicity and education.

Figure 9.2: **Participant Demographics**

proof such as a voter taking a selfie with their ballot containing the coercer-dictated choices (P-Selfie), and an app offering compensation for voting as directed (P-App). They also rated the following four potential sources of coercion on the same scale: a party operative (S-Party), an authority figure (S-Authority), a family member (S-Family), and an employer (S-Employer). Ratings are then converted to a score between -3 to 3, where -3 is 'completely unlikely' and 3 is 'completely likely'.

Among the coercion scenarios, participants find P-Selfie as the most likelihood scenario with a mean score of 0.51 and a median of 1 (somewhat likely). A quarter (24%) of participants rated P-Selfie as extremely likely, with this scenario being perceived as having a statistically significantly higher likelihood compared to the other three coercion scenarios. The calculated Cohen's d effect sizes further quantify these differences: 0.41 when comparing P-Selfie with



(a) **Coercion Scenarios.** Participants perception of likely coercion scenarios; ballot selfies received the highest average likelihood, while forceful coercion received the lowest.

(b) **Coercion Sources.** Participants perception of likely sources of coercion; family members received the highest average likelihood rating, while authority figures received the lowest.

Figure 9.3: Participants' Perceptions of Coercion Scenarios, and Coercion Sources

C-Forceful, 0.39 with P-Ballot and 0.26 with P-App. In contrast, participants perceived C-Forceful as the least likely scenario with a mean score of -0.34 and a median of -1 (somewhat unlikely). We also see that the distribution of responses for C-Forceful almost inversely mirrors that of P-Selfie. The remaining two coercion scenarios, P-Ballot and P-App also bear a visual similarity in their distributions, demonstrating two distinct peaks at somewhat likely and extremely unlikely.

Perceived Sources of Coercion

Participants perceive S-Family to be the most likely source of coercion (mean score of 0.53 and a median rating of somewhat likely), with 21% of participants rating it as extremely likely. Moreover, S-Family is statistically significantly higher than the three other coercion sources. The calculated Cohen's d effect sizes are: 0.38 when comparing S-Family with S-Authority, 0.34 with S-Employer, and 0.25 with S-Party. In contrast, S-Authority is the least likely source with a mean score of -0.25. For both S-Employer and S-Party sources, participants have two dominant contrasting opinions, with one cohort seeing the source as somewhat likely and the other as very unlikely.

Instances of Coercion

A quarter of participants (26%) report experiencing or know of someone who has experienced at least one form of voter coercion, while two-thirds (67%) of participants report no such experiences; the remaining 7% of participants preferred not to say. Following on the most likely source of coercion, the most prevalent source of coercion is S-Family, as reported

Chapter 9. Study Results

Scenario	Happened	Not Happened
C-Forceful	12%	81%
P-Ballot	8%	85%
P-Selfie	6%	86%
P-App	3%	88%
Total Count	44	511
Unique Count	26	112
Source		
S-Party	10%	81%
S-Family	15%	77%
S-Authority	7%	85%
S-Employer	11%	83%
Total Count	63	491
Unique Count	31	108
Unique Participants	39	98

Figure 9.4: **Participants’ Reported Instances with Coercion Scenarios, and Coercion Sources.** Participants state whether these items have happened to them or someone they know. The remaining participants chose not to disclose.

by 15% of participants. Instances of such coercion include multiple accounts of spousal oversight during voting, leading to dictated voting choices. Despite being viewed as the least likely coercion scenario, 12% of participants report S-Forceful, making it the most reported coercion scenario. Around 10% of participants identify S-Employer and S-Party as sources of coercion. Such incidents include unions dictating votes, co-workers pressuring attendance at undesired political rallies, and prominent party members pressuring to vote along party lines.

Discussion

The substantial reporting rate of recognized coercion instances at 26%, with family members cited as the predominant coercion source (15%), highlights the need for coercion-resistant strategies to counter *persistent* adversarial oversight. In such circumstances, the practice of deniable re-voting in use in Estonia [64] falls short. Family members, typically having substantial unrestricted access, can potentially cast a vote on behalf of the targeted relative just before the election concludes. They can also choose to stay with the relative or retain their device/voting credential until the end of the voting process.

9.1.3 Trust Rating in Voting Channels

We now look at the participants’ trust rating of voting channels.

Most Trusted. Participants express the most trust towards I-BMD, with 69% deeming it as highly trustworthy or higher, and assigning it an average trust score of 1.68 out of 3 (table 9.1). Moreover, I-BMD is statistically significantly higher than that of all other voting channels,

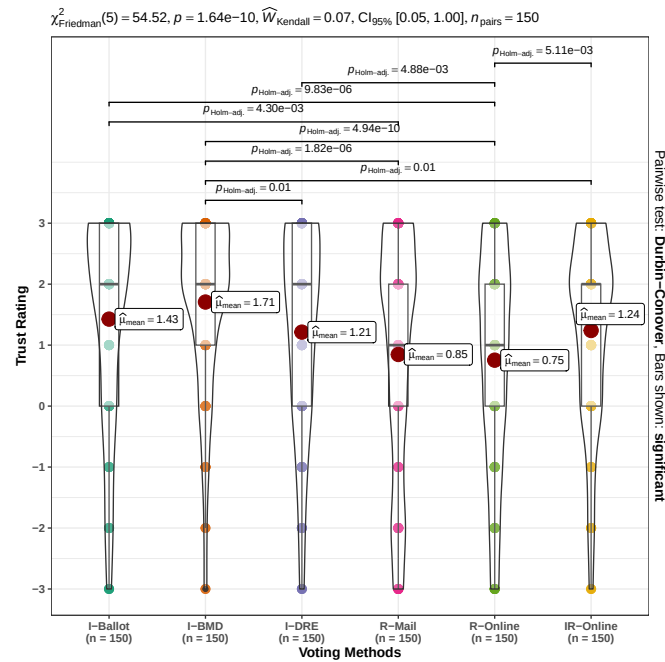


Figure 9.5: **Voting Methods Trust Rating from All Participants.** All 150 participants' trust ratings across voting methods.

except for I-Ballot (fig. 9.6), which earns the same level of trust from 60% of participants and an average trust score of 1.38.

Least Trusted. Participants express the least trust towards R-Mail, with only 46% of participants considering this system as highly trustworthy or higher (mean score of 0.72). Participants also view R-Online similarly, yielding similar mistrust (mean score of 0.85).

All Participants. Figure 9.5 depicts the trust ratings across voting channels from all 150 participants, along with our statistical results. IR-Online was not statistically significantly higher than R-Online, but it is now: participants consider in-person registration critical for online voting.

Discussion. In the U.S. 46% of voters cast an absentee ballot in the 2020 presidential election [9]. Despite this, mail-in voting has the second lowest trust score (fig. 9.5) and even the lowest trust score (fig. 9.6 & fig. 9.7) in our study. Participants lack confidence in the U.S. postal service's capacity to reliably deliver mail.

Despite concerns with BMDs [116, 8], including a study finding that only 6.6% of participants can detect a *manipulated* printed ballot [20], I-BMD received the highest trust score across voting channels.

By introducing the control group (fig. 9.5), we find that IR-Online becomes statistically significantly higher than R-Online. This might suggest that voters find a system more trustworthy

Chapter 9. Study Results

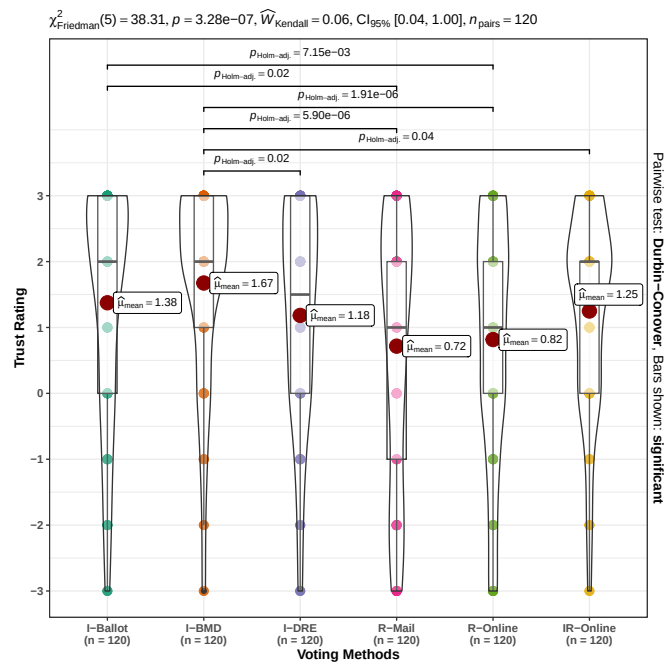


Figure 9.6: **Voting Methods Trust Rating from Participants Exposed to Fake Credentials.** The trust ratings for various voting methods, as given by the 120 participants exposed to fake credentials.

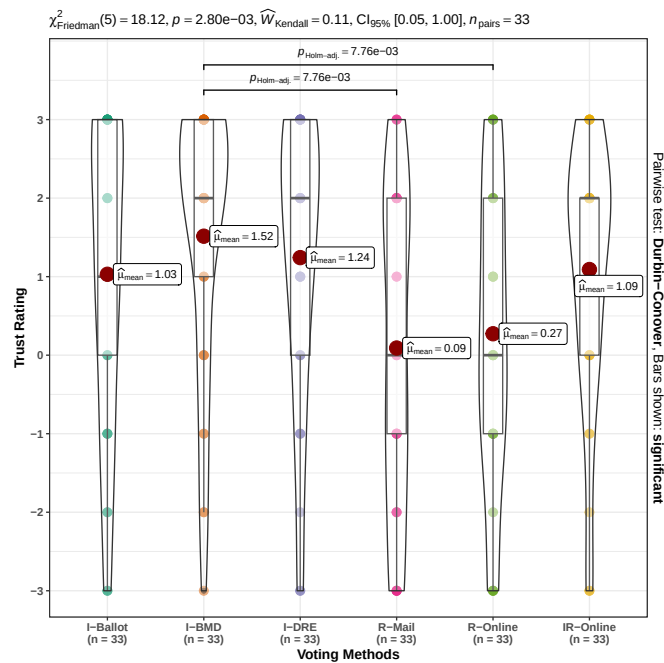


Figure 9.7: **Voting Methods from Coercion-Happened Participants.** The trust ratings for various voting methods, as given by the 39 participants who experienced coercion or know someone who has experienced coercion.

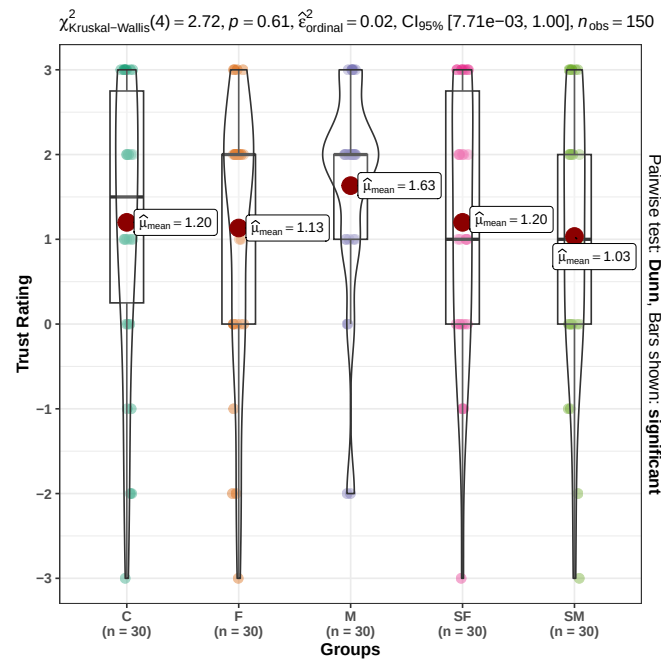


Figure 9.8: **IR-Online Voting Method Across Groups.** The participants' trust ratings across groups for the online voting with in-person registration voting method.

Figure 9.9: **Voting Methods.**

that requires some sort of in-person registration. Some participants also asked whether this system would require individuals to prove their eligibility to vote since as part of the study we did not ask for ID to respect participants' privacy.

We now investigate the level of trust the 120 participants in treatment groups place our coercion-resistant system versus other voting methods. In the exit survey, participants rate their level of trust on the following voting methods on a 7 point Likert Scale (scores from -3 to 3): (I)n-person voting with hand-marked paper ballot (I-Ballot), in-person voting with a ballot marking device (I-BMD), in-person voting with a direct electronic device (I-DRE), (R)emote voting via mail-in ballot (R-Mail), a fully remote voting system where both voter registration and voting are online (R-Online), and the online voting system they just experienced with in-person voter registration (IR-Online). Table 9.1 presents our summarized results and Figure 9.9 contains our statistical results.

9.2 Usability Results

9.2.1 Qualitative Results

We present the results from participants' single-word summaries and the system aspects they liked or disliked the most. For the single-word summaries, we classified each word based on

Chapter 9. Study Results

Voting Channel	I-Ballot	I-BMD	I-DRE	R-Mail	R-Online	IR-Online
Mean	1.38	1.68	1.18	0.72	0.82	1.25
Median	2	2	1.5	1	1	2
Trust Rating	Participants (%)					
Trusted (3)	35	36	26	22	20	20
Highly	25	33	24	24	22	35
Somewhat	12	11	18	15	17	16
Neutral (0)	12	10	17	12	18	17
Somewhat	7	5	7	7	9	5
Highly	6	3	4	12	6	4
Untrusted (-3)	3	3	3	8	7	3
Voting Channel	Pairwise Statistical Outcomes (fig. 9.6)					
I-Ballot		×	×	↑	↑	×
I-BMD	×		↑	↑	↑	↑
I-DRE	×	↓		×	×	×
R-Mail	↓	↓	×		×	×
R-Online	↓	↓	×	×		×
IR-Online	×	↓	×	×	×	

Table 9.1: **Voting Channels.** This table presents the trust ratings for various voting channels, as given by the 120 participants exposed to fake credentials, and a summary of the statistical outcomes (fig. 9.6). × means no statistical difference, ↓ means statistically significantly lower, and ↑ means statistically significantly higher.

sentiment, marking them as positive, neutral or negative. Table 9.2 shows the terms mentioned more than once by participants. We also devised categories that best encapsulate the items expressed and split them into two groups: most liked and most disliked (Table 9.3).

We observe that the control group perceived the system most positively. As the level of engagement increased, subsequent groups showed a consistent decline in positive ratings. Meanwhile, neutral and negative ratings vary between 13 to 27%, with the exception of group M (a 37% negative rating). This surge partly derives from participants' suspicion of the kiosk's unusual behavior, expressing their surprise with words like “hacked” and “suspect”—terms absent in other groups' feedback, including SM. Moreover, while participants in group M has the least number of participants valuing security among the treatment groups, SM records the highest. Despite these observations, we cannot confirm statistical differences between the groups and the ratings (Fisher's Exact Test, $p = 0.24$).

We find a roughly equal divide between those who find the system easy to use and those who consider it complex. We also observe a roughly equal split between those who appreciated the system's instructional guidance and those who disliked the handling (e.g., scanning, storage) of paper credentials. We also find that given the lack of remote voting in the study location, 15% of participants appreciate the ability to vote remotely. A minority (4%) expressed dissatisfaction with the need for in-person registration. Finally, we observe that participants in the treatment

easy	positive	17	new	neutral	3
interesting	positive	13	streamlined	positive	3
complicated	negative	8	long	negative	2
good	positive	7	fine	neutral	2
simple	positive	7	fast	positive	2
straightforward	positive	4	complex	negative	2
great	positive	4	convenient	positive	2
smooth	positive	4	cumbersome	negative	2
confusing	negative	4	efficient	positive	2

Table 9.2: **Distribution of Single-Word Summaries by Sentiment.** This table lists the words used more than once by participants to describe their impressions of the system along with each word's associated sentiment (positive, neutral or negative).

groups valued the system's security at least threefold compared to the control group, with half of these comments praising fake credentials.

9.2.2 Use Errors

Process-Induced Use Errors

During the study, we record observational notes for each participant, including use errors and facilitator interventions. We focus on use errors fundamental to the credentialing process, while we discuss interface and device-induced use errors in §9.2.2. We classify use errors into two categories: Mistakes and Violations. We characterize mistakes when participants' plan or intended action is flawed, typically resulting from misinterpretation of instructions. In contrast, violations arise when participants intentionally disregard or skip instructions. We now present a discussion of process-induced use errors we observed, as reported in Table 9.3.

Credentialing. During the real credential creation process with an honest kiosk (C, F, SF), 5 participants (6%) prematurely tore the receipt after the kiosk printed the first QR code—a mistake that required us to intervene during activation. An additional 8 participants (9%) attempted to tear the receipt but when they encountered resistance from the receipt printer, they rectified their mistake by consulting the kiosk display guiding them to pick and scan an envelope. In terms of envelope selection, 5 participants (6%) initially opted for an envelope that did not correspond with the symbol on the receipt; the kiosk alerted them to this mistake and they all successfully scanned a correct envelope on their second attempt. A single participant misinterpreted the discard check-in screen after creating their real credential and mistakenly threw away their credential.

Activation. The majority of use errors occurred during the activation phase, post “leaving the government office”, with approximately 19% of participants encountering difficulties and 8% requiring facilitator assistance. Participants commonly skipped the instructional screen, a

Chapter 9. Study Results

violation error. Participants therefore removed the entire receipt from the envelope when the device prompted to scan three QR codes. These participants likely associated the “three QR codes” to be the three QR codes on the receipt rather than the two QR codes from the receipt and one QR code from the envelope. For those participants who read the instructions yet still faced challenges, mistakes often involved scanning only a subset of QR codes at once instead of all three simultaneously, and placing the credential on the device’s screen instead of on the table to be scanned by the device’s camera. Considering these activation errors, it naturally follows that credential handling emerged as the second most disliked aspect of the system among participants.

Success Rate. The initial success rate stands at 87%, with 19 participants (13%) needing assistance. Upon closer examination, registration has a 95% success rate, while activation has a 92%. However, including participants who mistakenly used their fake credential to cast their vote drops the final success rate to 83%, a detail we present in §9.3.

Statistical Analysis. We categorized participants into four age groups: 18-30, 31-45, 46-65, and 65+. Analysis of use errors across these age groups using the chi-squared test revealed no statistically significant differences ($p = 0.8791$). Further examination employing logistic regression to assess the interaction effects between age groups, ethnicity, education, and gender on use errors also indicated no statistically significant associations as determined by the Wald test.

System Improvements. We now explore strategies to improve this rate. For torn receipts, we suggest integrating a receipt detection mechanism, enabling the kiosk to restart the real credential creation process. To help address the activation issues, the device should store read QR codes incrementally, compared to statically processing each image for three impeccable QR codes. Moreover, redisplaying the activation instructional screen in response to unsuccessful attempts may help, particularly when the participant initially skipped over these instructions. Introducing an animation showing the use of the device’s rear-facing camera for credential scanning could also prove beneficial.

Study Comparisons We compare our success rate with three usability studies containing seven variants of voting systems: the only other JCJ-style coercion-resistant voting study we are aware of by Neto et al. [81], a study by Acemyan et al. [4] on three voter-verifiable systems (Helios [7], Prêt à Voter [102], Scantegrity II [24]), and a recent study on STAR-Vote [5], a secure, auditable and transparent ballot marking device for in-person voting. The Neto et al. [81] between-subjects study involved 80 university-affiliated individuals aged 18-39 acquiring their real credential using one of the following three variants and then casting both real and fake votes: (1) in-person acquiring a pen drive, (2) remotely via email, and (3) in-person with a password set by the voter. The success rate for the first two variants is 100% and the last one is 85%. Acemyan et al. [4] conducted a within-subjects study with 37 diverse participants on Helios, Prêt à Voter and Scantegrity II where Helios is an voter-verifiable online voting system while Prêt à Voter and Scantegrity II are coercion-resistant, voter-verifiable, in-person voting

systems. The success rate for Helios is 60%, Prêt à Voter is 60%, and Scantegrity II is 50%. The study on STAR-Vote [5] reported a success rate of 93%.

Discussion. TRIP outperforms three out of the seven variants, including prominent coercion-resistant, voter-verifiable, in-person voting systems Prêt à Voter and Scantegrity II, which TRIP most closely resembles as a coercion-resistant, voter-verifiable, in-person registration system. Despite the remaining four variants surpassing TRIP, they either fall short in achieving one or more of these properties—illustrating the challenge in designing a usable coercion-resistant system—or have a homogeneous study population. TRIP substantially improves this success rate to 83%, up from 60% (Prêt à Voter) and 50% (Scantegrity II), and closer to the 93% success rate that STAR-Vote achieves.

To our knowledge, NIST has established standards for electronic voting [86], but no official success rate exists. Only a NIST internal document [85] recommends a success rate of 98% (known as Total Completion Score) but where 98% only needs to fall within the 95% confidence interval.

Interface-Induced Use Errors

This section discusses device- or interface-induced errors, which include premature scanning of check-in tickets, unawareness to answer the quiz, and unfamiliarity with the device's camera.

Preemptive Scanning. Out of the 150 participants, 56 preemptively scanned their check-in ticket before the kiosk prompted them to do so. The premature action occurred 51 times in groups C, F and SF. Fortunately, 47 (84%) corrected their mistake without our intervention upon recognizing the correct intended action, such as pressing “Continue” or “Begin”. No participants in groups M and SM required our intervention. This pattern is mostly observed in groups C, F and SF due to an instructional slide, for which groups M and SM were not privy due to the kiosk's malicious setting. Upon viewing this instructional slide's four steps, starting with “Scan Check-in ticket”, participants scanned their ticket instead of pressing “Begin” to initiate the process. When we were requested to intervene, we guided participants towards the intended action, typically by pressing “Begin”, although 84% of participants did not require us to intervene. We hypothesize that these errors result from a mix of environmental factors such as inadequate contrast between the button and the screen in an outside environment, alongside behavioral factors. Examples of behavioral factors include participants scanning their check-in ticket prior to completely reading the instruction slide, owing likely to their prior exposure to an instructional video. To address these issues, we propose permitting a combination of inputs. For instance, interpreting the scanning of the check-in ticket during an instructional slide as an indication that the participant is prepared for subsequent steps.

Mobile Device-induced. An additional 12 participants had initial difficulty with activating their credential as they either placed the QR codes directly on the device's screen or used the

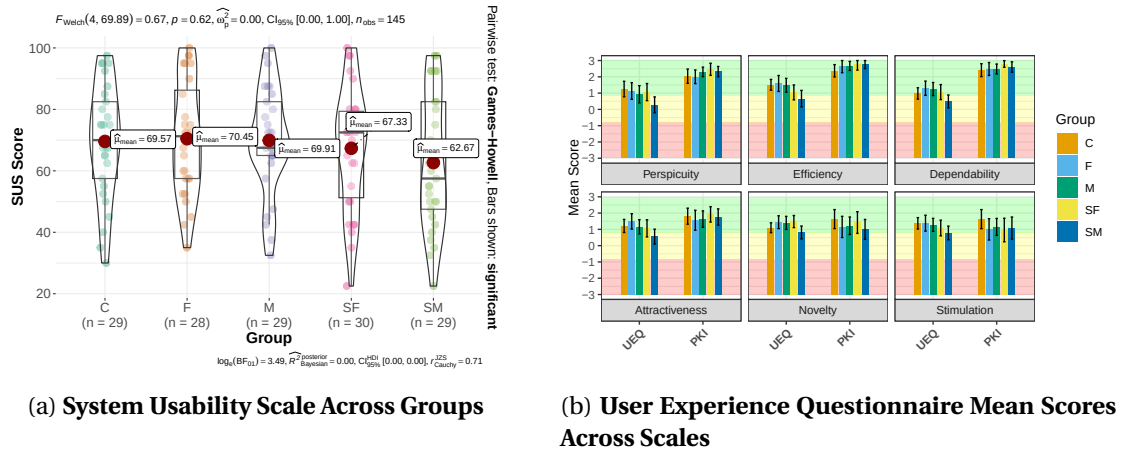


Figure 9.10: Usability Metrics

front-facing camera. In each instance, we intervened to instruct the participant to use the back-facing camera. Participants who continued to need assistance beyond this guidance are reported in the main body of the paper.

Kiosk-Induced. Three participants faced technical difficulties with the kiosk device, necessitating our intervention. Two of these instances arose from participants not realizing they had to complete a quiz before proceeding, while one participant had trouble locating the QR code scanner.

9.2.3 Perceived Usability and User Experience

We now present the system usability scale and the user experience questionnaire scores and compare them to benchmarks.

System Usability Scale. In Figure 9.10a, we present the System Usability Scale (SUS) scores across our study groups, while removing 5 participants due to inconsistent answers (agreeing to most positive and negative items), as suggested by Sauro [106]. We observe that groups C, F, and M demonstrate similar average and median scores while groups SF and SM exhibit lower average scores, with group SM having a score decrease of 10% over the control group. Despite these observations, Welch’s one-way ANOVA reveals no statistically significant differences ($p = 0.62$). Nonetheless, we can compare our SUS mean scores with a benchmark consisting of 446 studies involving a range of products and services, from business/consumer software to hardware devices [106]. These 446 studies reveal an average SUS score of 68, with standard deviation of 12.5. Groups C, F, and M achieve marginally superior SUS scores, ranking in the 55.1st, 57.8th and 56.1st percentiles, respectively. Conversely, groups SF and SM underperform with scores ranking in the 47.8th and 35.0th percentiles. Some works such as Bangor et al. [11] have come up with adjective ratings based on the SUS score, and they would classify group F—the expected normal group in reality—as “Acceptable,” and achieving a “Good” adjective

rating.

SUS: Study Comparisons. We now compare group F's SUS score with Neto et al. and Acemyan et al.'s study [4, 81]. While Neto et al.'s variant 2 and 3 achieve a higher mean SUS score of 77.5 (81st percentile) and 77.4 (81st percentile), respectively, these were not statistically significantly higher than group F's score (Welch one-sided t-Test, $p = 0.094$ and $p = 0.051$). Neto et al., unfortunately, did not administer the SUS questionnaire to participants for variant 1, which is the most comparable to TRIP that involves the in-person delivery of voter materials. From Acemyan et al.'s [4] study², Group F's mean SUS score of 70.4 was statistically significantly higher than Prêt à Voter's score of 61 (Welch one-sided t-test, $p < 0.05$), and Scantegrity II's score of 59 (Welch one-sided t-test, $p < 0.05$) but lacked conclusive statistical difference of Helios' score of 76 (Welch two-sided t-test, $p = 0.17$).

SUS: Discussion. The comparable percentile scores and the absence of statistical difference between Group C and F & M suggest that the introduction of fake credentials do not change participants' perceptions of usability. Further, even with voters engaging in a 4-step protocol, F achieved an Acceptable rating and achieved statistically significantly higher SUS scores than Scantegrity II and Prêt à Voter.

User Experience Questionnaire. Figure 9.10b and Table 9.4 illustrate the UEQ scores for each scale (Attractiveness, Dependability, Efficiency, Novelty, Perspicuity, and Stimulation), along with the UEQ Key Performance Indicator (KPI) scores, which represent participants' perception of an ideal registration experience. Traditional usability aspects encompass Efficiency, Perspicuity, and Dependability, while Novelty and Stimulation relate to user experience. Upon comparison, participants view this system as slightly surpassing their expectations in Novelty, Stimulation, and Attractiveness. However, it falls short in the traditional usability aspects by approximately a full point. Nonetheless, scores above -2 and 2 are extremely rare due to differing opinions and answer tendencies [110] (e.g., avoidance of extreme responses). Typically, a positive evaluation is a score above 0.8, neutral evaluation is a score between -0.8 and 0.8, and negative evaluation is a score less than -0.8 [110]. Based on this metric, Group F has a positive evaluation for each of the scales.

UEQ: Benchmark. Similar to SUS, to better assess usability, we need to compare the UEQ scores with other studies. We first compare our scores to the UEQ benchmark [110], consisting of 21,175 participants from 468 diverse studies. While group F's scores for efficiency, stimulation and novelty rank in the top quartile, the perspicuity score are between the 50th and 75th percentiles. This suggests that despite participants finding it more challenging to familiarize with this system compared to the benchmark, they still complete their tasks without excess effort, while also perceiving the system as both engaging and innovative.

UEQ: Study Comparisons. We now compare our scores with other voting-related studies, although the number of studies that administer UEQ is substantially more limited. However,

²These numbers are not present in the text of their work [4]; we extract these numbers from their figure 4 and assume a 95% confidence interval.

one comparable study conducted by Marky et al. [75] focuses on voter-verifiability—though no coercion-resistance—through the use of code voting [103] as prominently used in the Swiss Online Voting System [117]. This approach provide voters with physical materials, typically by mail, to cast a vote in a way that prevents potential vote alteration by the voter’s device. Marky et al. tests three voting code variants—manual codes, QR codes, and tangibles—with 18 participants. Our interest lies in the QR-codes variant, as it is most similar to TRIP, and is also the most favored option in Marky et al.’s study.

Unlike in TRIP, their scores for user experience (stimulation and novelty) are neutral although their scores for the traditional usability aspects (perspicuity, efficiency and dependability) are all positive. Marky et al.’s study also achieves higher perspicuity and efficiency scores, 2.2 vs. 1.13, and 2 vs. 1.59, respectively, while TRIP achieves a marginally higher dependability score 1.3 vs. 1.2.

9.2.4 Malicious Kiosk

We present our findings from participants exposed to our malicious kiosk (groups M & SM). We assess their ability to identify the kiosk’s misbehavior during their interaction with the kiosk and via a survey question probing if they detected any irregularities while creating their real credential (part III). We administered the same survey question to groups F & SF as a control to identify false positives rates. Table 9.3 presents the reporting rates.

Kiosk Reporting Rate. Among group M participants, 10% reported the kiosk’s misbehavior to the facilitator, whereas this rate significantly increased to 47% for group SM. Further, we observe the reporting rates as statistically significant different (Chi-squared, $\chi^2 = 8.2079$, $p < 0.01$; Cramér’s $V = 0.4068^3$), indicating that instructional video 3, shown to group SM, substantially improved the rate of reporting.

Survey Reporting Rate. A greater percentage of participants reported the kiosk’s misbehavior in the exit survey: 20% for group M and 57% for group SM. We still observe a statistically significant difference in the reporting rates between groups M and SM (Chi-squared test, $\chi^2 = 7.0505$, $p < 0.01$; Cramér’s $V = 0.3771$). Unlike the kiosk reporting rate, however, the survey reporting rate was non-zero for groups C, F, and SF, averaging at 8%; these participants primarily cited confusion about the process.

Real-World Scenario. According to participants’ responses to our survey questions, 85% are willing to watch an instructional video before participating in voter registration in a real world context. However, only 59% indicate their willingness to “lock up” their personal devices in a locker before entering the booth and retrieve them afterwards.

Study Comparison. Bernhard et al. [20] conducted a study among diverse participants to examining the rate at which voters could detect malicious ballot manipulation from ballot

³For Cramér’s V , values around 0.1 are typically considered small, around 0.3 indicate a moderate effect size, and values of 0.5 or greater signify a strong effect.

marking devices (BMDs). Both our and Bernhard et al.'s studies involved participants operating under the election authority's supervision, interacting directly with an official device: the BMD in their case and the kiosk in ours. In both studies, we assess whether participants can visually discern anomalous behavior from these devices. In their study, without any guidance, only 6.6% of 31 participants reported the error to the facilitator. When participants were asked before submission if they had carefully reviewed their ballot, 12.9% of 31 reported the error to the facilitator. In contrast, the reporting rate for TRIP is 10% without security priming, and 47% with priming.

9.3 Usability of Fake Credentials

This section evaluates the comprehension and usability of fake credentials among the 120 participants exposed to them.

9.3.1 Understanding Fake Credentials

We examine voter comprehension of fake credentials in two distinct stages: during interaction with the kiosk with a pop quiz and during the survey. This dual-stage assessment measures understanding of passive instruction and efficacy of active instruction. The quiz poses the question "Which of the following purposes can you use a test credential for?" (§8.3). Not selecting "To cast a vote that counts in election" underscores the crucial understanding that fake credentials cannot be used to cast a real vote; the remaining three correct options must be concurrently selected to pass the quiz.

Quiz Results: Test Credential Usage. We find that 70% of participants answer the quiz correctly on their first attempt (table 9.5). An additional 15% succeed on the second attempt, including 3 who initially chose the incorrect answer. The third and final attempt sees the remaining 15% (18 participants), split as follows: 4 participants answer correctly, 7 fail to select all correct options simultaneously, and 7 choose the incorrect option. In this initial stage, 92% of participants do not select the incorrect option in all attempts, thereby demonstrating an understanding of fake credentials.

Exit Survey. Among the 8% (3 from second attempt + 7 from third attempt) of participants who incorrectly selected "To cast a vote that counts in an election" in the quiz, five of them correctly identified the use of fake credentials in the survey. In the remaining five, four could not recall the purpose of fake credentials, and one only wrote "voting". We infer that 4% (5 participants) likely finished the study without a clear understanding of fake credentials' role, with one participant thinking that fake credentials can be used to cast real votes.

Among the participants who did not select the incorrect option, 89% remembered, in their free-text responses, the use of fake credentials to resist coercion. The remaining 11% wrote about their use to educate others, test the system, and even generate a profit by selling their

fake credentials.

9.3.2 Usability of Real and Fake Credentials

Considering that both real and fake credentials are used the success of using real or fake credentials ultimately hinges on the voter's ability to distinguish between the two. In the study, out of the 120 participants exposed to fake credentials, 92 choose to create one or more fake credentials.

Quiz Results: Credential Distinguish. We initially verify participants' understanding on how to distinguish their credentials by administering the Credential Distinguish quiz (§8.3). Remarkably, 99% select the correct option "Only myself with my pen markings" on their first attempt, thereby showcasing their comprehension (table 9.5).

Distinguishing Credentials. We assess participants' ability to distinguish their credentials by directing them to cast a vote using their real credential after completing voter registration. Among the 92 participants, 90% (83) accurately identified their real credential. For the 10% (9 participants) who failed to do so, we explore the reasons behind it.

One participant discarded their real credential during credentialing, as mentioned in §9.2.2. With respect to the remaining eight participants, we examined each set of credentials they created to ascertain whether the real credential had distinct markings; our findings confirm that this was the case. Misinterpretation of instructions or environmental distractions may then contribute to some of this error. For details about how study participants generally marked their credentials, see Appendix 9.3.3. Nonetheless, to gain deeper insights into individuals' recall ability for sensitive data, we consider password-related studies [22, 32], which indicate a wide range of retention rates from 23% to 98%. With a 90% success rate, our rate lies not only in the upper end of this spectrum but is same rate as in *Déjà Vu* [32], a study conducted on using images for authentication. This is anticipated outcome as voters marking their own credentials incorporate several known retention-enhancing strategies: user-generated content and favoring visual imagery over text. However, we highlight the need for a long-term study to evaluate memory recall over extended periods. Nonetheless, the *Déjà Vu* study provides encouraging evidence that imagery-based memory degradation is significantly lower than that of PIN/passwords. After one week, only 1 out of 20 participants failed to login with images while 7 to 6 individuals failed to login with PIN & passwords, respectively. Voters who do forget which is their real credential have two options: "reset their password" by re-registering to obtain a fresh set of credentials, or cast the same vote using all of their credentials. Social recovery [108], where voters could recover their real credential from a set of trustees—friends or family members, may also be possible.

Confidence in Distinguishing Credentials. Participants also rated how confident they were in recalling their real credential on a 7 point scale, where 1 is "No confidence" and 7 is "Extremely confident". 87% of participants gave a confidence rating of 5 or more, with 55% of participants

giving the extremely confident rating. 7% rated a 4 (neutral), and ratings 1 & 2 each got one participant.

Fake Credentials In Reality? We ask participants about their willingness to create fake credentials if such a system existed; 53% of participants affirm they would. Testimonies vary from “because I have been in situations where others forced me to vote”, “to argue less with people voting for another candidate” to “[...] a fun souvenir”, and “I’m in a demographic where I cannot imagine having someone trying to solicit my vote, if they did, I would simply tell them no without fear.” Additional testimonies, including a taxonomy of these testimonies, are available in Appendix 9.3.4.

Discussion. Despite introducing an unforeseen and unprecedented concept to 120 participants, only 4% (5 participants) may have finished the study without a complete grasp of the use of fake credentials. Furthermore, although voters were not required to generate fake credentials, 76% of participant chose to create at least one. This proactive engagement is further expressed by 53% of participants who are willing to create fake credentials in a real-world context. Lastly, in spite of the identical nature of fake and real credentials, 90% of participants successfully distinguished between the two when aiming to activate and cast a real vote, on par with a study of using using images for authentication.

9.3.3 Credential Markings

We examine how participants mark their credentials to distinguish their real credentials from their fake ones. We analyzed and categorized the set of credentials for 88 participants (four sets were unaccounted for), first by looking at the type of marking and then the level of differentiation based on the patterns across credentials. We find six general categories for marking types:

- Envelope Symbol: No pen markings.
- Number: Random numbers
- Scribble: Indecipherable writings, including signatures.
- Symbols: Predominantly shapes such as stars or squares but can include smiley faces or animal drawings.
- Text: Written text (e.g., “Real”, “Test”), or participants’ initials.
- Type Change: Alternation between two or more of the above categories across the real and test credentials.

Regarding differentiation level, we looked at each set to determine if the real credential could be explicitly (e.g., “Real” vs “Test”) or implicitly (e.g., “Checkmark” vs. “X”) differentiated from the test credential(s), or if no distinguishable pattern (e.g., Star / Circle shapes) was exhibited.

Our findings, detailed in Table 9.6, show that 49% of participants used *Text* to differentiate their credentials, followed by *Type Change* at 18%. Furthermore, 42% of participants marked their credentials indistinguishably, making it impossible to differentiate the real credential

Chapter 9. Study Results

from the fake ones. Participants may lie about their real credential by marking their real one as “Fake” and their fake one as “Real” but participants in our study were not influenced to do so.

9.3.4 Fake Credentials in Reality?

We examine whether participants exposed to fake credentials would be willing to create fake credentials alongside their real ones if such a system existed in the real world. We found that 53% would do so, citing the following reasons for creating a fake credential:

- Security (27 participants): “Because I have been in situations where others forced me to vote”, “On the off chance someone tries to force me to vote in a particular way”.
- Convenience (15 participants): “no harm in creating a test credential”, “might come in handy”, and “to argue less with people voting for another candidate”.
- Education (7 participants): “It might be useful to show my students what the process was like”, “To see that it worked and educate family and friends”, and “It would be good for teaching people how to vote. Also might be a fun souvenir”.
- Other (12 participants): “I would want to create as many test credentials as possible before the market became flooded with test credentials”, and “I’d like to test my device setup”.
- No Reason Given (2 participants)

The following were reasons for not creating a test credential:

- Unnecessary (39 participants): “I’m in a demographic where I cannot imagine having someone trying to solicit my vote, if they did, I would simply tell them no without fear,” and “Not interested in the uses.”
- Cumbersome or Confusing (11 participants): “I don’t want to take the risk of being confused between my real credentials and the fake one. I can use videos or websites to teach someone else how to vote,” and “More to lose, and I would mix them up.”
- No Reason Given (7 participants).

9.4 Key Takeaways and Limitations

This section primarily offers key takeaways of our findings, building on the many results we have presented earlier. We also expand on our study’s limitations.

The Need For Coercion-Resistance. Our study reveals that a quarter of participants have either personally experienced coercion, or is aware of someone who has. These findings highlight the necessity of coercion-resistance to uphold free and fair elections. In essence, votes should not just be private but incorruptible. This importance is further reflected with these same participants rating (fig. 9.7) statistically significantly lower trust levels for both mail-in and online voting methods compared to in-person voting (with BMD).

Usability of Coercion-Resistant Voting Systems. Designing a usable coercion-resistant voting system is a difficult task. As seen in §9.2.2, voting systems which lack coercion resistance, such as STAR-Vote [13], exhibit much higher success rates than systems that address coercion, such as Prêt à Voter [58] and Scantegrity II [24] (93% versus 60% and 50% respectively). This disparity has motivated other coercion-resistant strategies with less intricate tasks such as deniable re-voting [6, 73], where voters only receive a single voting credential and perform the same vote casting process to override their previous vote. However, this strategy is not only vulnerable to domestic coercion—the primary source of coercion in our findings—who can demand the voting credential from the voter but from last-minute coercion. The 83% success rate achieved by TRIP shows promise in narrowing the success gap between coercion-resistant and non-coercion-resistant systems, though this gap remains an important usability challenge for future work.

The Impact of Security Vigilance on User Experience. Our findings show an interesting tradeoff between user experience and security vigilance. Participants from group SF typically rated system usability and user experience lower than group F, despite the only change being an educational brief in the instructional video about the rare possibility of a compromised kiosk. However, this increase in discomfort resulted in a statistically significant improvement in the detection rate of a malicious kiosk in group SM compared to group M (47% vs. 10%), and with no increase in false positive rates. Meanwhile, twice as many participants in SM compared to M liked the system's security the most. These observations confirm that finding the right balance between security education and user comfort is an important challenge.

Limitations. This study has important limitations: it did not evaluate long-term aspects, such as whether voters can distinguish their real credential from their fake ones after months or years. Such a study would be longitudinal in nature, requiring participants to participate over an extended period, and we know of no deployed voting system of this kind that could facilitate such a study. The system's accessibility to voters with disabilities is important but beyond the scope of this study. In the future, a thorough design for accessibility and corresponding study should include voters with disabilities, which would likely yield further insights. Although we aim for diversity, the study being conducted in a single geographical location may affect the generalizability of our findings. Finally, the study did not replicate a government office setting, which might influence differences in participants' views and behaviors. Similarly, the study does not replicate circumstances where a voter is truly under coercion.

Chapter 9. Study Results

Groups	(C)ontrol	(F)ake Creds.	(M)alicious Kiosk	SF	SM
Rating	System Approval Rating				
Positive	73%	63%	60%	56%	53%
Neutral	13%	20%	3%	17%	23%
Negative	13%	17%	37%	27%	23%
	System Usability Scale				
Score	69.6	70.4	69.9	67.3	62.7
SD	18.6	18.6	17.4	19.8	21.9
N	29	28	29	30	29
Percentile	55.1	57.8	56.1	47.8	35.0
Usability	69.5	69.8	68.4	67.7	62.5
Learnability	69.8	73.2	75.9	65.8	63.4
Scale	User Experience Questionnaire (Score vs. Benchmark)				
Attractiveness	↑	↑	↓	↓	↓↓
Perspiciuity	↑	↓	↓	↓	↓↓
Efficiency	↑↑	↑↑	↑	↓	↓
Dependability	↓	↑	↑	↓	↓↓
Stimulation	↑↑	↑↑	↑	↑	↓
Novelty	↑	↑↑	↑↑	↑↑	↑
Category	Most Liked (%)				
Ease of Use	20	27	27	43	17
Instructions	27	20	13	13	27
Remote Voting	23	10	20	10	13
Security	3	20	10	17	23
QR Codes	7	13	17	3	10
Other (Positive)	3	10	10	7	10
Other	17	0	3	7	0
Category	Most Disliked (%)				
Process Complexity	27	23	30	23	23
Credential Handling	20	23	7	17	30
Confusion	7	17	17	13	17
Security / Coercion	7	10	23	13	7
In-Person Reg.	7	3	7	0	3
(None)	17	13	10	23	10
Other	17	10	7	10	10
Types	Use Errors (# Events)				
Tore Receipt	3	1	0	0	1
Almost Tear	3	4	0	1	0
Envelope Pick	1	2	0	1	1
Discarded Real Cred.	0	0	0	0	1
Activate Difficulty	3	5	6	5	10
Total (48)	10	12	6	7	13
Levels	Use Errors (# Events)				
Mistakes	10	7	5	7	11
Violations	0	5	1	0	2
Types	Use Errors (# Required Facilitator Events)				
Kiosk	4	1	0	0	2
Activation	2	3	3	2	2
Study-Wide	0	0	0	0	1
Reporting	Kiosk Reporting Rate				
Facilitator	0%	0%	10%	0%	47%
Survey	10%	7%	20%	7%	57%

Table 9.3: Usability Results Overview. The top section up to “Like Least” represents participants’ perception of system usability and user experience. The middle section represents their use errors and facilitator interventions during the study. The final distinct table represents the kiosk reporting rate. ↑↑ is top 25% of benchmark studies, ↑ is top 50%, ↓ is bottom 50% and ↓↓ is bottom 25%.

9.4 Key Takeaways and Limitations

Group	Type	Sample Size	Attractiveness	Perceptibility	Efficiency	Dependability	Stimulation	Novelty
C	UEQ	27	1.21 (1.13)	1.25 (1.59)	1.51 (0.77)	0.98 (0.85)	1.37 (0.87)	1.10 (0.64)
C	PKI	27	1.81 (1.70)	2.04 (1.34)	2.37 (1.01)	2.41 (1.10)	1.63 (2.40)	1.63 (2.40)
F	UEQ	27	1.49 (1.56)	1.13 (1.84)	1.59 (1.69)	1.30 (1.35)	1.38 (1.66)	1.44 (1.09)
F	PKI	27	1.56 (2.64)	2.00 (1.23)	2.63 (1.01)	2.48 (1.11)	1.00 (3.00)	1.15 (2.98)
M	UEQ	27	1.16 (1.33)	0.93 (1.96)	1.48 (1.26)	1.25 (1.12)	1.28 (1.11)	1.40 (1.12)
M	PKI	27	1.63 (1.78)	2.30 (0.60)	2.67 (0.54)	2.48 (0.64)	1.15 (1.90)	1.22 (2.03)
SF	UEQ	24	1.06 (1.73)	1.06 (1.73)	1.04 (1.30)	1.06 (1.33)	1.08 (1.44)	1.48 (0.86)
SF	PKI	24	1.92 (1.38)	2.46 (0.87)	2.75 (0.72)	2.79 (0.26)	0.96 (3.26)	1.42 (2.77)
SM	UEQ	25	0.55 (1.36)	0.28 (1.54)	0.66 (1.67)	0.49 (1.01)	0.78 (1.14)	0.81 (1.01)
SM	PKI	25	1.76 (1.61)	2.32 (0.64)	2.76 (0.44)	2.60 (0.67)	1.08 (2.99)	1.00 (2.42)

Table 9.4: **UEQ Scale Scores.** This table presents the UEQ and UEQ PKI mean scores (and variance) for each scale, categorized by group.

Quiz Attempts	1	2	3	Continue (Missing Correct Options)	Continue (Incorrect Option)	Count
Real Credential Steps	65	21	1	0	3	90
Real Credential Usage	148	2	0	0	0	150
Test Credential Usage	84	18	4	7	7	120
Distinguish Credentials	91	1	0	0	0	92

Table 9.5: **Quiz Results.** This table represents the quiz attempts; “Continue” signifies that these participants could not pass the quiz after the third attempt. “Missing Correct Options” indicates that these participants only selected correct answers but did not simultaneously choose all correct answers. “Incorrect Option” indicates that these participants selected the incorrect option in one of the quiz attempts and could not rectify it by the third attempt. §8.3 discusses participants’ assignment to quizzes.

Chapter 9. Study Results

Differentiator	Envelope Symbol	Number	Scribble	Symbol	Text	Type Change	Count
Explicit	0	0	0	0	30	2	32
Implicit	0	0	0	5	4	10	19
Indistinguishable	5	1	10	8	9	4	37
Count	5	1	10	13	43	16	88

Table 9.6: **Credential Markings.** Type of markings that participants used to differentiate their fake credentials from their real credential. “Type Change” signifies participants use distinct types, “Scribble” includes signatures, and “Envelope Symbol” indicates participants used no markings, instead relying on distinct envelope symbols. Four sets of envelopes are unaccounted for.

Conclusion and Future Work **Part IV**

This dissertation aimed to achieve three goals that are often viewed as at odds in e-voting systems: coercion resistance, end-to-end verifiability, and usability. The result is VoteGral, a concrete system that realizes fake-credential-based coercion resistance without trusted voter hardware at registration and with a verifiable, practical tally that scales to large electorates. By combining a user-studied, paper-based registration protocol (TRIP) with a tallying scheme (VLT) that eliminates the quadratic bottlenecks of JCJ-style filtering, we show that online voting need not trade away usability or practicality to achieve end-to-end transparency and coercion-resistance.

TRIP provides, to our knowledge, the first concrete, systematically user-studied realization of real and fake credentials that preserves individual verifiability without trusted hardware. Its core mechanism is an interactive zero-knowledge proof materialized on paper: real-credential transcripts are sound, while fake-credential transcripts are deliberately unsound. The two are structurally indistinguishable once printed; only the in-booth order of operations (commit-then-challenge vs. challenge-then-commit) tells the voter which is real. Outside the booth, no transferable evidence exists to distinguish real from fake, giving the voter plausible deniability against a coercer. TRIP also strengthens dispute resolution: failed attempts by an integrity adversary result in non-repudiable cryptographic evidence of misbehavior. Motivated by deployment realities, we introduced iterative and strong iterative individual verifiability, showing that the adversary’s probability of undetected manipulation decays rapidly at scale.

VLT addresses the principal performance and robustness limitations of JCJ-style tallying. Given TRIP’s credential construction, the ledger enforces at acceptance that each ballot is signed by a registrar-issued credential ($c_{pk} \in \mathbb{L}_C$ or $c_{pk} \in \mathbb{L}_P$ for party ballots) and that each credential contributes at most one accepted ballot, mitigating ballot flooding before any expensive cryptographic steps. VLT also introduces the notion of standing votes: delegations chosen at registration are counted as multiplicities on the party’s single ballot, and per-party delegation counts are publicly reported, providing both auditability for delegating voters and coercion-evidence.

A system is only secure if voters can use it. In a main study with 150 diverse participants, 83% successfully completed registration and voting. Among the 120 exposed to fake credentials, 96% understood their purpose. Participants expressed higher trust in online voting with in-person registration than in fully remote registration and voting, supporting our design choice. On the performance side, our prototype tallies 1,000,000 ballots in approximately 14 hours on a 128-core, 256 GB RAM machine, comparable to state-of-the-art verifiable systems and orders of magnitude faster than prior JCJ-style designs.

Limitations. VoteGral of course has several limitations. First, side-channel attacks inside the booth (timing, sound) are out of scope; we discuss mitigations in §6.6.2. Second, small electorates reduce statistical uncertainty, making tally-induced leakage more plausible; tally-hiding can help mitigate. Third, while VoteGral is compatible with cast-as-intended checks (e.g., Be-

naloh challenge), our evaluation did not include such a mechanism, which remains important; we discuss the ability for voters to have long-term verifiability through a voting history in §7.6.3. Finally, TRIP’s public $\mathbb{L}_R$ allows for impersonation detection but creates a tension with “surprise” coercion; we discuss policy mitigations (credential expiration and re-registration cover) in §6.1.4.

Future Work. We see several directions to pursue as future work.

- **Usability and accessibility.** Larger, and more longitudinal studies can help quantify learning and retention (e.g., real/fake recall over months), and improve on device-guided recovery for device/interface-induced errors.
- **Performance at national scale.** Parallel implementations of shuffles and tag computation, batch verification, GPU acceleration, and proof aggregation constitute some steps to reduce end-to-end latency and produce short public transcripts.
- **Post-quantum instantiations.** Migrating cryptographic primitives to post-quantum-secure primitives and evaluating the performance cost relative to the current instantiation.
- **Credential management on device.** On-device credential storage that ensures plausible deniability in the number of credentials stored on the device (e.g., a voter can appear compliant by stating that there are X credentials on the device but in reality they have $Y > X$ credentials).

Bibliography

- [1] Number of registered voters U.S. 2024.
- [2] Voter FAQs | U.S. Election Assistance Commission.
- [3] Election Administration and Voting Survey (EAVS) 2024 Comprehensive Report, June 2025.
- [4] Claudia Z Acemyan, Philip Kortum, Michael D Byrne, and Dan S Wallach. Usability of Voter Verifiable, End-to-end Voting Systems: Baseline Data for Helios, Prêt à Voter, and Scantegrity II. 2(3), 2014.
- [5] Claudia Ziegler Acemyan, Philip Kortum, Michael D. Byrne, and Dan S. Wallach. Summative Usability Assessments of STAR-Vote: A Cryptographically Secure e2e Voting System That Has Been Empirically Proven to Be Easy to Use. *Human Factors*, 64(5):866–889, August 2022.
- [6] Dirk Achenbach, Carmen Kempka, Bernhard Löwe, and Jörn Müller-Quade. Improved Coercion-Resistant Electronic Elections through Deniable Re-Voting. In *USENIX Journal of Election Technology and Systems*, volume 3 of 2, pages 26–45. USENIX Association, 2015.
- [7] Ben Adida. Helios: Web-based Open-Audit Voting. In *USENIX Security Symposium*, volume 17, pages 335–348. USENIX Association, 2008.
- [8] Andrew W. Appel, Richard A. DeMillo, and Philip B. Stark. Ballot-Marking Devices Cannot Ensure the Will of the Voters. *Election Law Journal: Rules, Politics, and Policy*, 19(3):432–450, September 2020.
- [9] Sara Atske. Pew Research Center - The voting experience in 2020, November 2020.
- [10] James Austgen, Andrés Fábrega, Sarah Allen, Kushal Babel, Mahimna Kelkar, and Ari Juels. DAO Decentralization: Voting-Bloc Entropy, Bribery, and Dark DAOs, November 2023.
- [11] Aaron Bangor, Philip Kortum, and James Miller. Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies*, 4(3):114–123, 2009.

Bibliography

- [12] Stephanie Bayer and Jens Groth. Efficient Zero-Knowledge Argument for Correctness of a Shuffle. In *Advances in Cryptology – EUROCRYPT 2012*, volume 7237, pages 263–280. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012.
- [13] Susan Bell, Josh Benaloh, Michael D. Byrne, Dana Debeauvoir, Bryce Eakin, Philip Kortum, Neal McBurnett, Olivier Pereira, Philip B. Stark, Dan S. Wallach, Gail Fisher, Julian Montoya, Michelle Parker, and Michael Winn. STAR-Vote: A Secure, Transparent, Auditable, and Reliable Voting System. 2013.
- [14] Josh Benaloh. *Verifiable Secret-Ballot Elections*. Ph.D. Thesis, Yale University, 1987.
- [15] Josh Benaloh. Ballot Casting Assurance via Voter-Initiated Poll Station Auditing. In *2007 USENIX/ACCURATE Electronic Voting Technology Workshop*, Boston, MA, August 2007.
- [16] Josh Benaloh. Rethinking Voter Coercion: The Realities Imposed by Technology. *USENIX Journal of Election Technology and Systems*, 1(1), 2013.
- [17] Josh Benaloh and Dwight Tuinstra. Receipt-free secret-ballot elections. In *Proceedings of the 26th annual ACM symposium on Theory of Computing*, STOC '94, pages 544–553, May 1994.
- [18] David Bernhard, Véronique Cortier, Pierrick Gaudry, Mathieu Turuani, and Bogdan Warinschi. Verifiability Analysis of CHVote, 2018.
- [19] Matthew Bernhard. Risk-limiting audits: A practical systematization of knowledge. In *Proceedings of the Sixth International Joint Conference on Electronic Voting E-Vote ID 2021*, pages 162–177, Virtual Event, 2021. Springer LNCS.
- [20] Matthew Bernhard, Allison McDonald, Henry Meng, Jensen Hwa, Nakul Bajaj, Kevin Chang, and J. Alex Halderman. Can Voters Detect Malicious Manipulation of Ballot Marking Devices? In *2020 IEEE Symposium on Security and Privacy*, pages 679–694. IEEE, May 2020.
- [21] John Brooke. SUS: A ‘Quick and Dirty’ Usability Scale. In *Usability Evaluation In Industry*. CRC Press, 1996.
- [22] Alan S. Brown, Elisabeth Bracken, Sandy Zoccoli, and King Douglas. Generating and remembering passwords. *Applied Cognitive Psychology*, 18(6):641–651, 2004.
- [23] Eleno Castro and Randy Kotti. Saving Democracy: Reducing Gang Influence on Political Elections in El Salvador. Master’s thesis, Harvard University, 2022.
- [24] David Chaum, Richard Carback, Jeremy Clark, Aleksander Essex, S. Popoveniuc, Rivest L. Rivest, Peter Y. A. Ryan, Emily Shen, and Alan T. Sherman. Scantegrity II: End-to-End Verifiability by Voters of Optical Scan Elections Through Confirmation Codes. *IEEE Transactions on Information Forensics and Security*, 4(4):611–627, December 2009.

-
- [25] David Chaum and Torben Pryds Pedersen. Wallet Databases with Observers. In *Advances in Cryptology — CRYPTO' 92*, pages 89–105, Berlin, Heidelberg, 1993.
- [26] Michael R. Clarkson, Stephen Chong, and Andrew C. Myers. Civitas: Toward a Secure Voting System. In *2008 IEEE Symposium on Security and Privacy*, pages 354–368. IEEE, May 2008.
- [27] Véronique Cortier, Alexandre Debant, Pierrick Gaudry, and Stéphane Glondou. Belenios with Cast as Intended. In Aleksander Essex, Shin'ichiro Matsuo, Oksana Kulyk, Lewis Gudgeon, Arian Klages-Mundt, Daniel Perez, Sam Werner, Andrea Bracciali, and Geoff Goodell, editors, *Financial Cryptography and Data Security. FC 2023 International Workshops*, Lecture Notes in Computer Science, pages 3–18, Cham, 2024. Springer Nature Switzerland.
- [28] Ronald Cramer, Ivan Damgaard, and Berry Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In *Annual International Cryptology Conference*, pages 174–187. Springer, 1994.
- [29] Anders Dalskov. anderspkd/groth-shuffle: Bayer-Groth shuffle, October 2021.
- [30] Al Danial. Counting Lines of Code. <http://cloc.sourceforge.net/>.
- [31] DEDIS. DEDIS Advanced Crypto Library for Go, 2025.
- [32] Rachna Dhamija and Adrian Perrig. Deja Vu: A User Study Using Images for Authentication. 2000.
- [33] Verena Distler, Marie-Laure Zollinger, Carine Lallemand, Peter B. Roenne, Peter Y. A. Ryan, and Vincent Koenig. Security - Visible, Yet Unseen? In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, pages 1–13, New York, NY, USA, May 2019. Association for Computing Machinery.
- [34] E-Estonia. e-Democracy & open data - e-Estonia, 2025. Accessed August 2025.
- [35] Piret Ehin, Mihkel Solvak, Jan Willemson, and Priit Vinkel. Internet voting in Estonia 2005–2019: Evidence from eleven elections. *Government Information Quarterly*, 39(4):101718, October 2022.
- [36] T. Elgamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, July 1985.
- [37] Ehsan Estaji, Thomas Haines, Kristian Gjøsteen, Peter B. Rønne, Peter Y. A. Ryan, and Najmeh Soroush. Revisiting Practical and Usable Coercion-Resistant Remote E-Voting. In *E-Vote-ID 2020: Electronic Voting*, pages 50–66, 2020.
- [38] Christopher Famighetti. Long voting lines: Explained. Technical report, Brennan Center for Justice, 2016.

Bibliography

- [39] Robert Faulkner. What Is TEMPEST? | Cybersecurity RF & Electromagnetic Shielding, April 2022.
- [40] Christian Feier, Stephan Neumann, and Melanie Volkamer. Coercion-Resistant Internet Voting in Practice. In *Informatik 2014*, 2014.
- [41] Dragan Filipovich, Miguel Niño-Zarazúa, and Alma Santillán Hernández. Voter coercion and pro-poor redistribution in rural Mexico. WIDER Working Paper 2021/141, The United Nations University World Institute for Development Economics Research (UNU-WIDER), July 2021.
- [42] Caitriona Fitzgerald, Pamela Smith, and Susannah Goodman. The Secret Ballot At Risk: Recommendations for Protecting Democracy. August 2016.
- [43] David Froelicher, Patricia Egger, João Sá Sousa, Jean Louis Raisaro, Zhicong Huang, Christian Vincent Mouchet, Bryan Ford, and Jean-Pierre Hubaux. UnLynx: A Decentralized System for Privacy-Conscious Data Sharing. In *Proceedings on Privacy Enhancing Technologies*, volume 4, pages 152–170, 2017.
- [44] Timothy Frye, Ora John Reuter, and David Szakonyi. Hitting Them With Carrots: Voter Intimidation and Vote Buying in Russia. *British Journal of Political Science*, 49(3):857–881, July 2019.
- [45] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Secure Distributed Key Generation for Discrete-Log Based Cryptosystems. *Journal of Cryptology*, 20(1):51–83, January 2007.
- [46] Ezequiel Gonzalez-Ocantos, Chad Kiewiet de Jonge, Carlos Meléndez, David Nickerson, and Javier Osorio. Carrots and sticks: Experimental evidence of vote-buying and voter intimidation in Guatemala. *Journal of Peace Research*, 57(1):46–61, January 2020.
- [47] A Barcode Scanning/Encoding Library for Go, February 2021.
- [48] Michael Graff and Nick Ochsner. ‘This Smacks of Something Gone Awry’: A True Tale of Absentee Vote Fraud, November 2021.
- [49] Guruchetan S. Grewal, Mark D. Ryan, Sergiu Bursuc, and Peter Y.A. Ryan. Caveat Coercitor: Coercion-Evidence in Electronic Voting. In *2013 IEEE Symposium on Security and Privacy*, pages 367–381, May 2013.
- [50] Jens Groth. A Verifiable Secret Shuffle of Homomorphic Encryptions. *Journal of Cryptology*, 23(4):546–579, October 2010.
- [51] Sufian Hameed, Syed Atyab Hussain, and Sohail Hussain Ali. SafePass: Authentication under duress for ATM transactions. In *2013 2nd National Conference on Information Assurance (NCIA)*, pages 1–5, December 2013.

-
- [52] Andreas Hinderks, Martin Schrepp, Francisco José Domínguez Mayo, María José Escalona, and Jörg Thomaschewski. Developing a UX KPI based on the user experience questionnaire. *Computer Standards & Interfaces*, 65:38–44, July 2019.
 - [53] Martin Hirt and Kazue Sako. Efficient Receipt-Free Voting Based on Homomorphic Encryption. In *Advances in Cryptology — EUROCRYPT 2000*, 2000.
 - [54] ID. E-voting and e-elections, 2025. Accessed August 2025.
 - [55] Markus Jakobsson and Ari Juels. Mix and Match: Secure Function Evaluation via Cipher-texts. In *Advances in Cryptology — ASIACRYPT 2000*, pages 162–177, 2000.
 - [56] Ari Juels, Dario Catalano, and Markus Jakobsson. Coercion-Resistant Electronic Elections. In *Towards Trustworthy Elections: New Directions in Electronic Voting*, pages 37–63. Berlin, Heidelberg, 2010.
 - [57] Kurt Jung. GoFPDF document generator, April 2024.
 - [58] Dalia Khader, Qiang Tang, and Peter YA Ryan. Proving prêt à voter receipt free using computational security models. In *2013 Electronic Voting Technology Workshop*, 2013.
 - [59] Jesper Kjeldskov and Mikael B. Skov. Studying Usability In Sitro: Simulating Real World Phenomena in Controlled Environments. *International Journal of Human–Computer Interaction*, 22(1-2):7–36, April 2007.
 - [60] Christoph Carl Kling, Jérôme Kunegis, Heinrich Hartmann, Markus Strohmaier, and Steffen Staab. Voting Behaviour and Power in Online Democracy: A Study of Liquid-Feedback in Germany’s Pirate Party. In Meeyoung Cha, Cecilia Mascolo, and Christian Sandvig, editors, *Proceedings of the Ninth International Conference on Web and Social Media, ICWSM 2015, University of Oxford, Oxford, UK, May 26-29, 2015*, pages 208–217. AAAI Press, 2015.
 - [61] Reto Koenig, Rolf Haenni, and Stephan Fischli. Preventing Board Flooding Attacks in Coercion-Resistant Electronic Voting Schemes. In *Future Challenges in Security and Privacy for Academia and Industry*, pages 116–127, Berlin, Heidelberg, 2011.
 - [62] Kristjan Kripts and Jan Willemson. On Practical Aspects of Coercion-Resistant Remote Voting Systems. In *E-Vote-ID 2019: Electronic Voting*, pages 216–232, 2019.
 - [63] Taisya Krivoruchko. Robust Coercion-Resistant Registration for Remote E-voting. In *Proceedings of the IAVoSS Workshop on Trustworthy Elections*, 2007.
 - [64] Oksana Kulyk and Stephan Neumann. Human Factors in Coercion Resistant Internet Voting—A Review of Existing Solutions and Open Challenges. In *Proceedings of the Fifth International Joint Conference on Electronic Voting*, page 189. TalTech press, 2020.
 - [65] The Kyber Cryptography Library, 2010 – 2025.

Bibliography

- [66] Ralf Küsters, Julian Liedtke, Johannes Müller, Daniel Rausch, and Andreas Vogt. Ordinos: A Verifiable Tally-Hiding E-Voting System. In *2020 IEEE European Symposium on Security and Privacy*. IEEE, 2020.
- [67] Bettina Laugwitz, Theo Held, and Martin Schrepp. Construction and evaluation of a user experience questionnaire. In *Symposium of the Austrian HCI and usability engineering group*, pages 63–76. Springer, 2008.
- [68] Corentin Lavaud, Robin Gerzaguët, Matthieu Gautier, Olivier Berder, Erwan Nogues, and Stephane Molton. Whispering Devices: A Survey on How Side-channels Lead to Compromised Information. *Journal of Hardware and Systems Security*, 5(2):143–168, June 2021.
- [69] James R. Lewis. The System Usability Scale: Past, Present, and Future. *International Journal of Human–Computer Interaction*, 34(7):577–590, July 2018.
- [70] Rensis Likert. A technique for the measurement of attitudes. *Archives of Psychology*, 22 140:55–55, 1932.
- [71] Mark Lindeman and Philip B. Stark. A Gentle Introduction to Risk-Limiting Audits. *IEEE Security & Privacy*, 10(5):42–49, September 2012.
- [72] Simon Luechinger, Myra Rosinger, and Alois Stutzer. The impact of postal voting on participation: Evidence for switzerland. *Swiss Political Science Review*, 13(2):167–202, 2007.
- [73] Wouter Lueks, Iñigo Querejeta-Azurmendi, and Carmela Troncoso. VoteAgain: A scalable coercion-resistant voting system. In *29th USENIX Security Symposium*, pages 1553–1570. USENIX Association, 2020.
- [74] Karola Marky, Oksana Kulyk, Karen Renaud, and Melanie Volkamer. What Did I Really Vote For? On the Usability of Verifiable E-Voting Schemes. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI ’18, pages 1–13, New York, NY, USA, April 2018. Association for Computing Machinery.
- [75] Karola Marky, Martin Schmitz, Felix Lange, and Max Mühlhäuser. Usability of Code Voting Modalities. In *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI EA ’19, pages 1–6, New York, NY, USA, May 2019.
- [76] Richard E. Mayer, Logan Fiorella, and Andrew Stull. Five ways to increase the effectiveness of instructional video. *Educational Technology Research and Development*, 68(3):837–852, June 2020.
- [77] Louis-Henri Merino, Alaleh Azhir, Haoqian Zhang, Simone Colombo, Bernhard Tellenbach, Vero Estrada-Galiñanes, and Bryan Ford. E-vote your conscience: Perceptions of coercion and vote buying, and the usability of fake credentials in online voting. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3478–3496, 2024.

-
- [78] Jeff Meyerson. The Go programming language. *IEEE Software*, 31(5):104–104, 2014.
 - [79] Tal Moran and Moni Naor. Receipt-Free Universally-Verifiable Voting with Everlasting Privacy. In *Advances in Cryptology*, 2006.
 - [80] C. Andrew Neff. A verifiable secret shuffle and its application to e-voting. In *Proceedings of the 8th ACM conference on Computer and Communications Security*, CCS '01, pages 116–125, New York, NY, USA, November 2001. Association for Computing Machinery.
 - [81] André Silva Neto, Matheus Leite, Roberto Araújo, Marcelle Pereira Mota, Nelson Cruz Sampaio Neto, and Jacques Traoré. Usability Considerations For Coercion-Resistant Election Systems. In *Proceedings of the 17th Brazilian Symposium on Human Factors in Computing Systems*, IHC 2018, pages 1–10, New York, NY, USA, October 2018.
 - [82] Stephan Neumann, Christian Feier, Melanie Volkamer, and Reto Koenig. Towards a practical JCJ/Civitas implementation. In *Informatik angepasst an Mensch, Organisation und Umwelt*, 2013.
 - [83] Stephan Neumann and Melanie Volkamer. Civitas and the Real World: Problems and Solutions from a Practical Point of View. In *Seventh Conference on Availability, Reliability and Security*, 2012.
 - [84] Raymond S. Nickerson. The production and perception of randomness. *Psychological Review*, 109(2):330–357, April 2002.
 - [85] NIST. Usability Performance Benchmarks For the Voluntary Voting System Guidelines. Technical report, NIST, August 2007.
 - [86] NIST. Voluntary Voting System Guidelines Version 2.0. Technical report, February 2021.
 - [87] OpenPrinting. OpenPrinting/cups, April 2024. <https://github.com/OpenPrinting/cups>.
 - [88] Sunoo Park, Michael Specter, Neha Narula, and Ronald L Rivest. Going from bad to worse: from Internet voting to blockchain voting. *Journal of Cybersecurity*, 7(1), February 2021.
 - [89] Indrajeet Patil. Visualizations with statistical details: The `ggstatsplot` approach. *Journal of Open Source Software*, 6(61):3167, 2021.
 - [90] Torben Pryds Pedersen. A Threshold Cryptosystem without a Trusted Party. In Donald W. Davies, editor, *Advances in Cryptology — EUROCRYPT '91*, Lecture Notes in Computer Science, pages 522–526, Berlin, Heidelberg, 1991. Springer.
 - [91] Birgit Pfitzmann. Breaking an efficient anonymous channel. In Alfredo De Santis, editor, *Advances in Cryptology — EUROCRYPT'94*, pages 332–340, Berlin, Heidelberg, 1995. Springer.

Bibliography

- [92] Rolf Ploetzner, Sandra Berney, and Mireille Bétrancourt. When learning from animations is more successful than learning from static pictures: learning the specifics of change. *Instructional Science*, 49(4):497–514, August 2021.
- [93] Gerald V. Post. Using re-voting to reduce the threat of coercion in elections. *Electronic Government, an International Journal*, 7(2):168–182, January 2010.
- [94] Swiss Post. E-Voting Architecture Document. Technical report, February 2024. v1.4.0.
- [95] Iñigo Querejeta-Azurmendi, David Arroyo Guardado, Jorge L. Hernández-Ardieta, and Luis Hernández Encinas. NetVote: A Strict-Coercion Resistance Re-Voting Based Internet Voting Scheme with Linear Filtering. *Mathematics*, 8(9):1618, September 2020.
- [96] Rahul Rathi. Effect of Cambridge Analytica’s Facebook ads on the 2016 US Presidential Election, January 2019.
- [97] Robert W. [R-OH-18 Rep. Ney. H.R.3295 - 107th Congress (2001-2002): Help America Vote Act of 2002, October 2002.
- [98] RFE/RL’s Moldovan Service. Moldovan Police Accuse Pro-Russian Oligarch Of \$39M Vote-Buying Scheme. *Radio Free Europe/Radio Liberty*, October 2024.
- [99] Ronald L Rivest. On the notion of ‘software independence’ in voting systems. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 366(1881):3759–3767, August 2008.
- [100] Gary Robertson. Plea agreements reached by 4 in NC Congress ballot probe, September 2022.
- [101] Horst Rutter. pdfcpu/pdfcpu, April 2024. <https://github.com/pdfcpu/pdfcpu>.
- [102] Peter Y. A. Ryan, David Bismark, James Heather, Steve Schneider, and Zhe Xia. Prêt À Voter: a Voter-Verifiable Voting System. *IEEE Transactions on Information Forensics and Security*, 4(4):662–673, December 2009.
- [103] Peter Y. A. Ryan and Vanessa Teague. Pretty Good Democracy. In Bruce Christianson, James A. Malcolm, Vashek Matyáš, and Michael Roe, editors, *Security Protocols XVII*, Lecture Notes in Computer Science, pages 111–130, Berlin, Heidelberg, 2013. Springer.
- [104] Peter B. Rønne, Arash Atashpendar, Kristian Gjøsteen, and Peter Y. A. Ryan. Coercion-Resistant Voting in Linear Time via Fully Homomorphic Encryption: Towards a Quantum-Safe Scheme. volume 11599, pages 289–298. 2020. arXiv:1901.02560 [quant-ph].
- [105] Kazue Sako and Joe Kilian. Receipt-Free Mix-Type Voting Scheme. In Louis C. Guillou and Jean-Jacques Quisquater, editors, *Advances in Cryptology — EUROCRYPT ’95*, Lecture Notes in Computer Science, pages 393–403, Berlin, Heidelberg, 1995. Springer.

-
- [106] Jeff Sauro. *A Practical Guide to the System Usability Scale*. Measuring Usability LLC, 2011.
- [107] Sinem Sav, Apostolos Pyrgelis, Juan Ramón Troncoso-Pastoriza, David Froelicher, Jean-Philippe Bossuat, Joao Sa Sousa, and Jean-Pierre Hubaux. POSEIDON: Privacy-Preserving Federated Neural Network Learning. In *Proceedings 2021 Network and Distributed System Security Symposium*, Virtual, 2021. Internet Society.
- [108] Stuart Schechter, Serge Egelman, and Robert W. Reeder. It's not what you know, but who you know: a social approach to last-resort authentication. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '09, pages 1983–1992, New York, NY, USA, April 2009. ACM.
- [109] Claus Schnorr. Efficient signature generation by smart cards. *Journal of Cryptology*, 4(3):161–174, January 1991.
- [110] Martin Schrepp. User Experience Questionnaire Handbook Version 10, May 2023.
- [111] Uwe Serdult, Micha Germann, Fernando Mendez, Alicia Portenier, and Christoph Wellig. Fifteen years of internet voting in Switzerland [History, Governance and Use]. In *2015 Second International Conference on eDemocracy & eGovernment (ICEDEG)*, pages 126–132, Quito, Ecuador, April 2015. IEEE.
- [112] Sky News. US election: Bomb threats and printer glitches disrupt voting in battleground states. *Sky News*, November 2024.
- [113] Warren D Smith. New cryptographic election protocol with best-known theoretical properties. Technical report, Center for Range Voting, September 2005.
- [114] Drew Springall, Travis Finkenauer, Zakir Durumeric, Jason Kitcat, Harri Hursti, Margaret MacAlpine, and J. Alex Halderman. Security Analysis of the Estonian Internet Voting System. In *ACM SIGSAC Conference on Computer and Communications Security*, 2014.
- [115] Oliver Spycher, Reto Koenig, Rolf Haenni, and Michael Schläpfer. A New Approach towards Coercion-Resistant Remote E-Voting in Linear Time. In *Proceedings of the 15th International Conference on Financial Cryptography and Data Security*, pages 182–189, 2011.
- [116] Philip B. Stark. There is no Reliable Way to Detect Hacked Ballot-Marking Devices. *Election Law Journal Rules, Politics and Policy*, 19(3), 2019.
- [117] Swiss Post. Protocol of the Swiss Post Voting System: Computational Proof of Complete Verifiability and Privacy. Technical Report 1.0.0, 2021.
- [118] The Associated Press. Four people plead guilty in North Carolina ballot probe of 2016 and 2018 elections. *NBC News*, September 2022.

Bibliography

- [119] Selen Türkay. The effects of whiteboard animations on retention and subjective experiences when learning advanced physics topics. *Computers & Education*, 98:102–114, July 2016.
- [120] U.S. Alliance for Election Excellence. Voting Time Estimator, 2025. Accessed August 2025.
- [121] King-Hang Wang, Subrota K. Mondal, Ki Chan, and Xiaoheng Xie. A review of contemporary e-voting: Requirements, technology, systems and usability. *Data Science and Pattern Recognition*, 1(1):31–47, 2017.
- [122] Stefan G. Weber, Roberto Araujo, and Johannes Buchmann. On Coercion-Resistant Electronic Elections with Linear Work. In *The Second Conference on Availability, Reliability and Security*, 2007.
- [123] Wendy Weiser. How to Protect the 2020 Vote from the Coronavirus | Brennan Center for Justice, March 2020.
- [124] Danielle Wiener-Bronner. Yes, There Are Elections in North Korea and Here’s How They Work, March 2014.
- [125] William Wilder. *Voter Suppression in 2020*. Brennan Center for Justice at New York University School of Law, 2021.
- [126] Margarita Zavadskaya. Why Does the Kremlin Bother Holding Sham Elections?, March 2024.

Louis-Henri Merino

Email: louishenrimerino@protonmail.com

Education

Swiss Federal Institute of Technology Lausanne (EPFL)

Sep 2019 – Sep 2025

Ph.D. Student in Computer and Communication Sciences

- Awarded **Swiss Cyber Defense Fellowship** by Armasuisse - Cyber Defense Campus (2022)
- Awarded **Fulbright Scholarship** by the Swiss Federal Commission & U.S. Department of State (2019)

University of Maryland College Park – Honors College (UMD)

Aug 2015 – May 2019

Bachelor of Science in Computer Science

Minor in Cybersecurity & Business Innovation and Entrepreneurship

- GPA: 3.8/4.0
- Awarded **Banneker Key Scholarship**, the university's most prestigious merit-based scholarship.

Employment History

Decentralized and Distributed Systems Lab @ EPFL

Sep 2019 – Sep 2025

Supervisor: [Prof. Bryan Ford](#)

- Designed an [online voting system resistant to coercion](#), formally proved that it satisfies coercion-resistance and conducted a [user study](#) with 150 participants to evaluate its real-world viability.
- Contributed to a [blockchain architecture](#) that mitigates front-running attacks on a per-transaction basis.

Chainlink Labs

June 2024 – Nov 2024

- Contributed the design of [Smart Value Recapture](#) to enables DeFi applications to recapture the non-toxic Maximal Extractable Value (MEV).

Breakerspace Lab @ UMD

Dec 2018 – Sep 2023

Supervisor: [Prof. Dave Levin](#)

- Contributed to an artificial intelligence tool named [Geneva](#) that manipulates network traffic to evade censors.
- Developed a system for large-scale detection of disrupted network traffic [deployed on Cloudflare](#).

Maryland Cybersecurity Center @ UMD

Aug 2016 – Jun 2020

Supervisor: [Prof. Michel Cukier](#)

- Designed a honeypot system suitable for cloud deployment to gather behavioral insights of cloud attackers, launched it on AWS in 15 regions and analyzed the data for a [paper publication](#).

AstraZeneca: Advanced Analytics Center

May 2016 – Aug 2016

Supervisor: [Dr. Nina Mian](#)

- Developed an application that determined the effectiveness of previously conducted clinical trials by looking at their eligibility criteria and comparing it to large patient databases.

NIH/NIAID National Institute of Allergy and Infectious Diseases

Jun 2014 – Sep 2019

Supervisors: [Dr. Jain Fraser](#) and Dr. Bhaskar Dutta

- Developed [CARD](#), a framework for computation-based analyses of siRNA screen data.

Leadership Activities

École Polytechnique Fédérale de Lausanne

Sep 2019 – Sep 2025

Technologies for Democratic Society (Fall 2019 - Present)

- Contributed to the creation of this course and have been a teaching assistant for the course ever since.

Proof-of-Personhood (Fall 2019 – Present)

- Designed protocols for electronic voting, social media and digital assets applications for a community-centered application; supervised 4-5 bachelor students each semester to implement these protocols.

Vulnerability Assessment of the Swiss Post E-Voting System (Fall 2021, Spring 2022)

- Supervised master students who reviewed the system's specifications and codebase for vulnerabilities.

University of Maryland – College Park

Jan 2017 – May 2019

Applied Cybersecurity Foundations (Spring 2018, Spring 2019)

- Contributed to the development of this two-credit class in January 2018 and helped lead the class.

Applied Cybersecurity Foundations II & Group Project in Cybersecurity (Fall 2018)

- Led weekly lab sessions and taught some lectures; designed an application to distribute lab content to students.

Frappa (Free Application Advising)

May 2019 – Present

Executive Co-Director

- Co-founded and currently directing a nonprofit organization geared towards advising Iranian women applying to graduate schools in the U.S.A. and Canada.

Awards

Cyber Defense Doctoral Fellowship (2022 – 2024): A highly competitive grant providing salary along with research & conference funds to doctoral students in Switzerland working in cybersecurity.

AMIA Best Paper Award (2022): The paper: [BEHRTDAY: Dynamic Mortality Risk Prediction using Time-Variant COVID-19 Patient Specific Trajectories](#)

Applied Network Research Prize (2021): An annual prize awarded by the Internet Research Task Force to recognize the best research in computer networks. Prize awarded based on the “[Come as You Are](#)” paper.

Fulbright Scholarship / Swiss Government Excellence Scholarship (2019): International grant awarded by U.S. Department of State and the Swiss Federal Commission to conduct research on electronic voting systems at EPFL.

Banneker Key Scholarship (2015 – 2019): University of Maryland’s most prestigious merit-based scholarship.

American Chemical Society Poster Presentation (2015): First prize winner of the 20th Annual Research Poster Session for the poster on the RNAi Screen Data Analysis Pipeline.

New York Office of the State Comptroller (2015): Award in recognition of hard work, dedication and success

American Federation of School Administrators (2015): Award in recognition of outstanding leadership.

The Richard Welling Alumni Association in Partnership with the Department of Education of the City of New York (2015): Outstanding Citizenship Service and Leadership in the Student Government Organization.

Publication List

TRIP: Trust-Limited Coercion-Resistant In-Person Voter Registration. *Louis-Henri Merino, Simone Colombo, Rene Reyes, Alaleh Azhir, Shailesh Mishra, Pasindu Tennage, Mohammad Amin Raeisi, Haoqian Zhang, Jeff Allen, Bernhard Tellenbach, Vero Estrada-Galiñanes, Bryan Ford. SOSP 2025.*

E-Vote Your Conscience: Perceptions of Coercion and Vote Buying, and the Usability of Fake Credentials in Online Voting. *Louis-Henri Merino, Alaleh Azhir, Haoqian Zhang, Simone Colombo, Bernhard Tellenbach, Vero Estrada-Galiñanes, Bryan Ford. IEEE S&P 2024.*

Measuring Censorship Passively and Ethically. Ram Sundara Raman, *Louis-Henri Merino*, Kevin Bock, Marwan Fayed, Dave Levin, Nick Sullivan, Luke Valenta. *ACM SIGCOMM '23.*

F3B: A Low-Latency Commit-and-Reveal Architecture to Mitigate Blockchain Front-Running: Haoqian Zhang, *Louis-Henri Merino*, Ziyang Qu, Mahsa Bastankhah, Vero Estrada-Galiñanes, Bryan Ford. *AFT '23.*

Using Shapes of COVID-19 Positive Patient-Specific Trajectories for Mortality Prediction: Alaleh Azhir, Soheila Talebi, *Louis-Henri Merino*, Thomas Lukasiewicz, Edgar Argulian, Jagat Narula, and Borislava Mihaylova, *AMIA. 2022.*

BEHRTDAY: Dynamic Mortality Risk Prediction using Time-Variant COVID-19 Patient Specific Trajectories: Alaleh Azhir, Soheila Talebi, *Louis-Henri Merino*, Yikuan Li, Thomas Lukasiewicz, Edgar Argulian, Jagat Narula, and Borislava Mihaylova, *AMIA. 2022*

Come as You Are: Helping Unmodified Clients Bypass Censorship with Server-Side Evasion. Kevin Bock, George Hughey, *Louis-Henri Merino*, Tania Arya, Daniel Liscinsky, Regina Pogolian, Dave Levin. *Proceedings of the ACM SIGCOMM Conference 2020.*

An Approach for Preventing and Detecting Attacks in the Cloud. *Louis-Henri Merino*, Michel Cukier, *IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC). 2022.*

Interactive Web-based Identification and Visualization of Transcript Shared Sequences. Alaleh Azhir, *Louis-Henri Merino*, David Nauen. *Genomics. 2018.*

An Interactive Web-based Application for Comprehensive Analysis of RNAi-screen Data. Bhaskar Dutta, Alaleh Azhir, *Louis-Henri Merino*, Yongjian Guo, Swetha Revanur, Piyush B Madhamshettiwar, Ronald N Germain, Jennifer A Smith, Kaylene J Simpson, Scott E Martin, Eugen Buehler, Iain D C Fraser. *Nature Communications. 2016.*